Detection & Segmentation

Instructor: Hao Wang

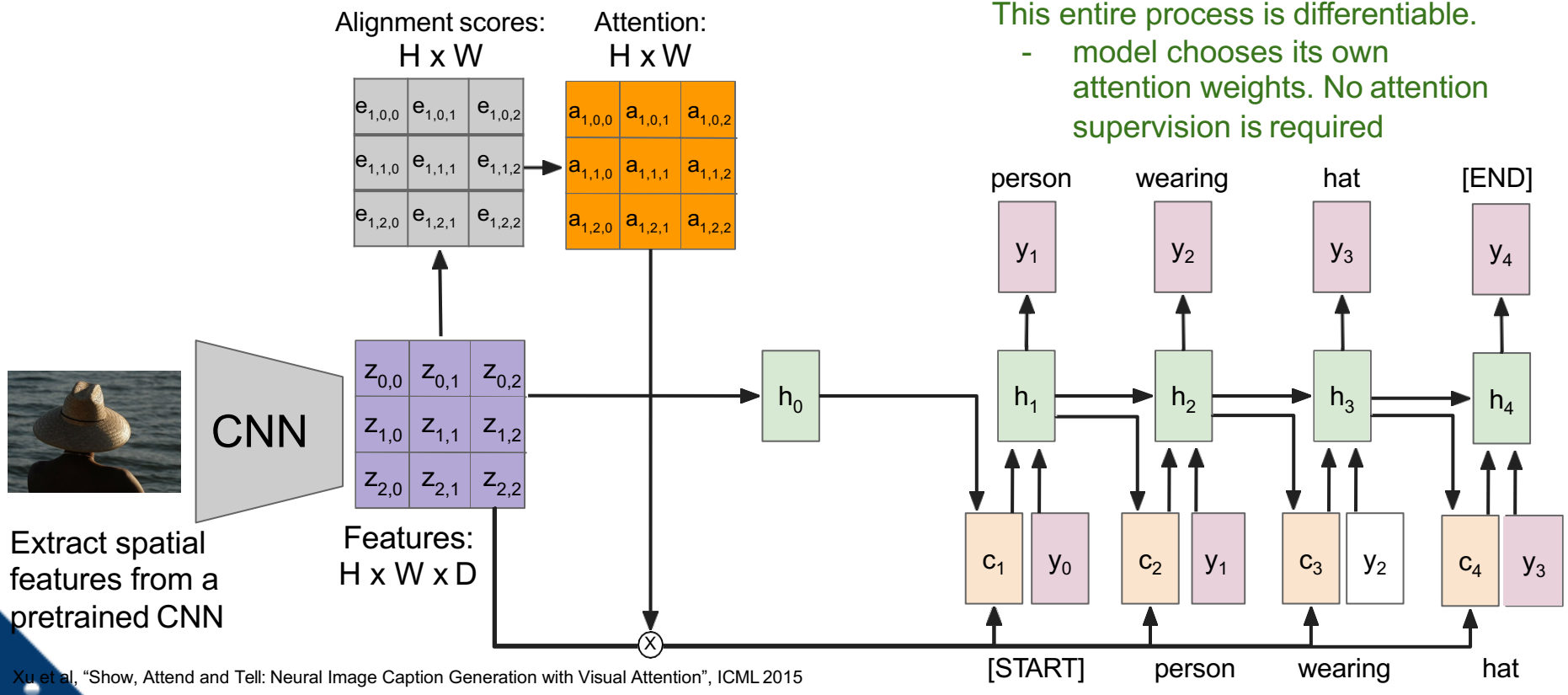
AI Thrust, Information hub



Recap of last week



Image Captioning with RNNs & Attention



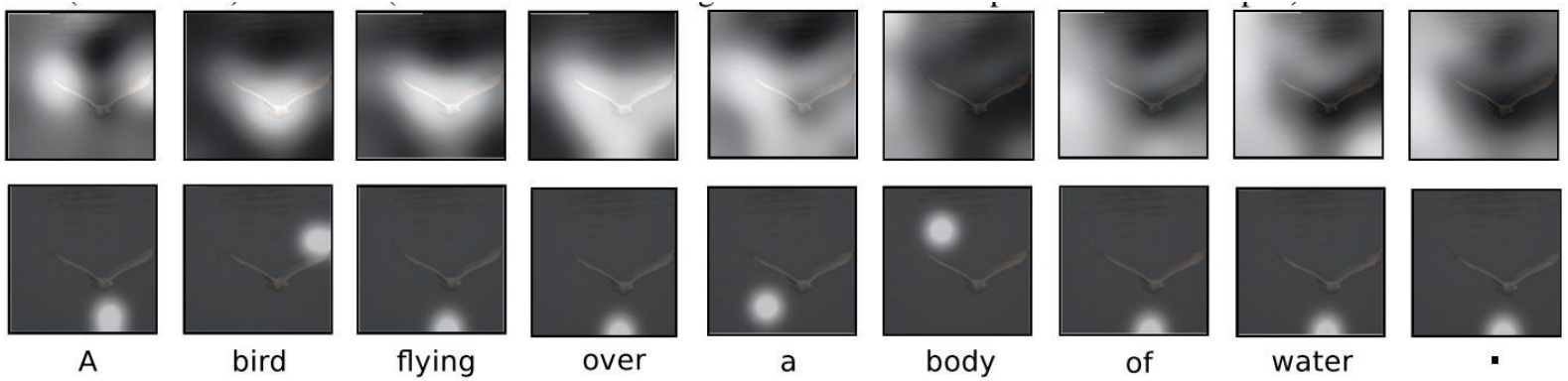
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with Attention

Soft attention



Hard attention
(requires reinforcement learning)



Xu et al, “Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention”, ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Benchio, 2015. Reproduced with permission.



Image Captioning with Attention



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



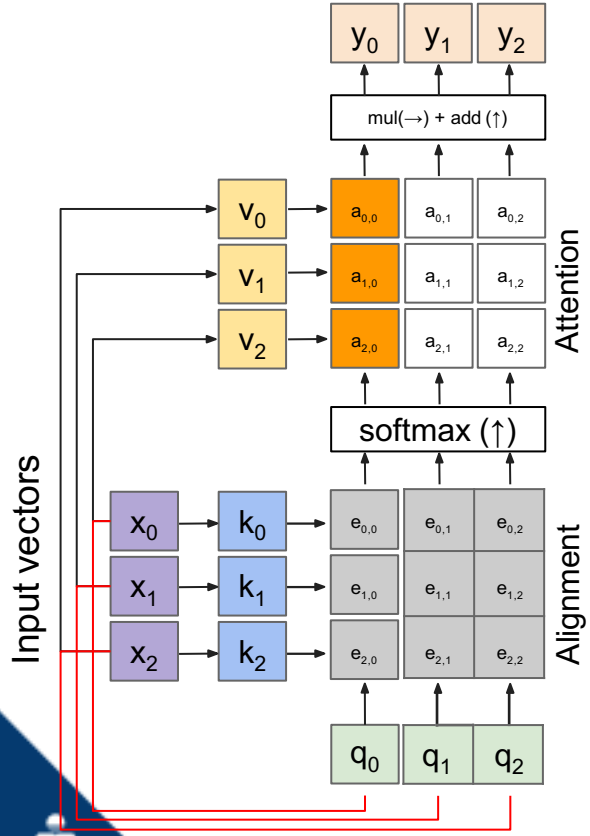
A giraffe standing in a forest with trees in the background.

Xu et al, “Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention”, ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Benchio, 2015. Reproduced with permission.



General attention layer ➡ Self-attention layer



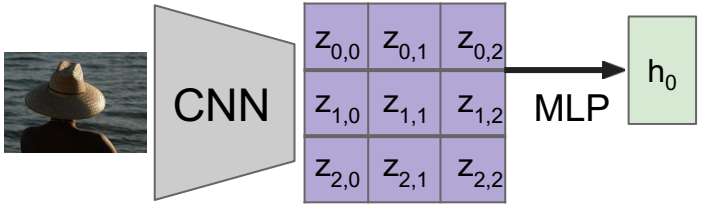
Outputs:
context vectors: **y** (shape: D_v)

Operations:
Key vectors: $\mathbf{k} = \mathbf{x}W_k$
Value vectors: $\mathbf{v} = \mathbf{x}W_v$
Query vectors: $\mathbf{q} = \mathbf{x}W_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

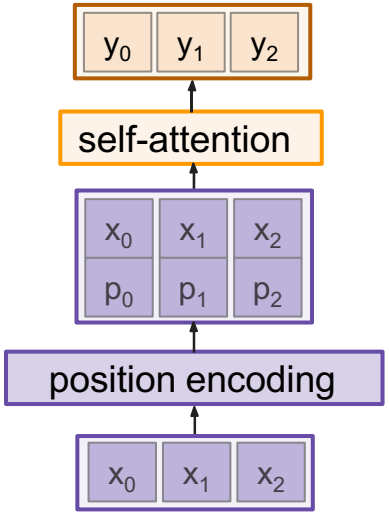
Inputs:
Input vectors: **x** (shape: N x D)
~~Queries: **q** (shape: M x D_k)~~

Recall that the query vector was a function of the input vectors

Encoder: $h_0 = f_W(\mathbf{z})$
where **z** is spatial CNN features
$f_W(.)$ is an MLP



Positional encoding



Concatenate special positional encoding p_j to each input vector x_j

We use a function *pos*: N $\rightarrow R^d$ to process the position j of the vector into a d-dimensional vector

So, $p_j = pos(j)$

Options for *pos*(.)

1. Learn a lookup table:
   - Learn parameters to use for *pos*(t) for t ε [0, T)
   - Lookup table contains T x d parameters.

2. Design a fixed function with the desiderata
   -

$$\mathbf{p(t)} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \vdots \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_d$$

Intuition:

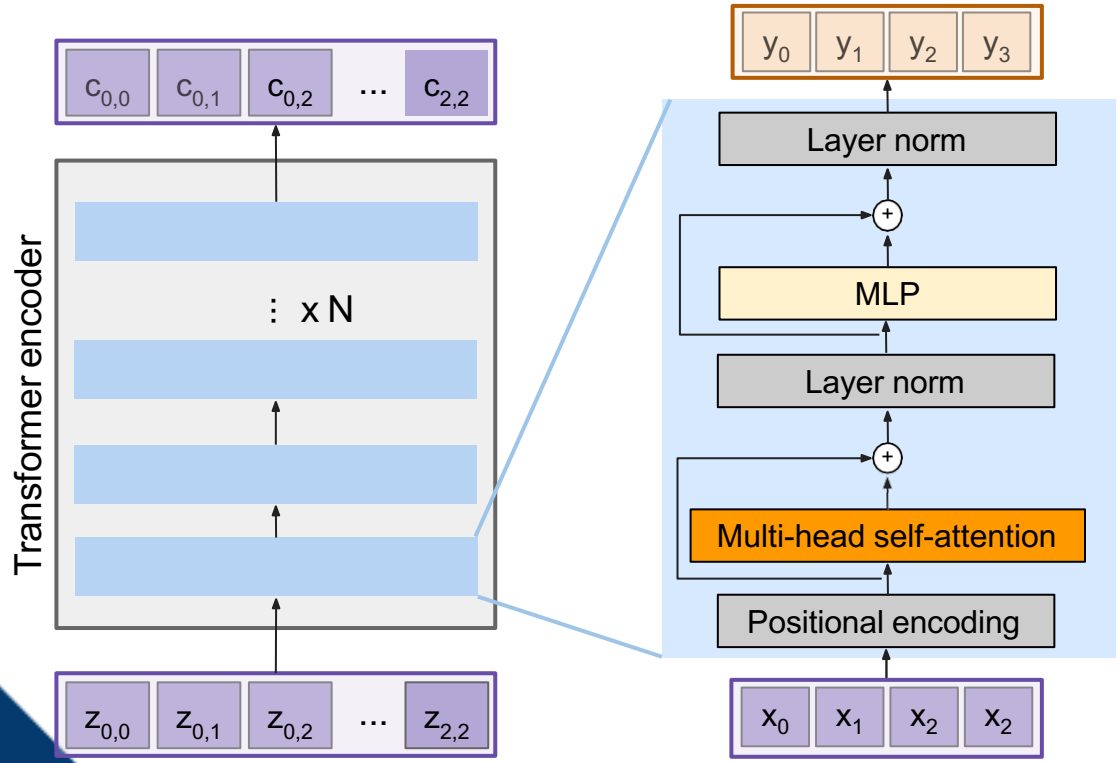
0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

where $\omega_k = \frac{1}{10000^{2k/d}}$

Vaswani et al, "Attention is all you need", NeurIPS 2017



The Transformer encoder block



Transformer Encoder Block:

Inputs: Set of vectors **x**
Outputs: Set of vectors **y**

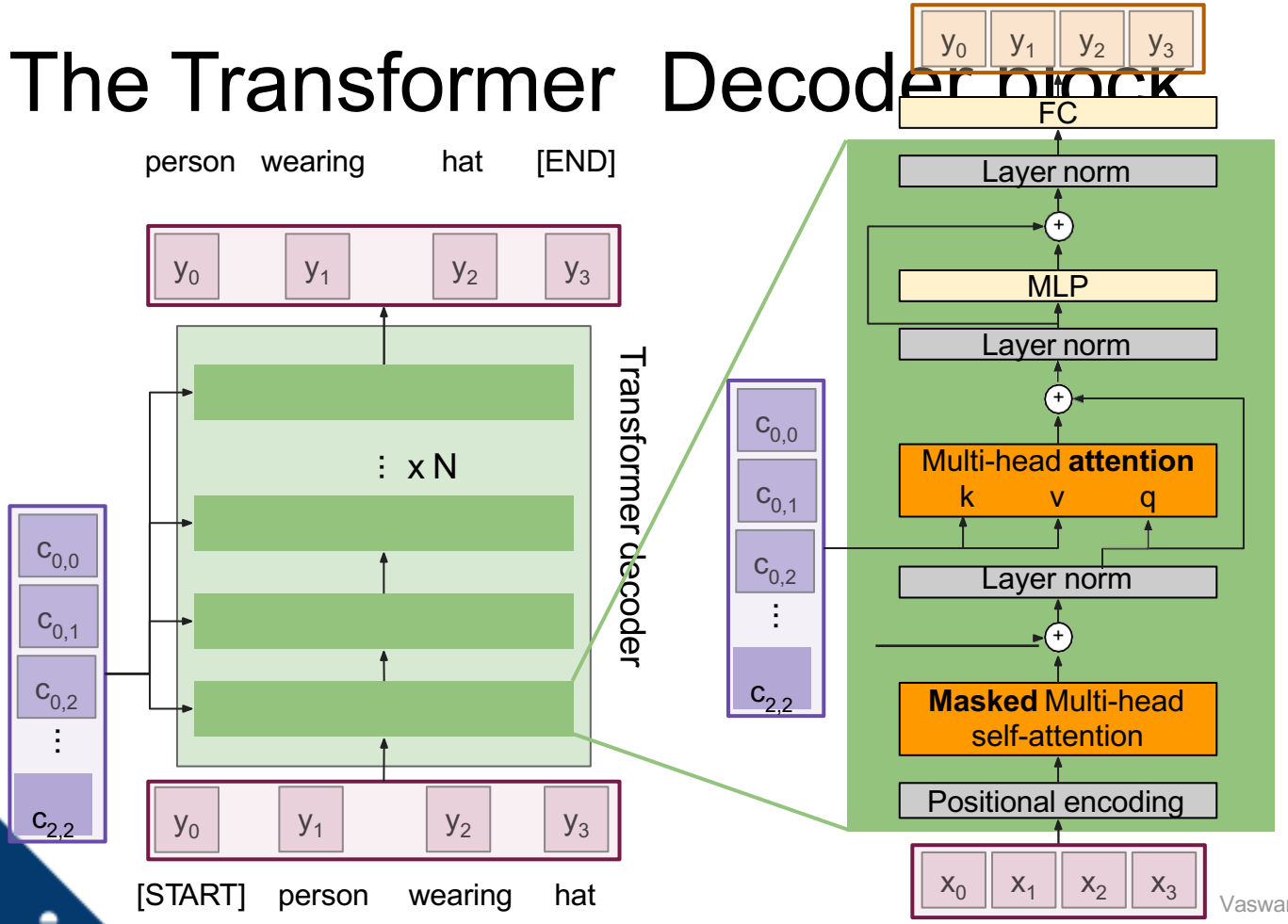
Self-attention is the only interaction between vectors.

Layer norm and MLP operate independently per vector.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, “Attention is all you need”, NeurIPS 2017

The Transformer Decoder block



Transformer Decoder Block:

Inputs: Set of vectors **x** and Set of context vectors **c.**
Outputs: Set of vectors **y.**

Masked Self-attention only interacts with past inputs.

Multi-head attention block is NOT self-attention. It attends over encoder outputs.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, "Attention is all you need", NeurIPS 2017



Today

- **Computer Vision Tasks**
  - **Semantic Segmentation**
  - Object Detection
  - Instance Segmentation

- Visualization & Understanding
  - Model Layers Visualization
  - Saliency Maps
  - CAM & Grad-CAM

- Foundation Models and Prompt Segmentation



Computer Vision Tasks

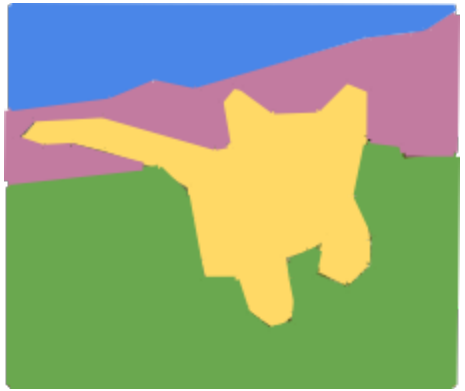
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple Object

This image is CC0 public domain

Recall: Image Classification: A core task in Computer Vision



This image by Nikita is licensed under CC-BY 2.0

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

cat

Semantic Segmentation

Classification

CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection

DOG, DOG, CAT

Instance Segmentation

DOG, DOG, CAT

Multiple Object

Semantic Segmentation: The Problem



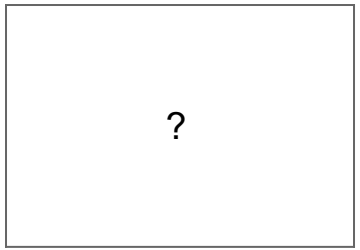


GRASS, CAT, TREE,
SKY, ...

Paired training data: for each training image,
each pixel is labeled with a semantic category.







At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window

Full image



Semantic Segmentation Idea: Sliding Window

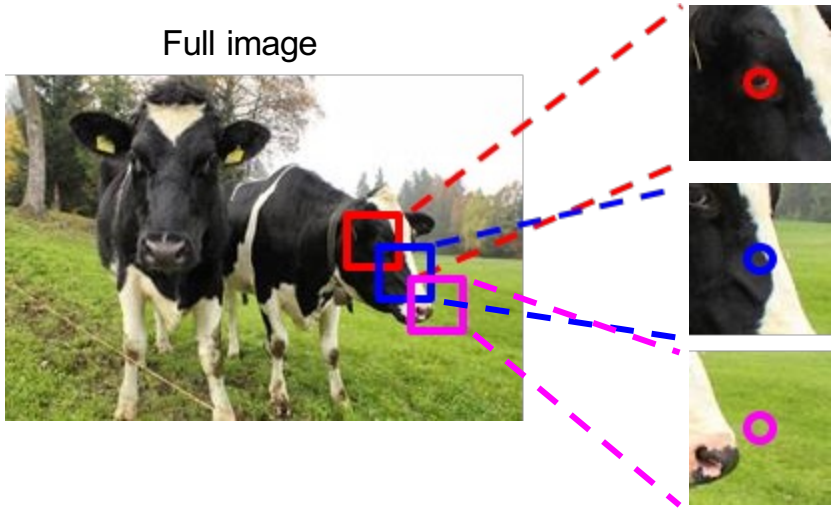
Full image



Impossible to classify without context

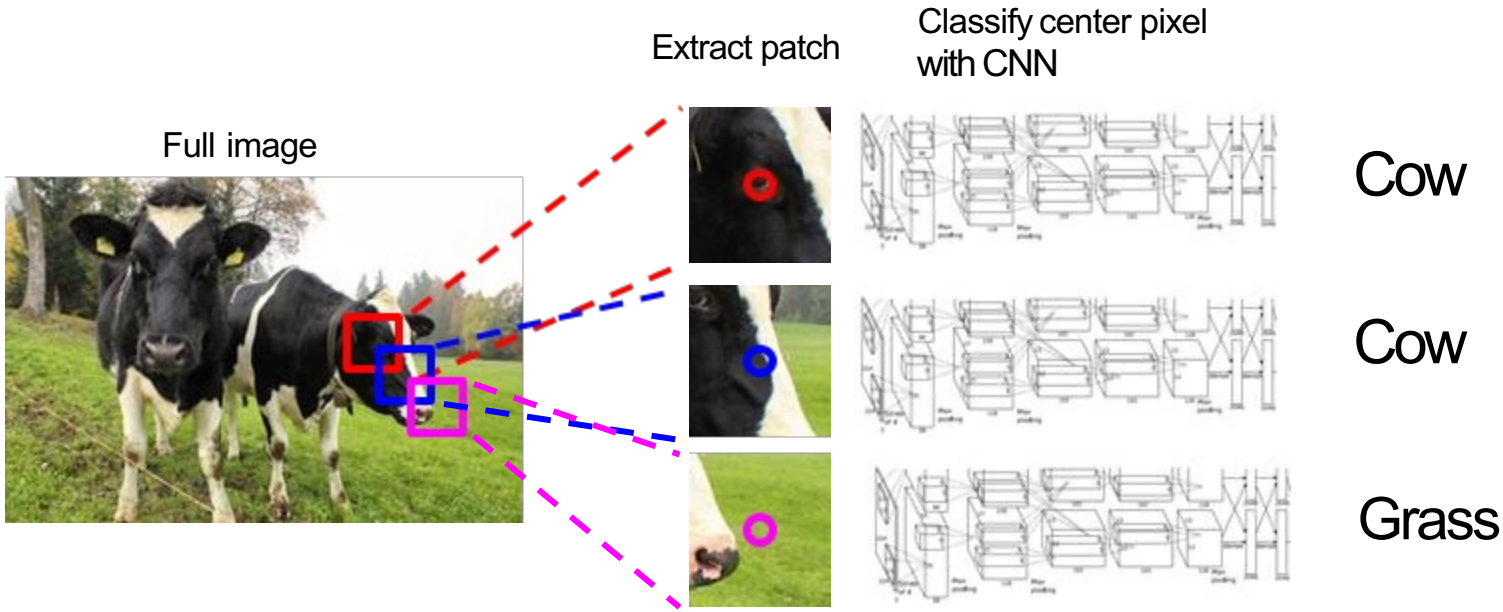
Q: how do we include context?

Semantic Segmentation Idea: Sliding Window



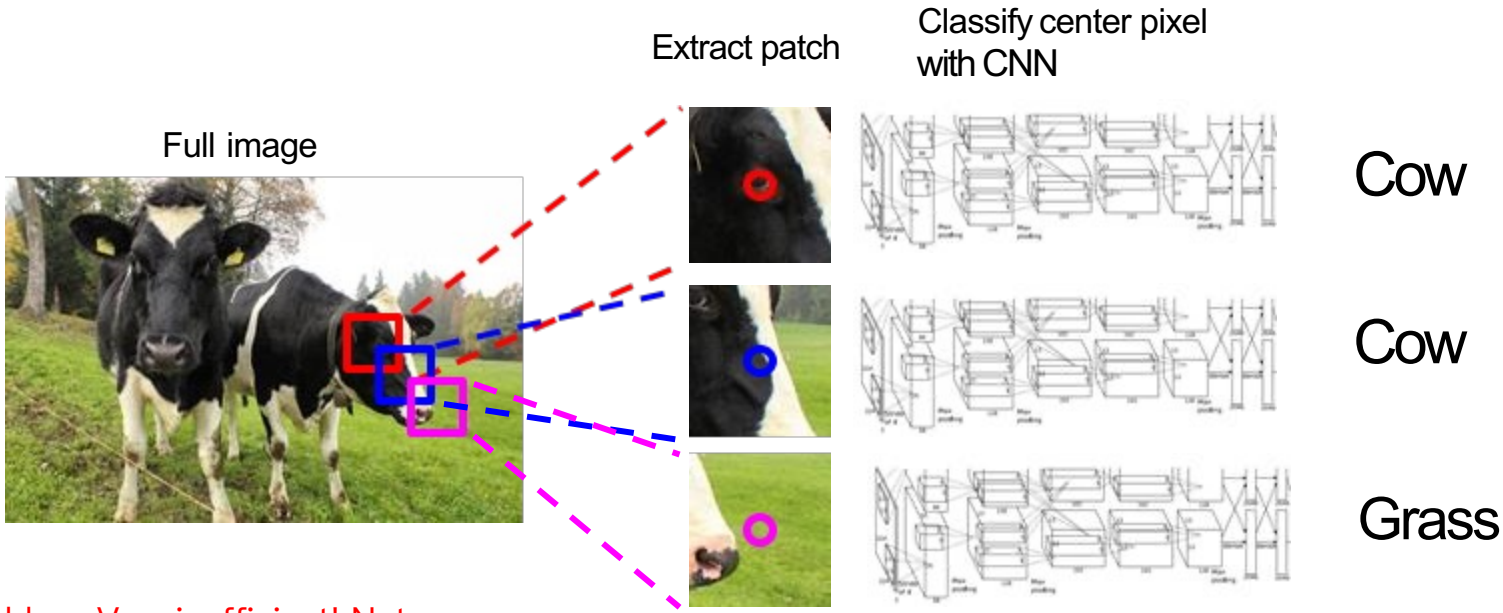
Q: how do we model this?

Semantic Segmentation Idea: Sliding Window



Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Semantic Segmentation Idea: Sliding Window



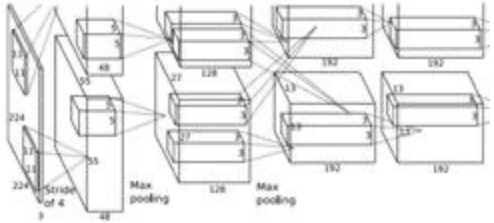
Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Semantic Segmentation Idea: Convolution

Full image





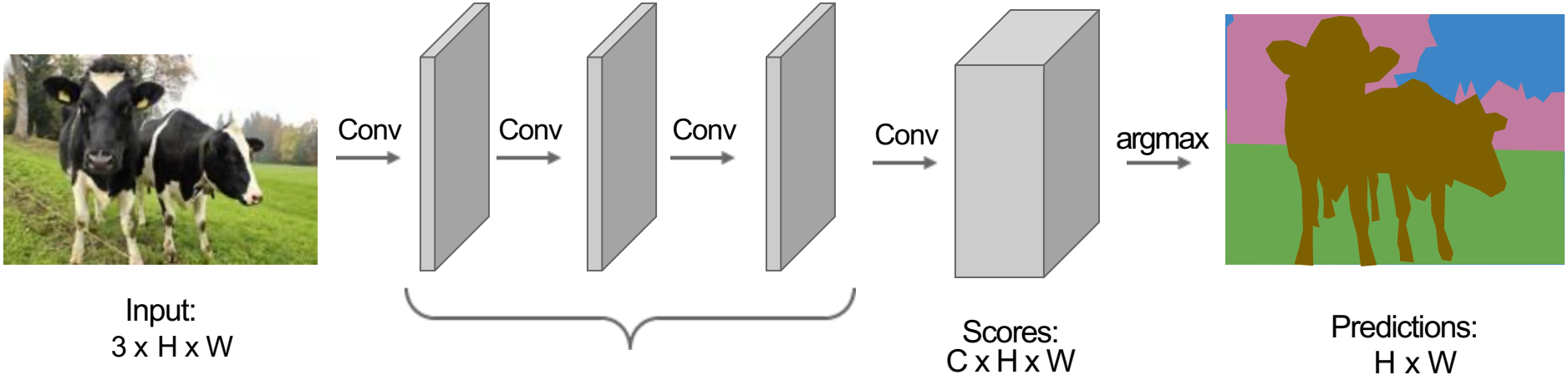


An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Problem: classification architectures often reduce feature spatial sizes to go deeper, but semantic segmentation requires the output size to be the same as input size.

Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Input:
3 x H x W

Conv → Conv → Conv → Conv → argmax

Convolutions:
Dx H x W

Scores:
C x H x W



Predictions:
H x W

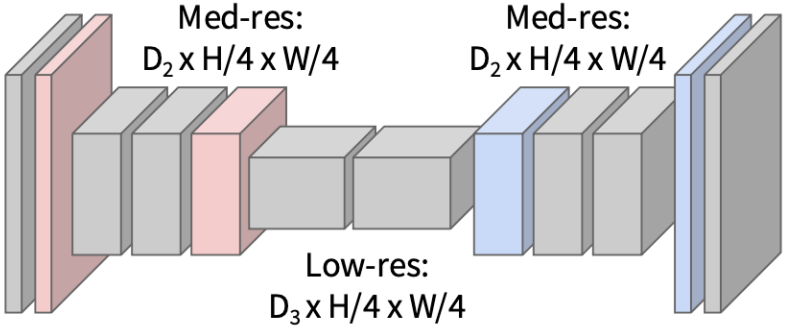
Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

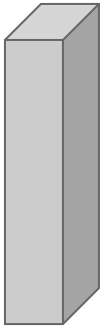


Input:
3 x H x W



High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2



C x H x W



Predictions:
H x W

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Semantic Segmentation Idea: Fully Convolutional

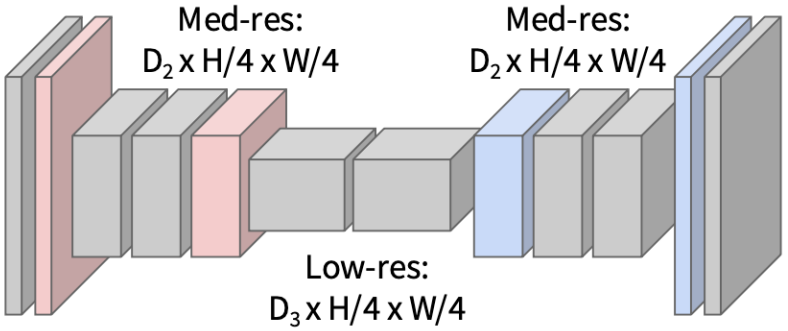
Downsampling:
Pooling, strided convolution

Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

Upsampling:
???

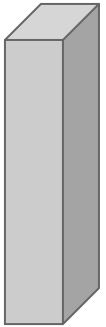


Input:
3 x H x W



High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2



C x H x W



Predictions:
H x W

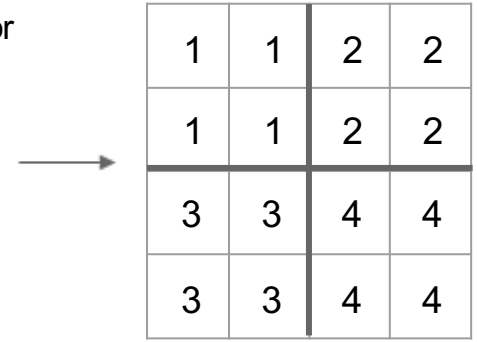
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4

Input: 2 x 2

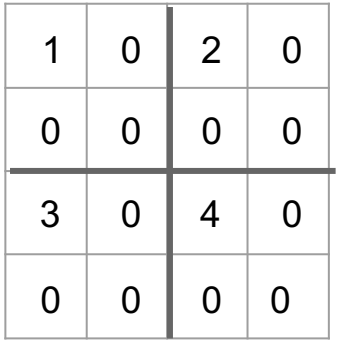


Output: 4 x 4

“Bed of Nails”

1	2
3	4

Input: 2 x 2



Output: 4 x 4

In-Network upsampling: “Max Unpooling”

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

Rest of the network

Max Unpooling
Use positions from pooling layer

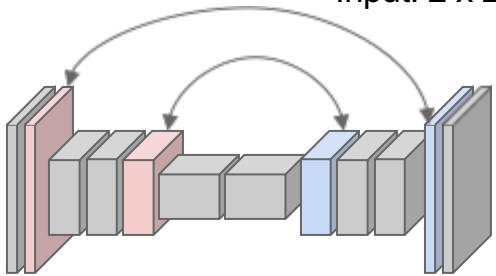
1	2
3	4

Input: 2 x 2

0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

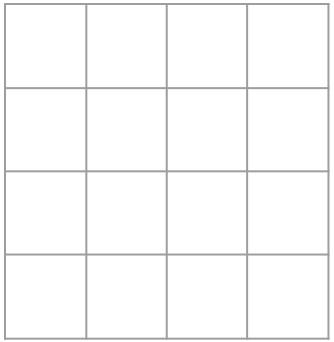
Output: 4 x 4

Corresponding pairs of downsampling and upsampling layers

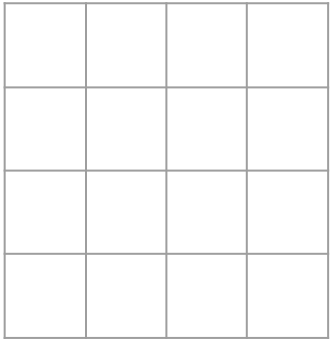


Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



Input: 4 x 4

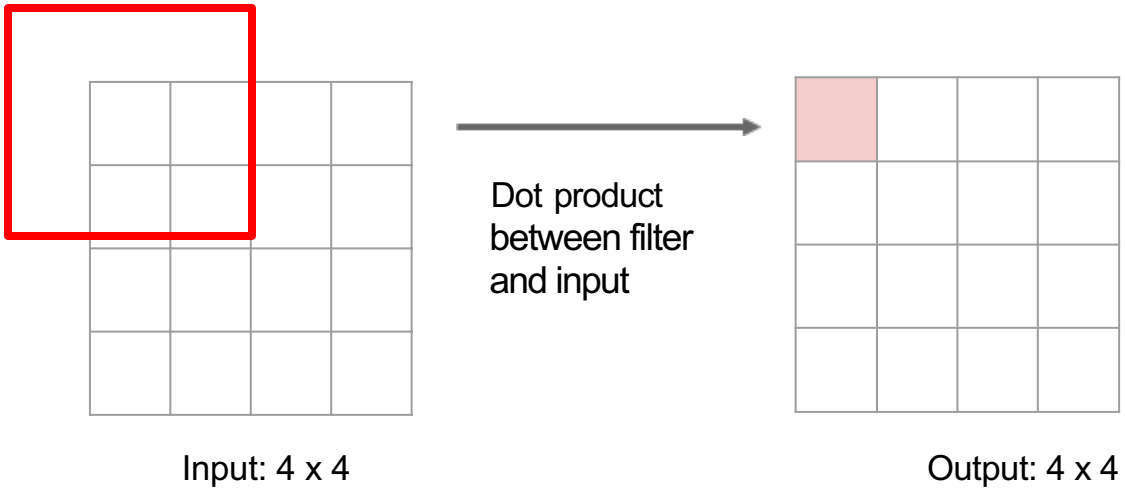


Output: 4 x 4



Learnable Upsampling

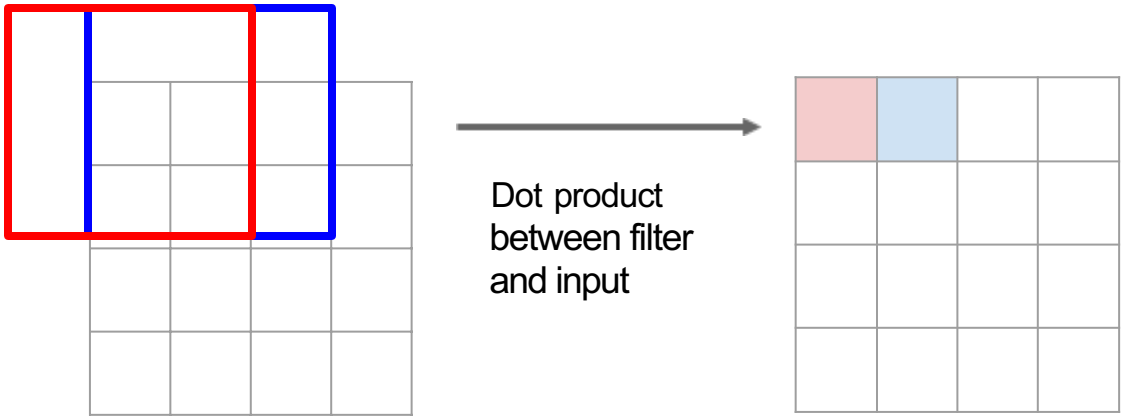
Recall: Normal 3 x 3 convolution, stride 1 pad 1





Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



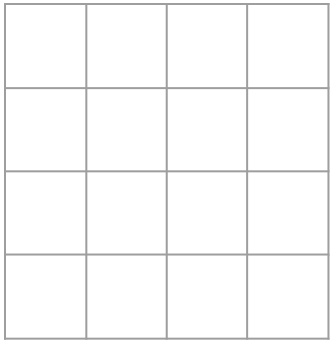
Input: 4 x 4

Output: 4 x 4

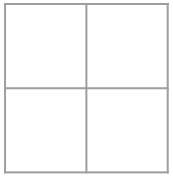


Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

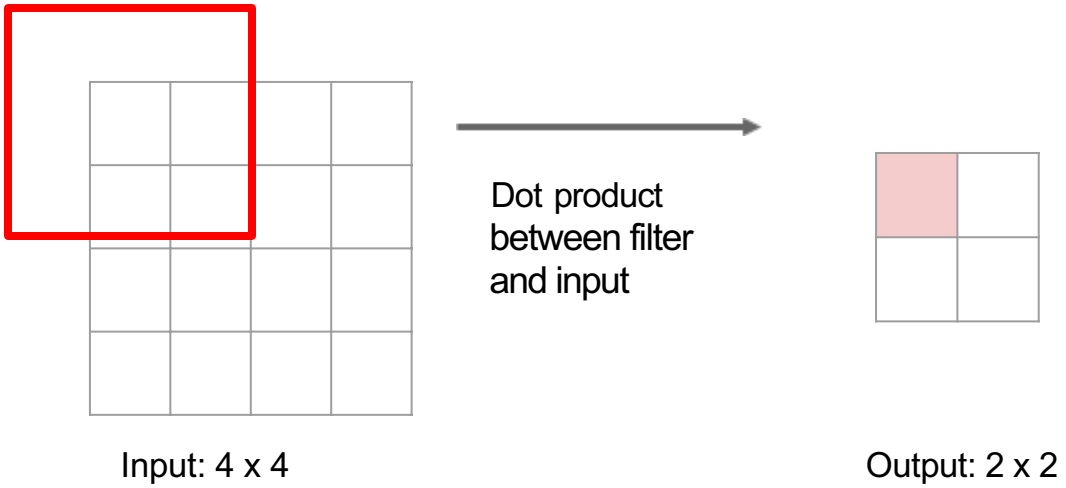


Output: 2 x 2



Learnable Upsampling

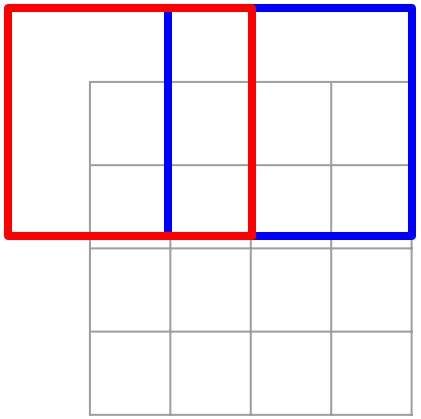
Recall: Normal 3 x 3 convolution, stride 2 pad 1





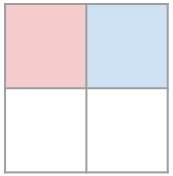
Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

Dot product between filter and input



Output: 2 x 2

Filter moves 2 pixels in the input for every one pixel in the output

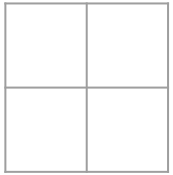
Stride gives ratio between movement in input and output

We can interpret strided convolution as “learnable downsampling”.

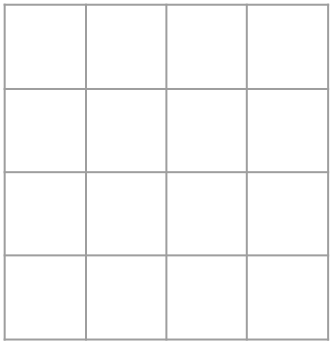


Learnable Upsampling: Transposed Convolution

3 x 3 transposed convolution, stride 2 pad 1



Input: 2 x 2

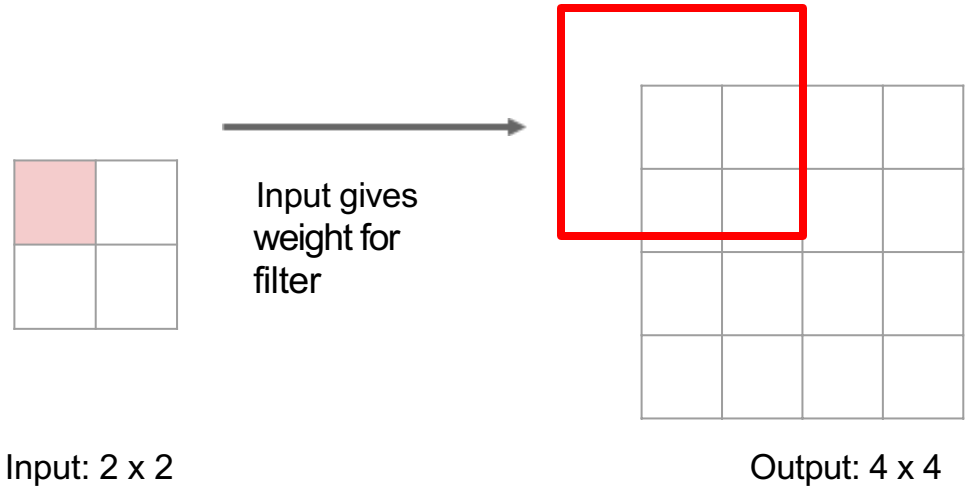


Output: 4 x 4



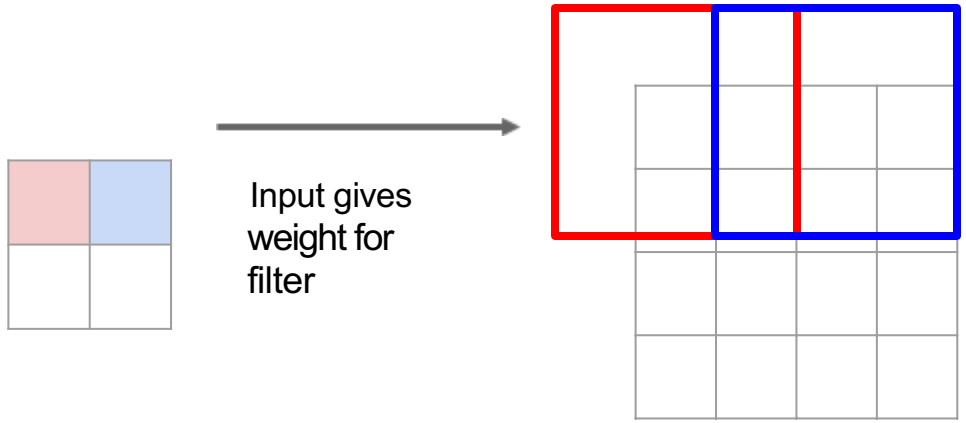
Learnable Upsampling: Transposed Convolution

3 x 3 transposed convolution, stride 2 pad 1



Learnable Upsampling: Transposed Convolution

3 x 3 transposed convolution, stride 2 pad 1

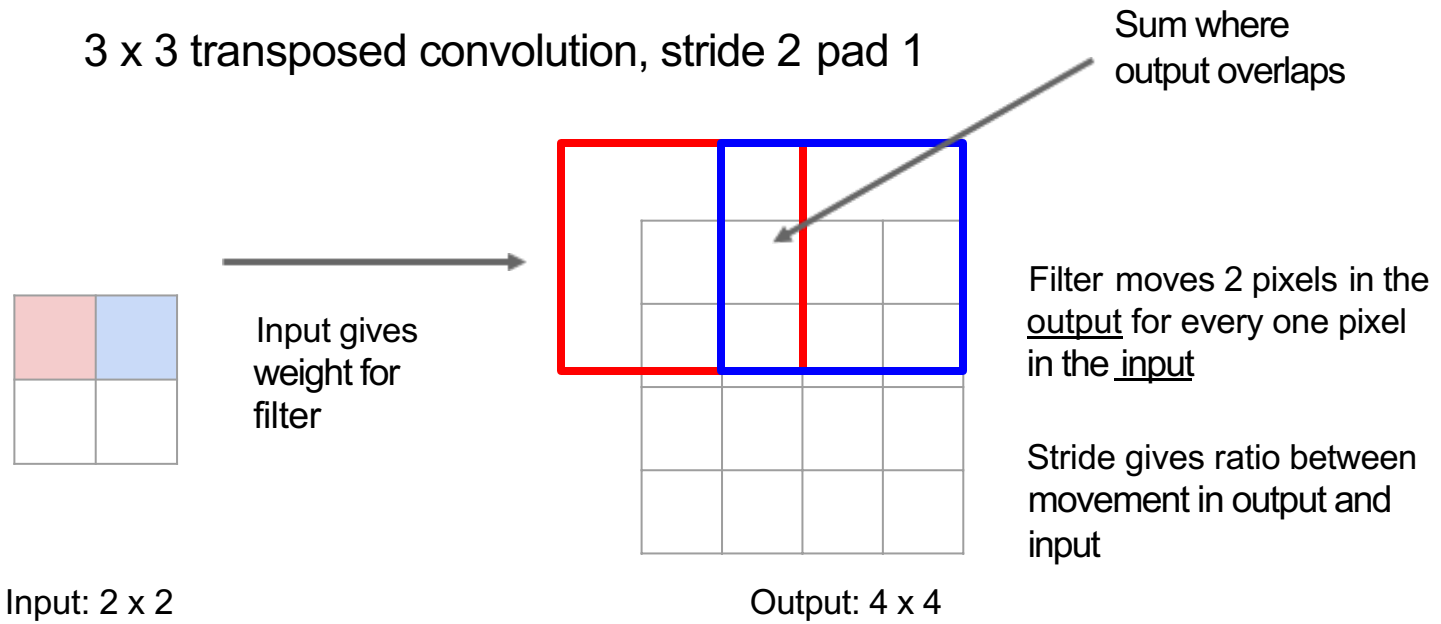


Filter moves 2 pixels in the output for every one pixel in the input

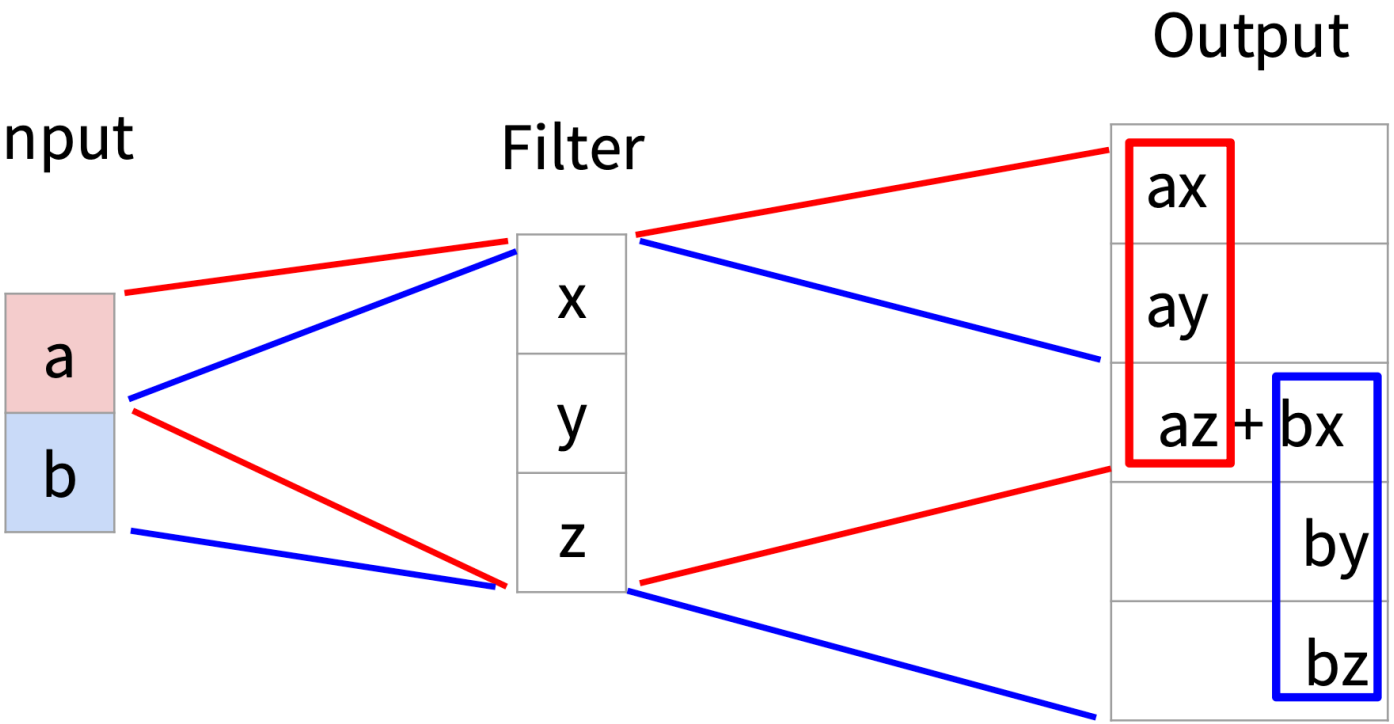
Stride gives ratio between movement in output and input



Learnable Upsampling: Transposed Convolution



Learnable Upsampling: 1D Example



Output contains copies of the filter weighted by the input, summing at where at overlaps in the output

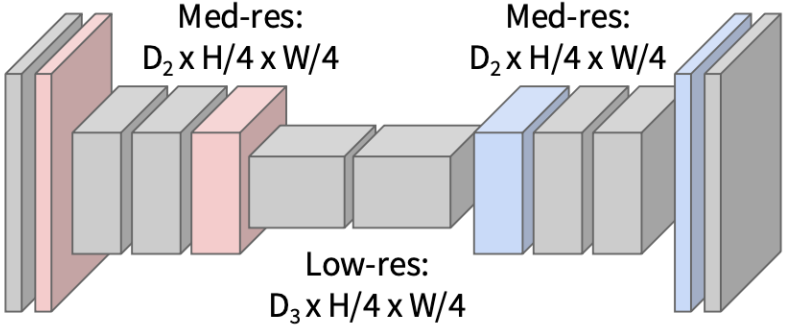
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided convolution

Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

Upsampling:
Unpooling or strided transposed convolution







Input:
3 x H x W

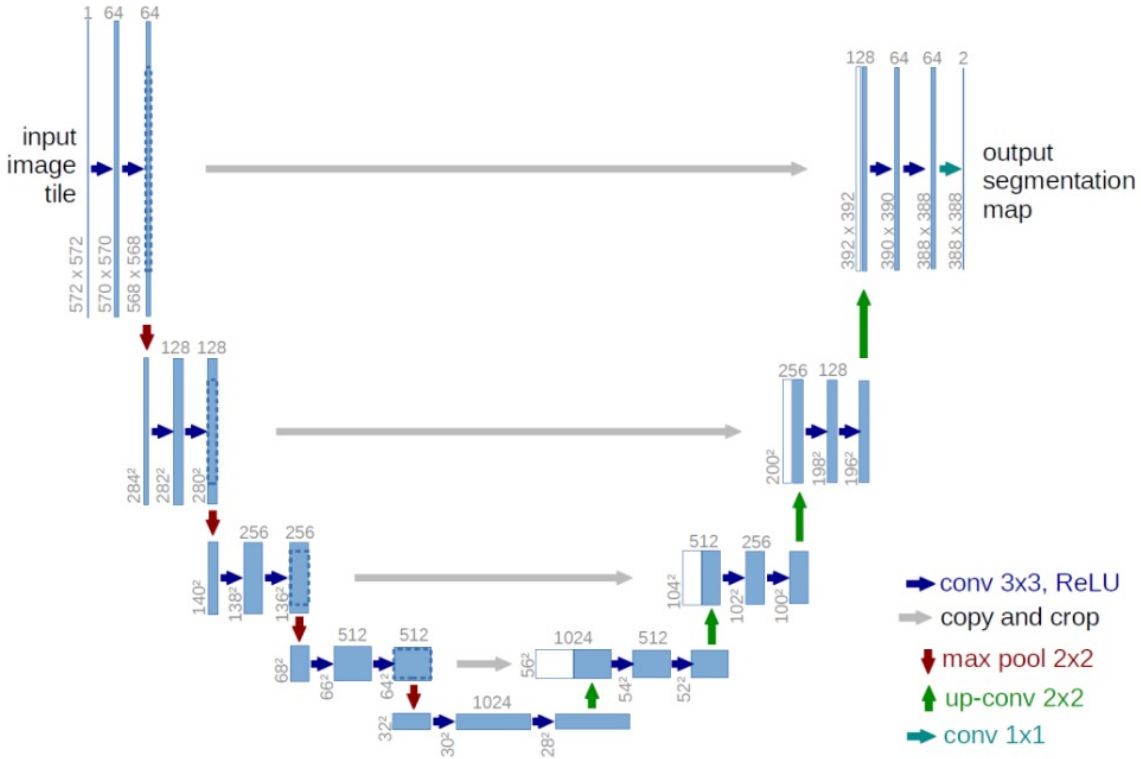
High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2

Predictions:
H x W

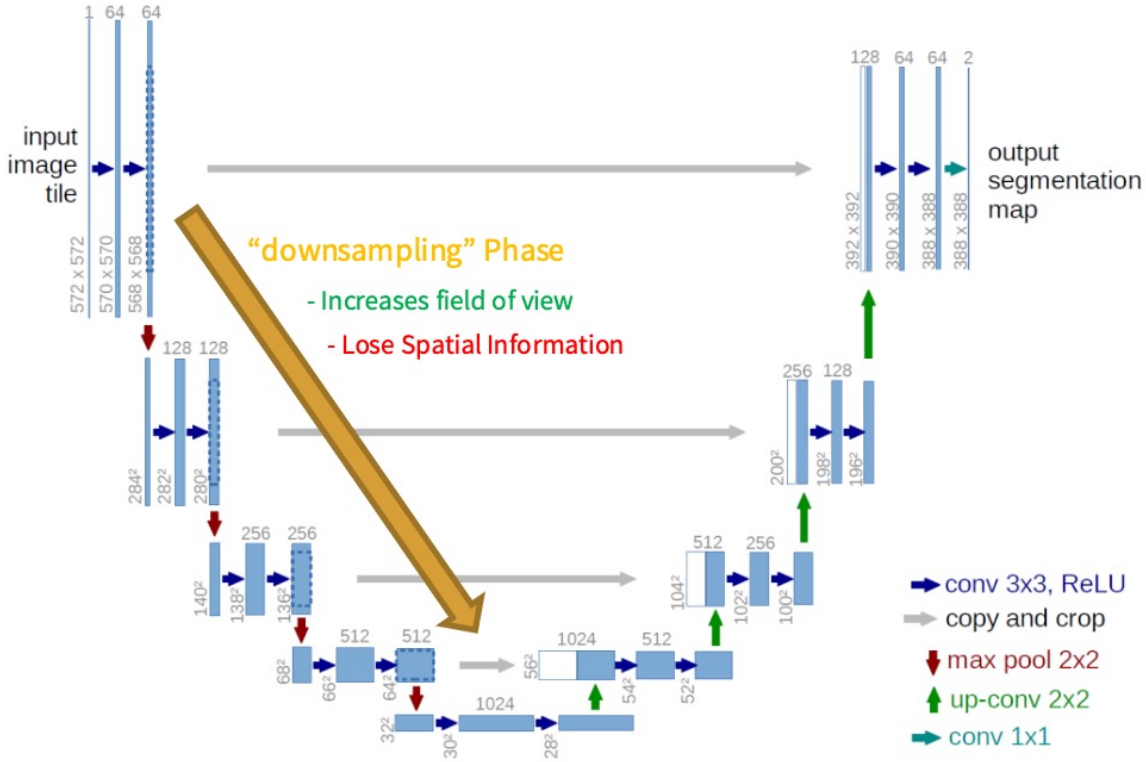
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

U-Net



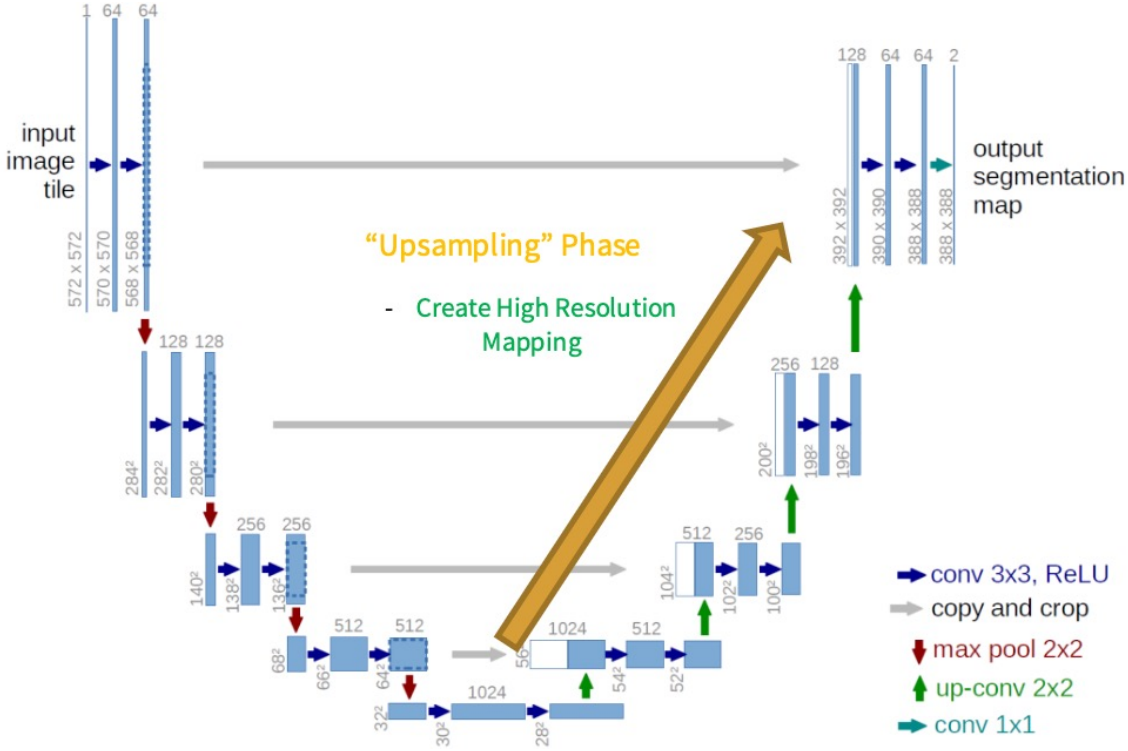
Ronneberger et al. (2015) U-net Architecture

U-Net



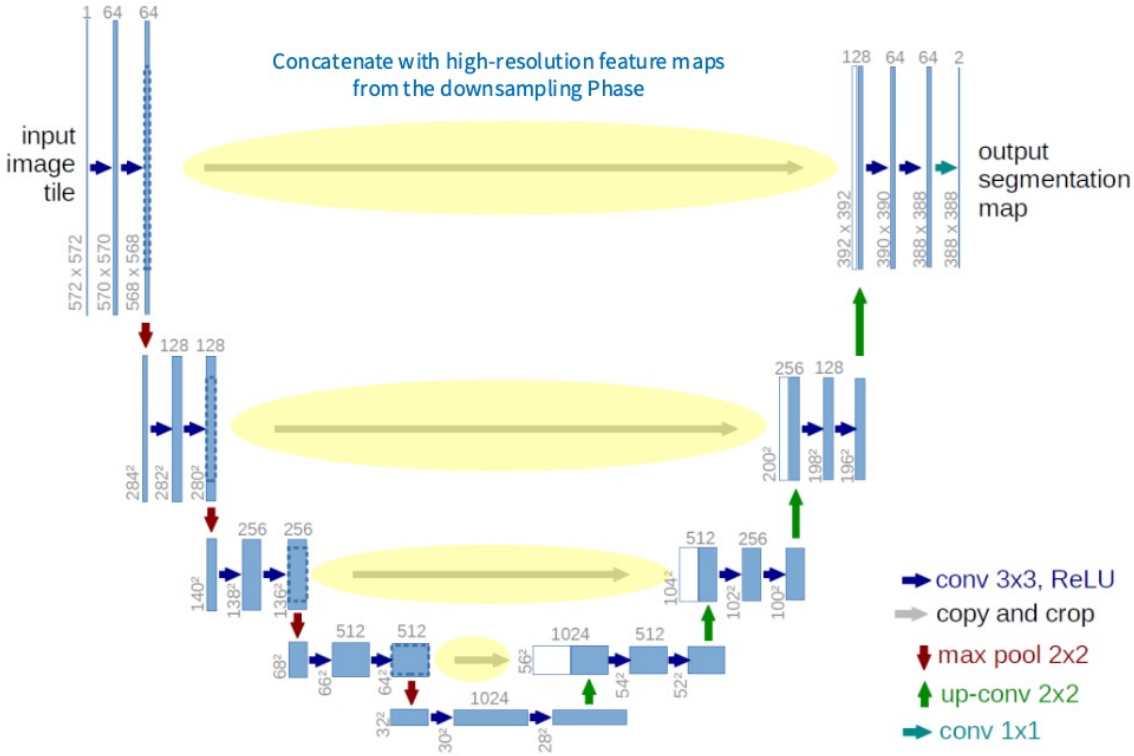
Ronneberger et al. (2015) U-net Architecture

U-Net



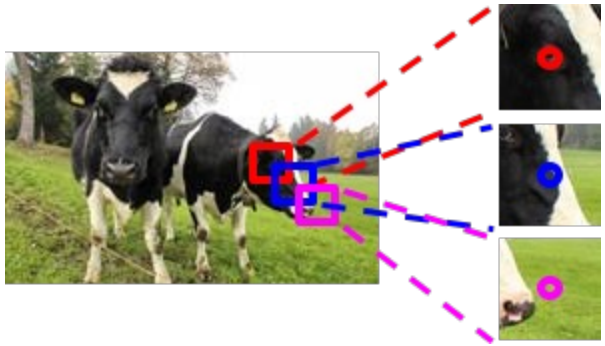
Ronneberger et al. (2015) U-net Architecture

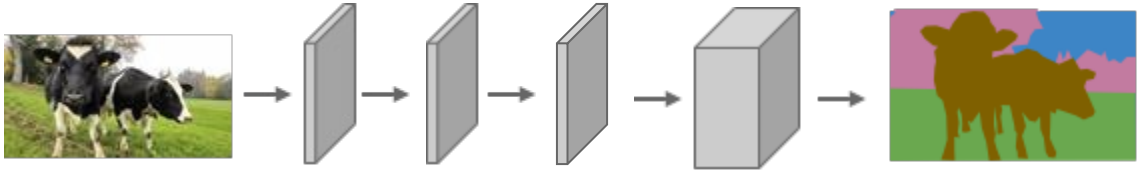
U-Net



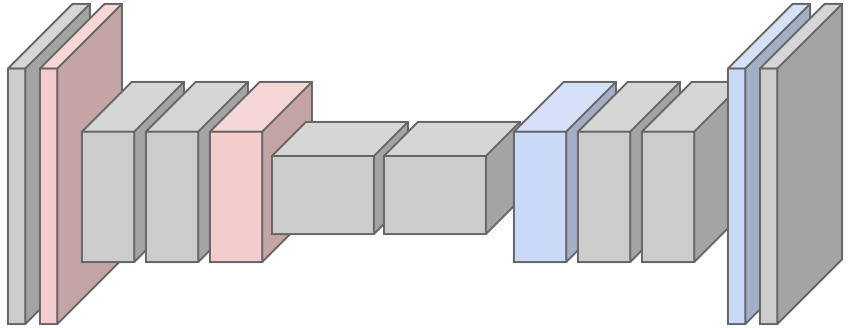
Ronneberger et al. (2015) U-net Architecture

Semantic Segmentation: Summary









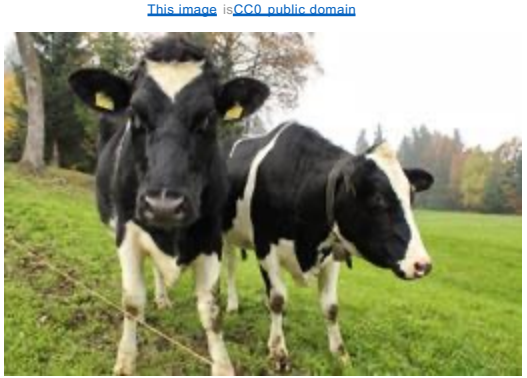


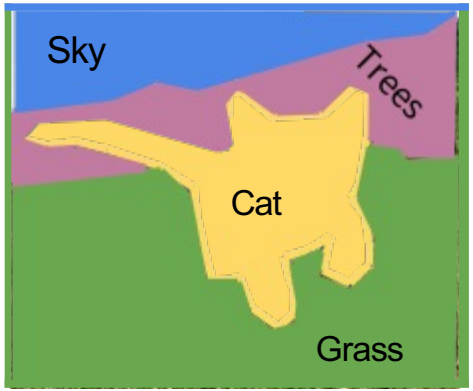
Semantic Segmentation

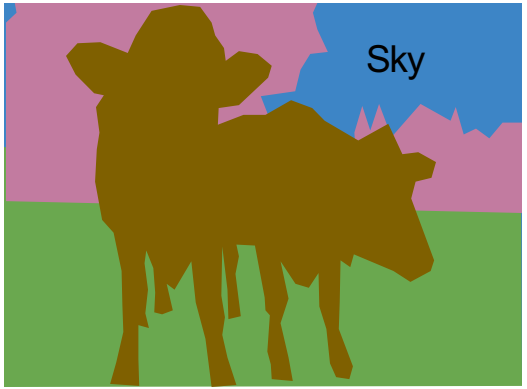
Label each pixel in the image with a category label

Don’t differentiate instances, only care about pixels









Today

- **Computer Vision Tasks**
  - Semantic Segmentation
  - **Object Detection**
  - Instance Segmentation

- Visualization & Understanding
  - Model Layers Visualization
  - Saliency Maps
  - CAM & Grad-CAM

- Foundation Models and Prompt Segmentation



Object Detection

Classification

CAT

No spatial extent

Semantic Segmentation

GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection

DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple Object

Object Detection

Classification

CAT

No spatial extent

Semantic Segmentation

GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

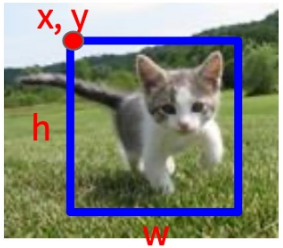
Instance Segmentation



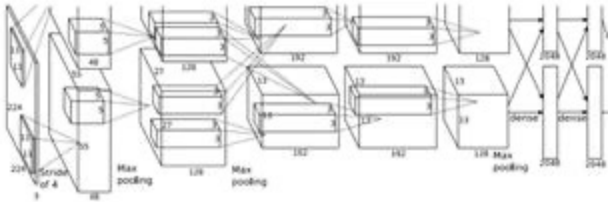
DOG, DOG, CAT

Multiple Object

Object Detection: Single Object
(Classification + Localization)



This image is CC0 public domain



Fully Connected:
4096 to 1000

Class Scores Cat: 0.9
Dog: 0.05
Car: 0.01
...

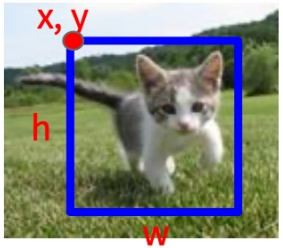
Vector: 4096

Fully Connected:
4096 to 4

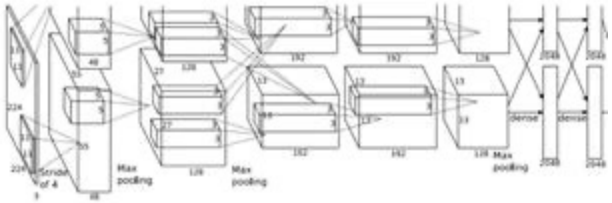
Top-left corner

Box Coordinates
(x, y, w, h)

Object Detection: Single Object
(Classification + Localization)



This image is CC0 public domain



Vector: 4096

Fully Connected: 4096 to 1000

Class Scores Cat: 0.9
Dog: 0.05
Car: 0.01
...

Softmax Loss

Correct label: Cat

Fully Connected: 4096 to 4

Box Coordinates (x, y, w, h)

L2 Loss

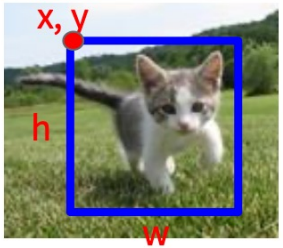
Correct box: (x', y', w', h')

Treat localization as a regression problem!

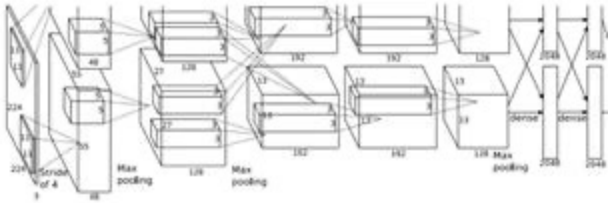


Object Detection: Single Object

(Classification + Localization)



This image is CC0 public domain



Fully Connected: 4096 to 1000

Class Scores Cat: 0.9
Dog: 0.05
Car: 0.01
...

Softmax Loss

Correct label: Cat

Multitask Loss

+

Loss

Vector: 4096

Fully Connected: 4096 to 4

Box Coordinates (x, y, w, h)

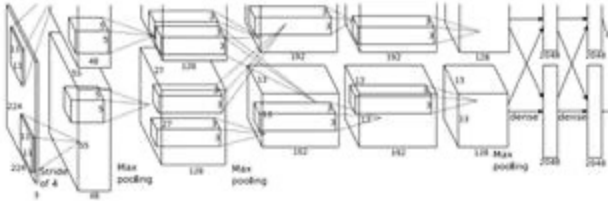
L2 Loss

Correct box: (x', y', w', h')

Treat localization as a regression problem!

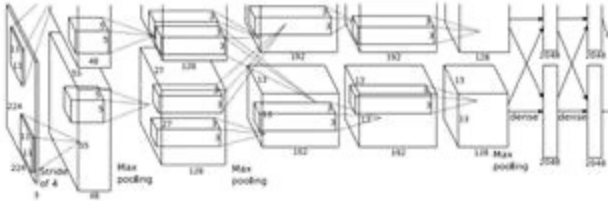
Object Detection: Multiple Objects





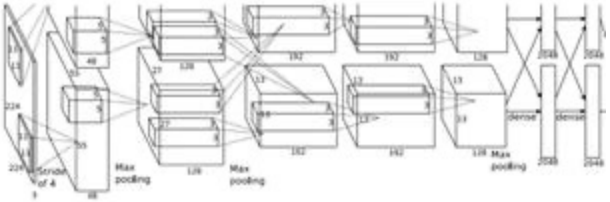
CAT: (x, y, w, h)





DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)



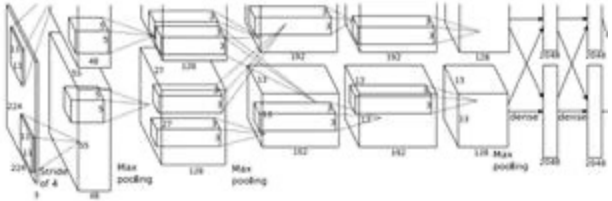


DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

Object Detection: Multiple Objects

Each image needs a different number of outputs!

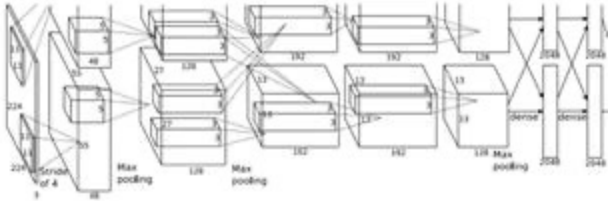




CAT: (x, y, w, h)

4 numbers

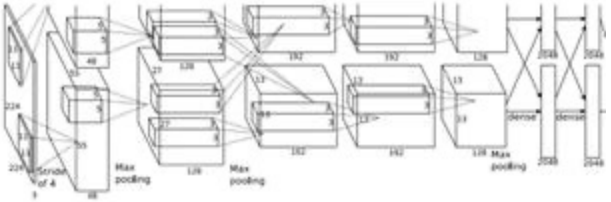




DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers





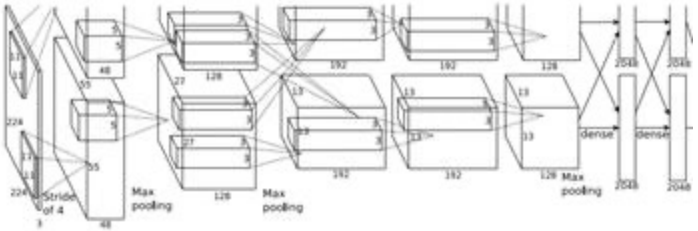
DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

Many numbers!

Object Detection: Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

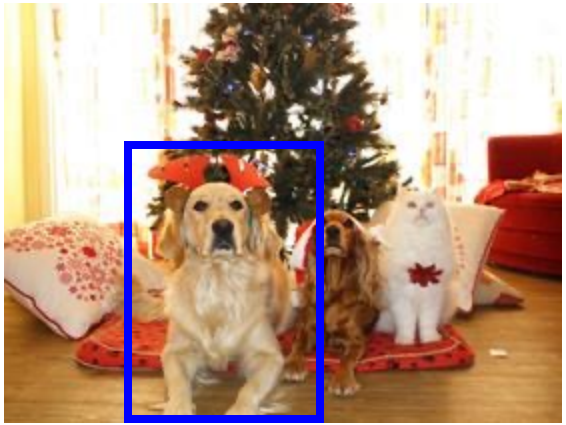


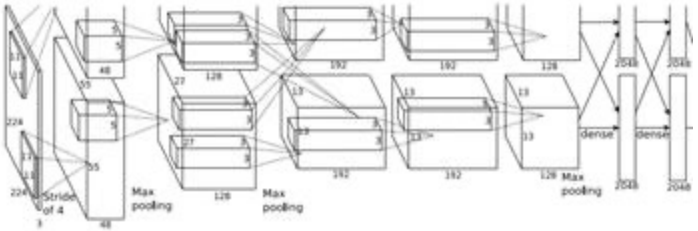


Dog? NO
Cat? NO
Background? YES

Object Detection: Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



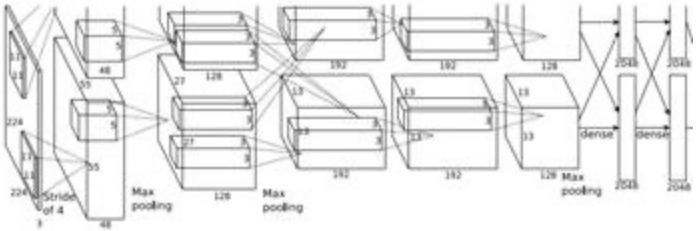


Dog? YES
Cat? NO
Background? NO

Object Detection: Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



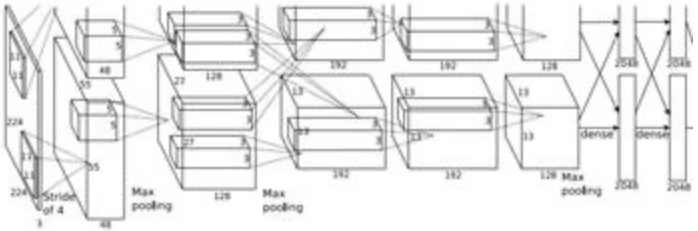


Dog? YES
Cat? NO
Background? NO

Object Detection: Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

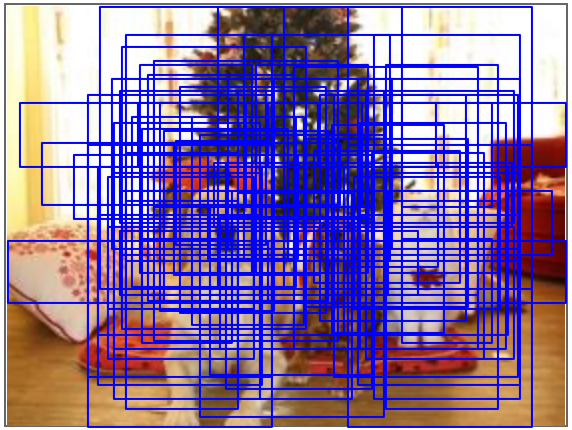




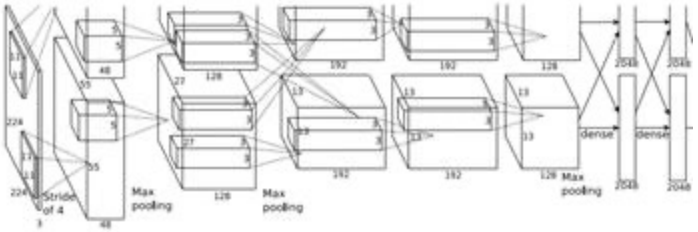
Dog? NO
Cat? YES
Background? NO

Q: What's the problem with this approach?

Object Detection: Multiple Objects



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

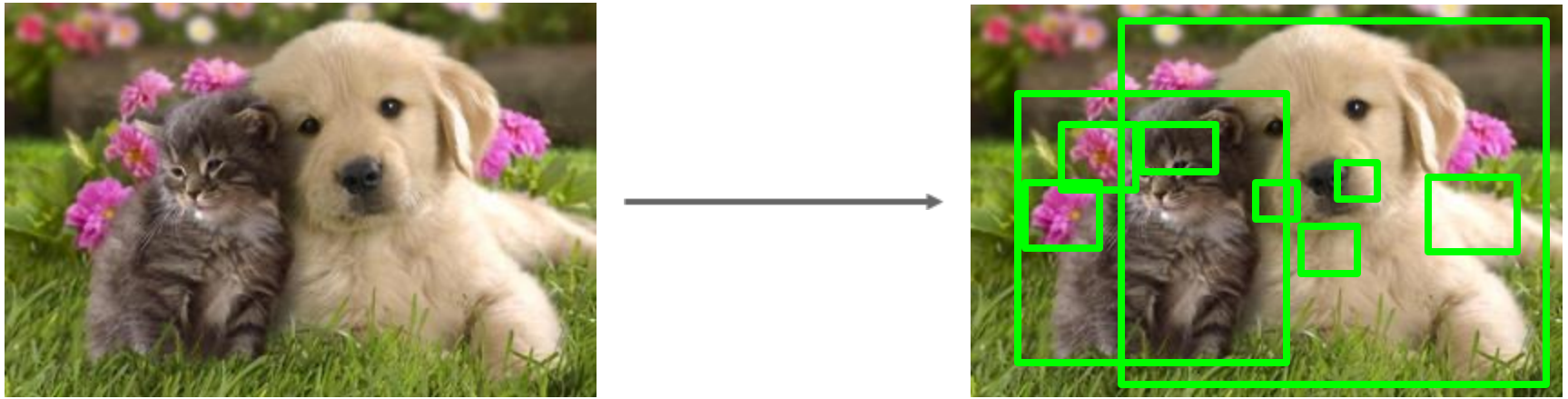


Dog? NO
Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations, scales, and aspect ratios, very computationally expensive!

Region Proposals: Selective Search

- Find “blobby” image regions that are likely to contain objects
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013
Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014
Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

R-CNN



Input image



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

R-CNN

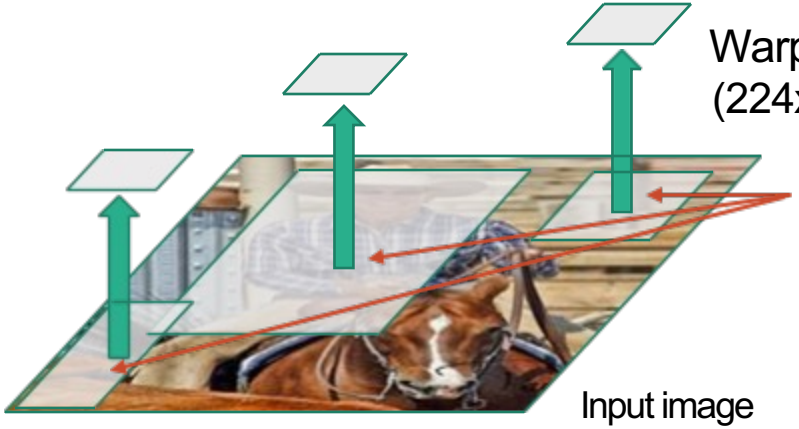


Input image

Regions of Interest (RoI) from a proposal method (~2k)

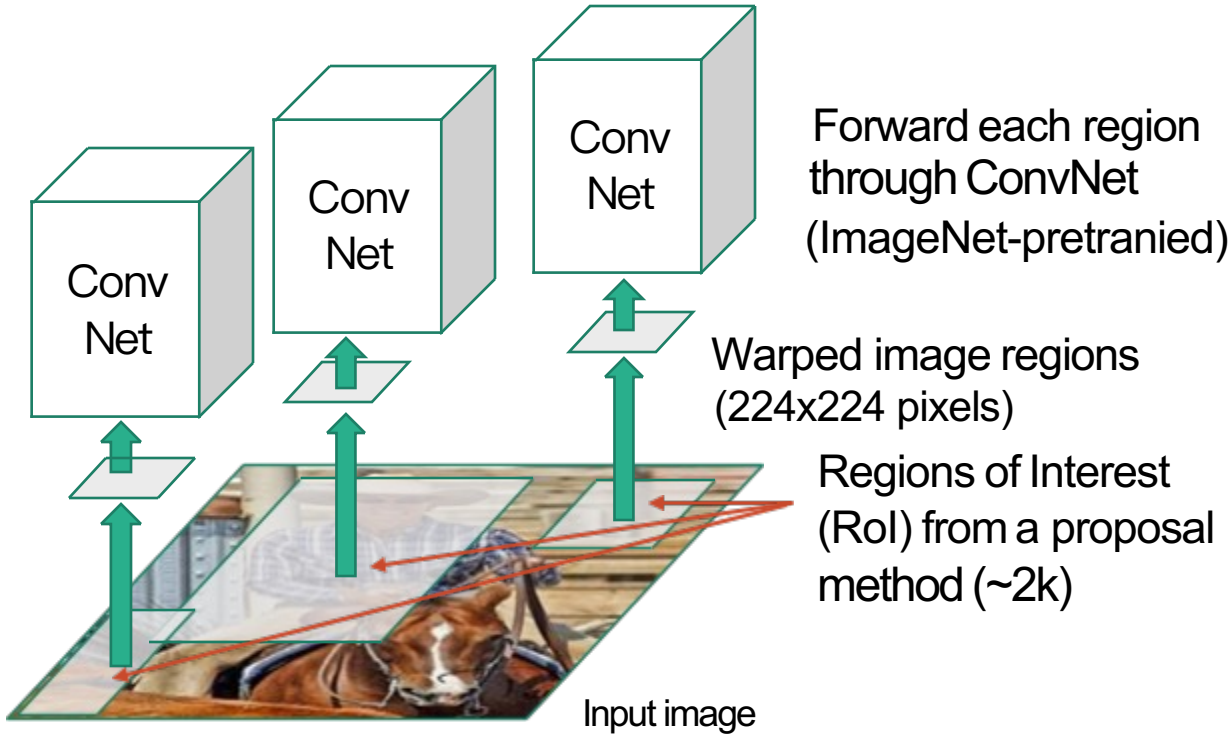
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

R-CNN



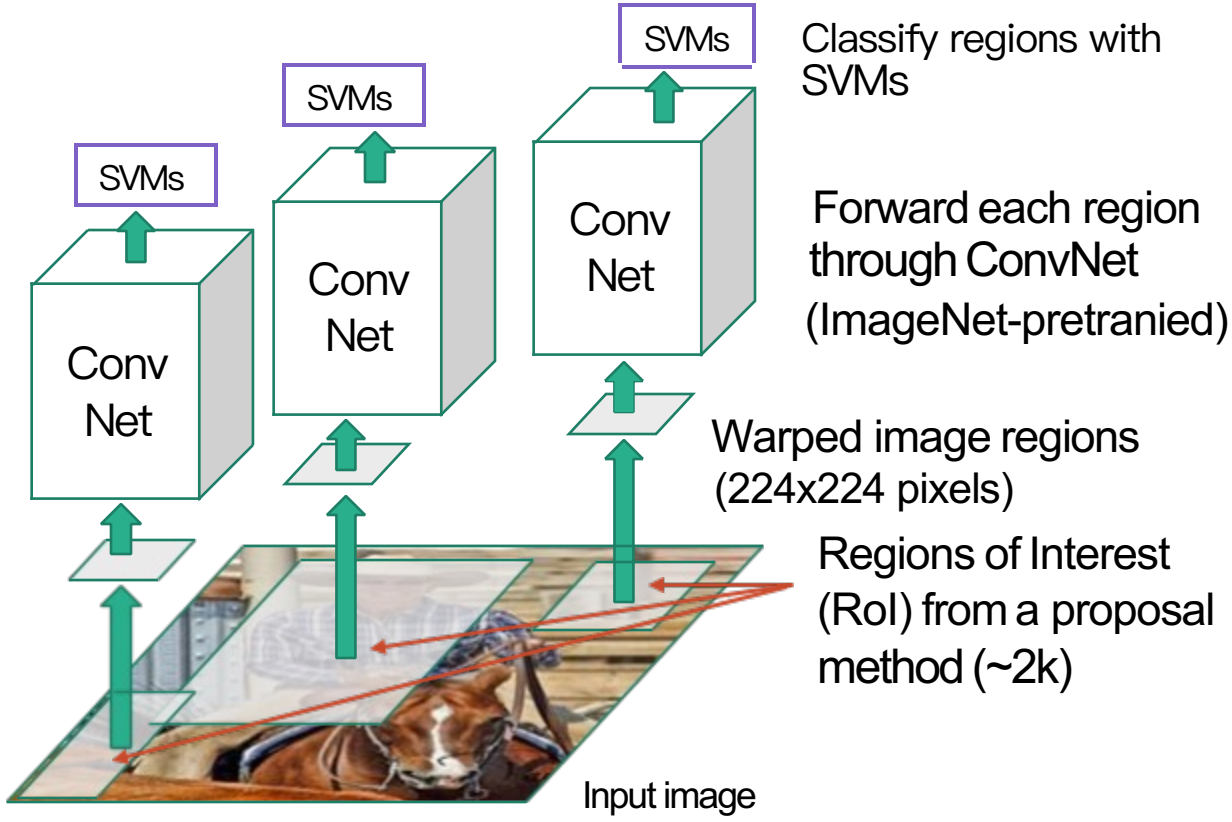
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

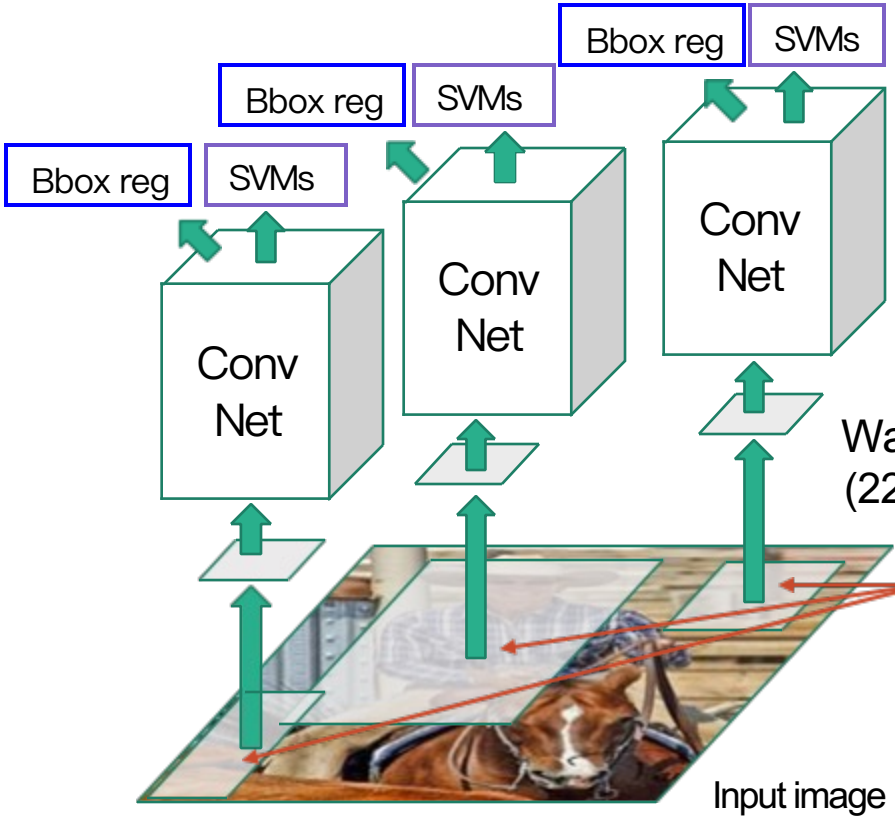
R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

R-CNN

Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)



Classify regions with SVMs

Forward each region through ConvNet (ImageNet-pretranied)

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

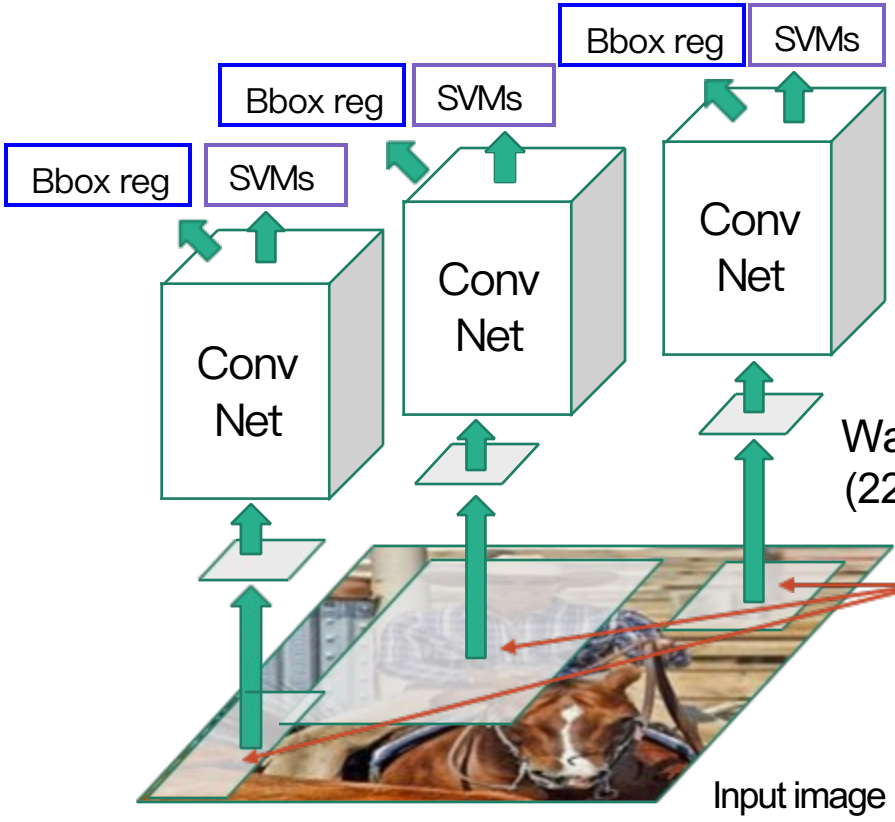


Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

R-CNN

Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)



Classify regions with SVMs

Forward each region through ConvNet (ImageNet-pretranied)

Warped image regions (224x224 pixels)

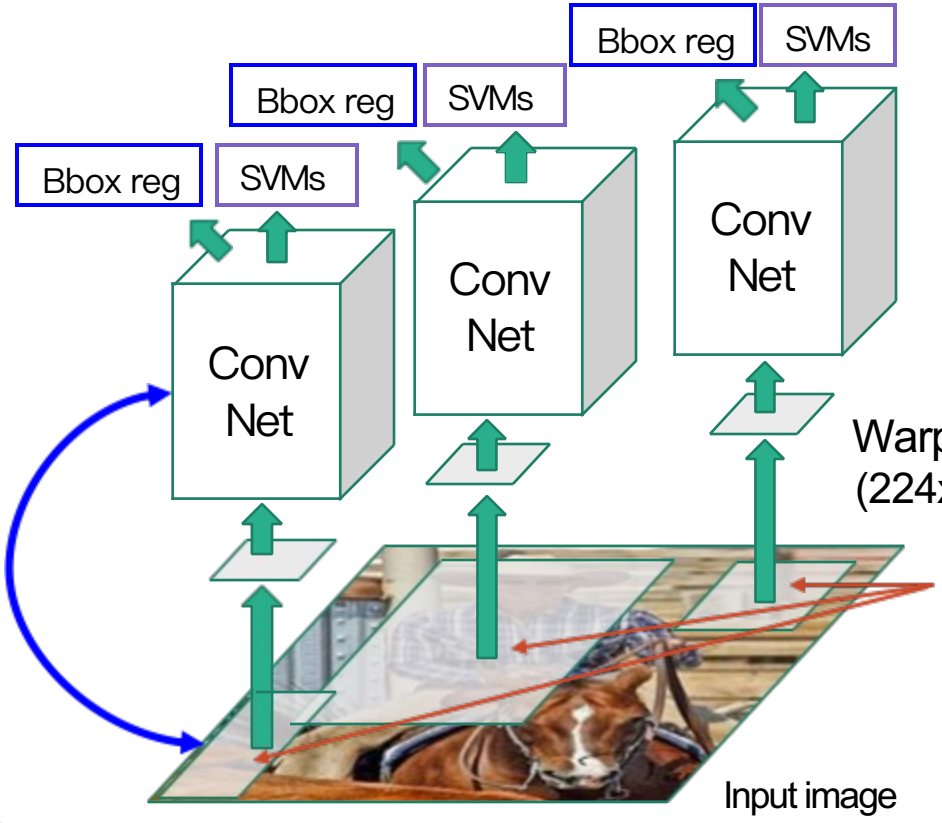
Regions of Interest (RoI) from a proposal method (~2k)

Problem: Very slow! Need to do ~2k independent forward passes for each image!

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

"Slow" R-CNN

Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)



Classify regions with SVMs

Forward each region through ConvNet (ImageNet-pretranied)

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

Problem: Very slow! Need to do ~2k independent forward passes for each image!

Idea: Pass the image through convnet before cropping! Crop the conv feature instead!



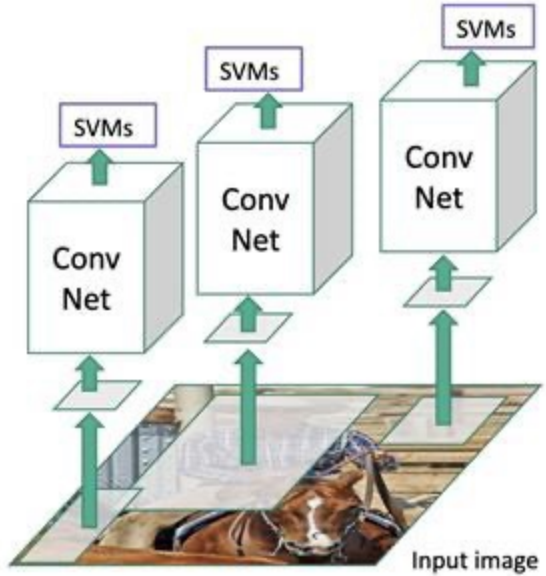
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Fast R-CNN



Input image

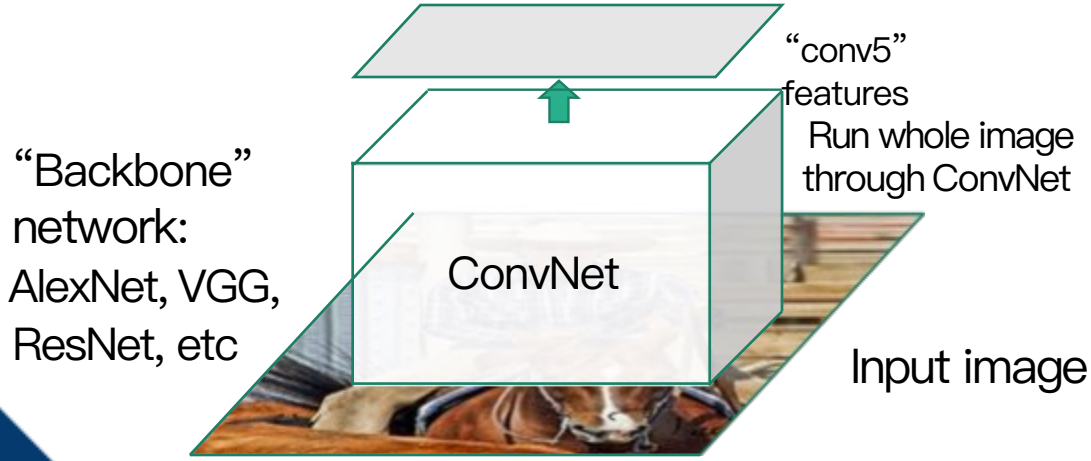
"Slow" R-CNN



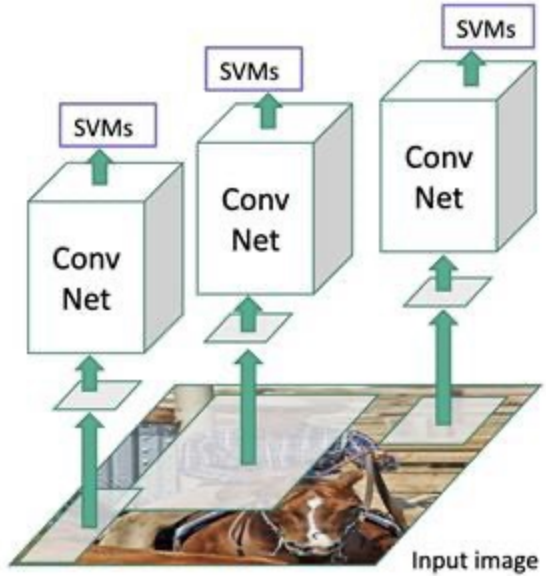


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Fast R-CNN



“Slow” R-CNN

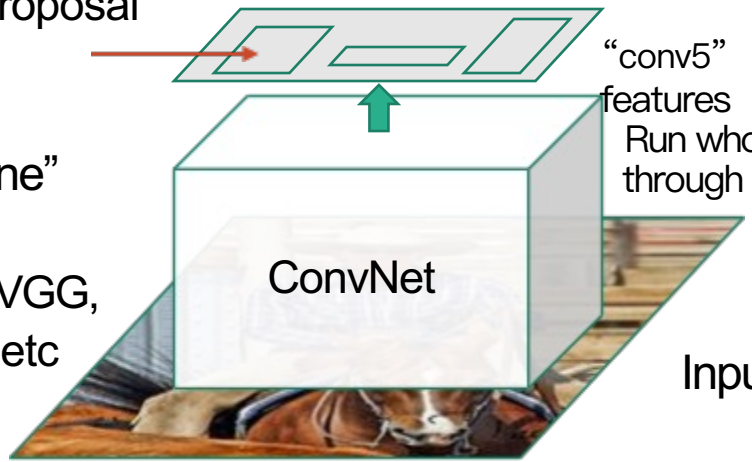


Girshick, “Fast R-CNN”, ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

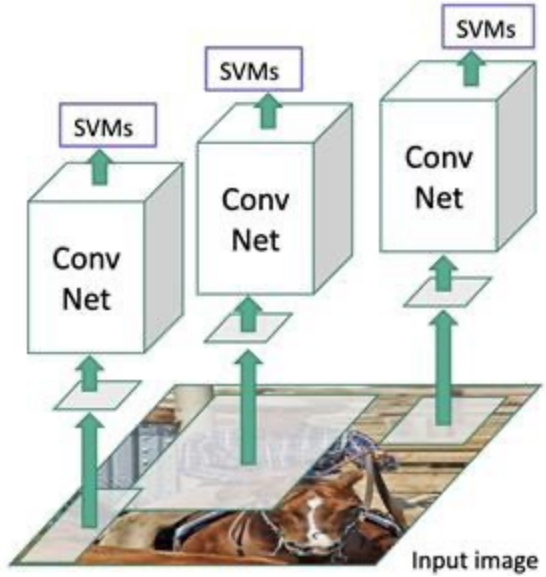
Fast R-CNN

Regions of Interest (RoIs) from a proposal method

"Backbone" network: AlexNet, VGG, ResNet, etc



"Slow" R-CNN

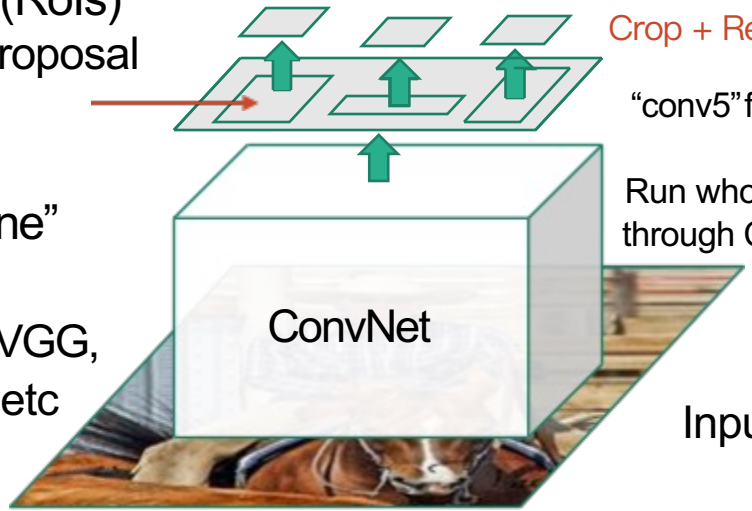


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

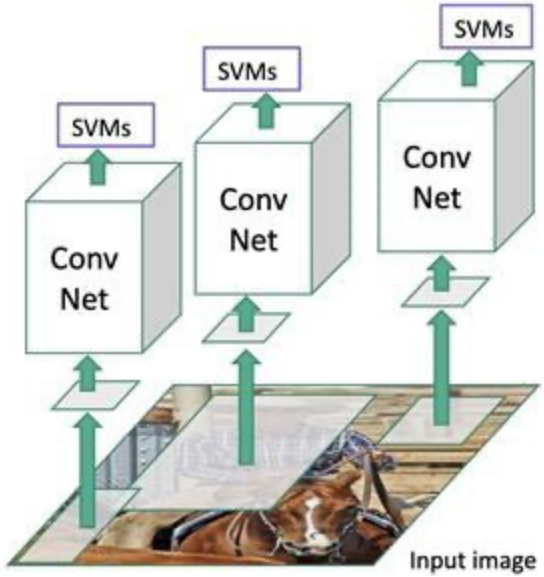
Fast R-CNN

Regions of Interest (RoIs) from a proposal method

"Backbone" network: AlexNet, VGG, ResNet, etc

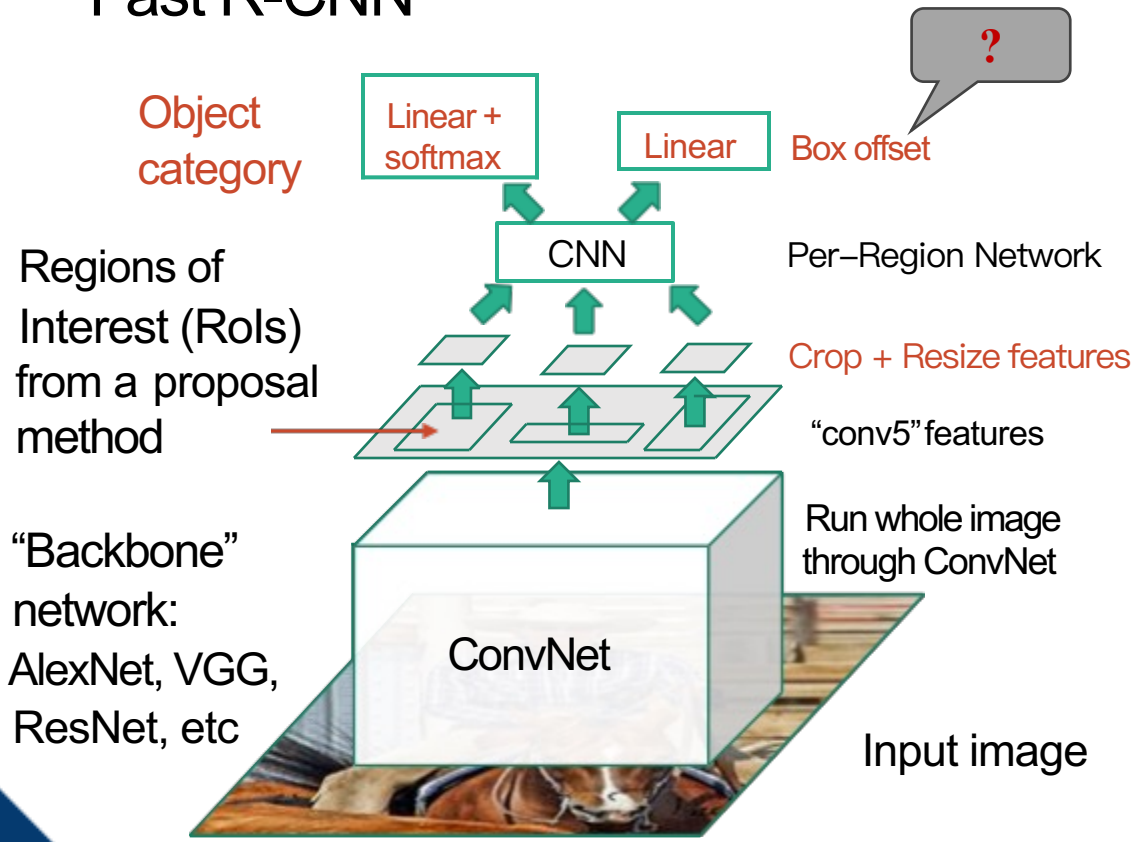


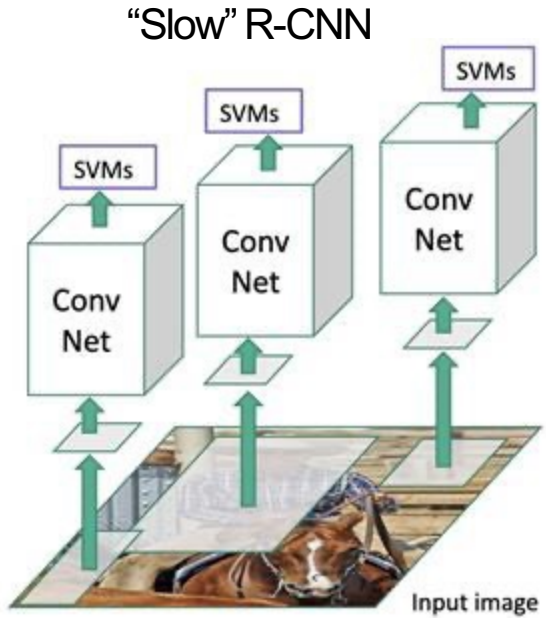
"Slow" R-CNN



Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

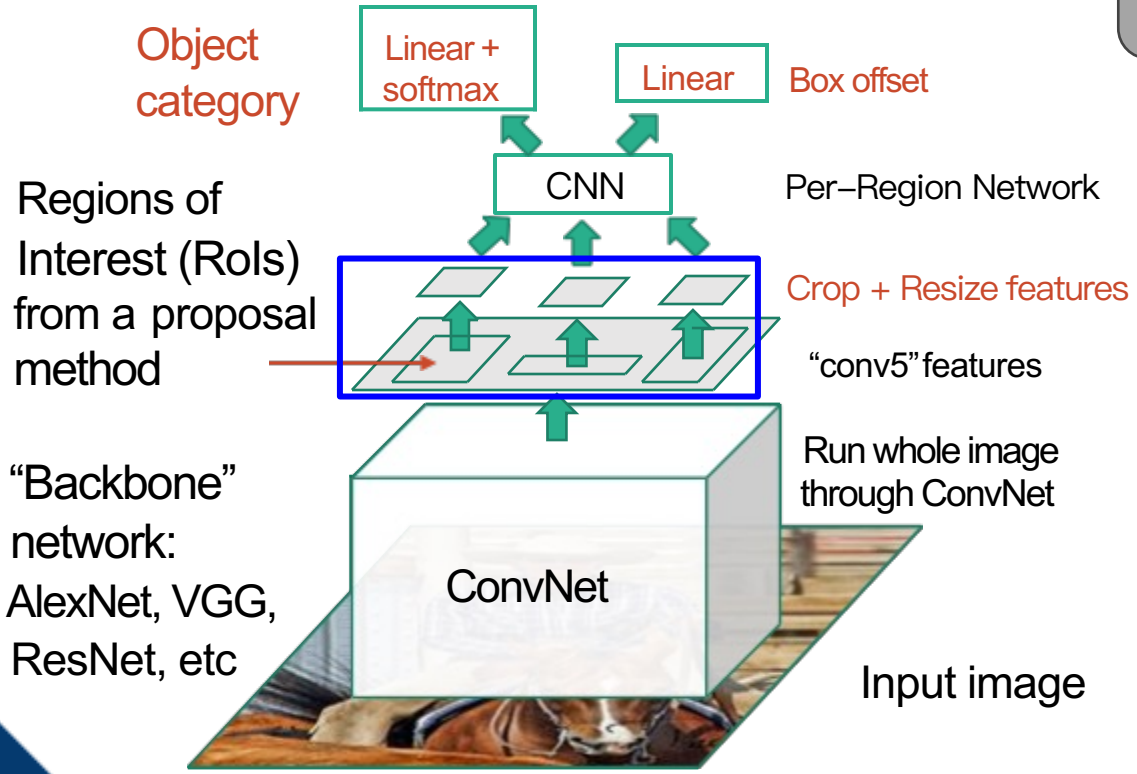
Fast R-CNN





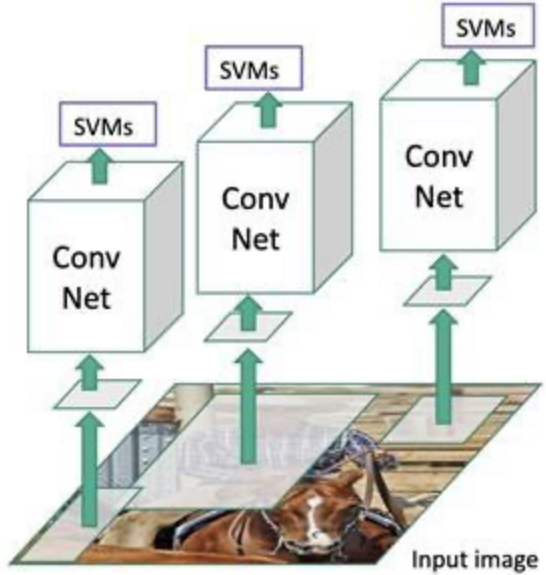
Girshick, "Fast R–CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Fast R-CNN



Limitations of Fast R-CNN?

"Slow" R-CNN



Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Region Proposal Network



Input Image
(e.g. 3 x 640 x 480)



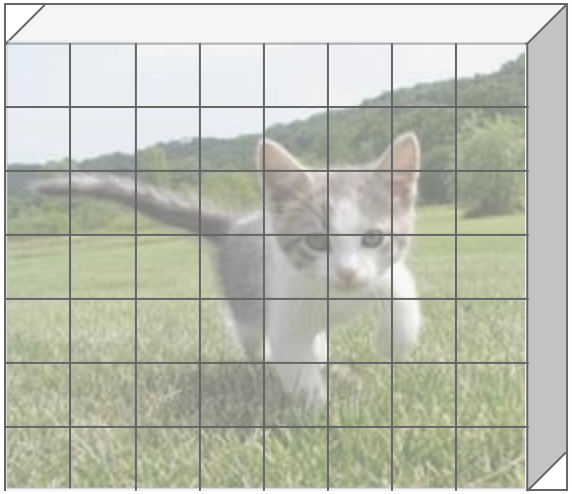


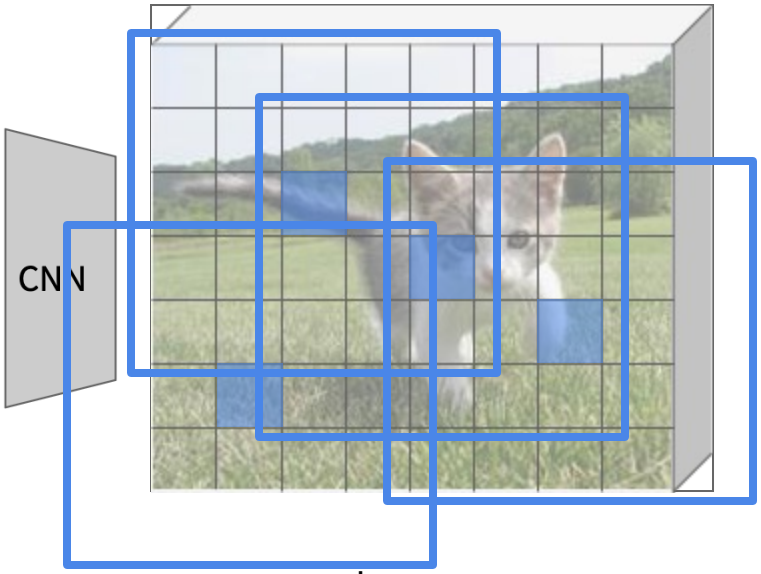
Image
features
(e.g. 512 x 20 x 15)

Region Proposal Network

Imagine an anchor box of fixed size at each point in the feature map



Input Image
(e.g. 3 x 640 x 480)



features
(e.g. 512 x 20 x 15)

Image

Region Proposal Network

Imagine an anchor box of fixed size at each point in the feature map



Input Image
(e.g. 3 x 640 x 480)

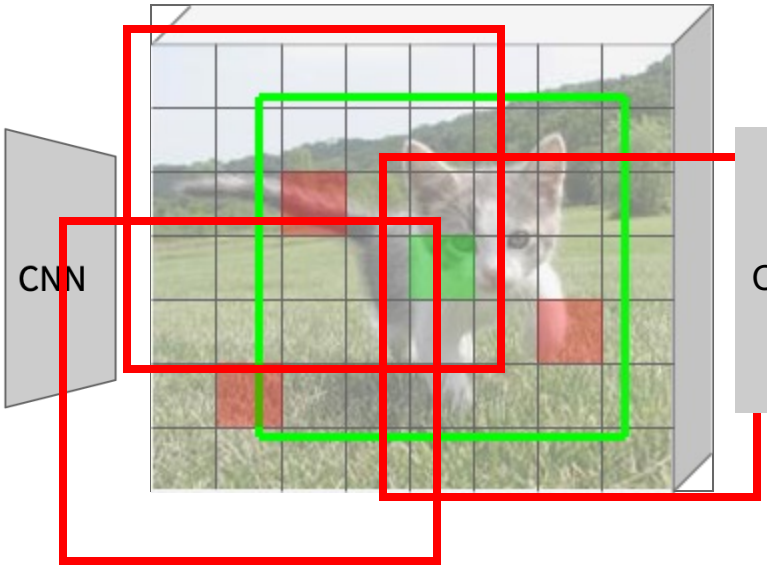


Image features
(e.g. 512 x 20 x 15)

Anchor is an object?
1 x 20 x 15

At each point, predict whether the corresponding anchor contains an object (binary classification)

Region Proposal Network

Imagine an anchor box of fixed size at each point in the feature map



Input Image
(e.g. 3 x 640 x 480)

CNN

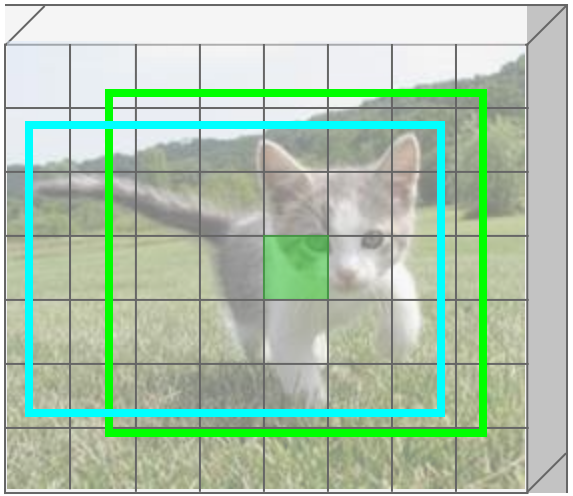


Image features
(e.g. 512 x 20 x 15)

Conv

Anchor is an object?
1 x 20 x 15

Box corrections
4 x 20 x 15
(dx, dy, dw, dh)

For positive boxes, also predict a corrections from the anchor to the ground-truth box (regress 4 numbers per pixel)

Region Proposal Network

In practice use K different anchor boxes of different size / scale at each point



Input Image
(e.g. 3 x 640 x 480)

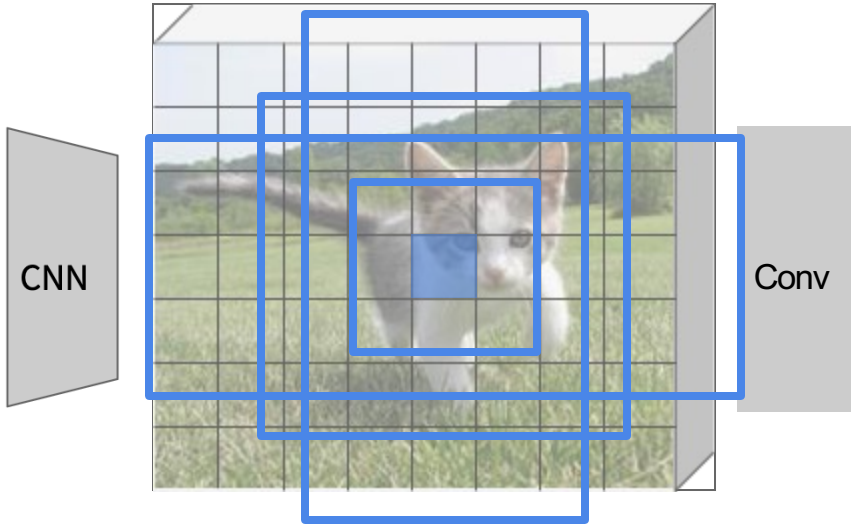


Image features
(e.g. 512 x 20 x 15)

Region Proposal Network

In practice use K different anchor boxes of different size / scale at each point



Input Image
(e.g. 3 x 640 x 480)

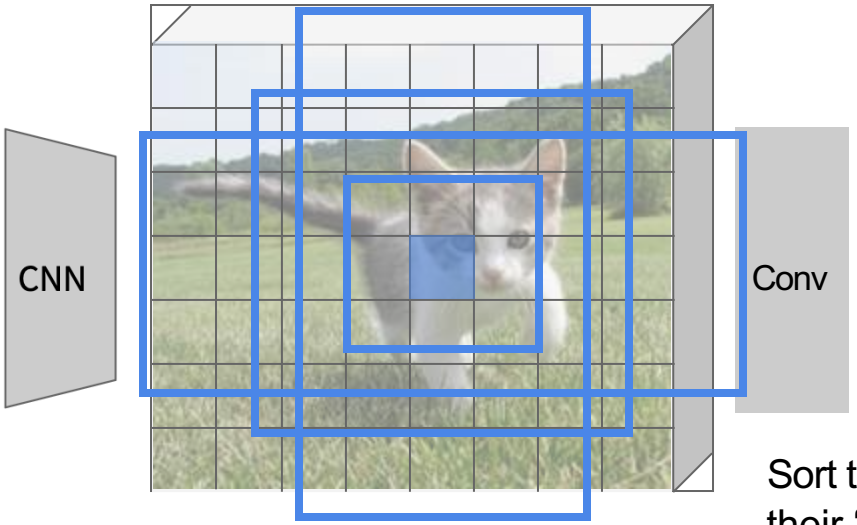
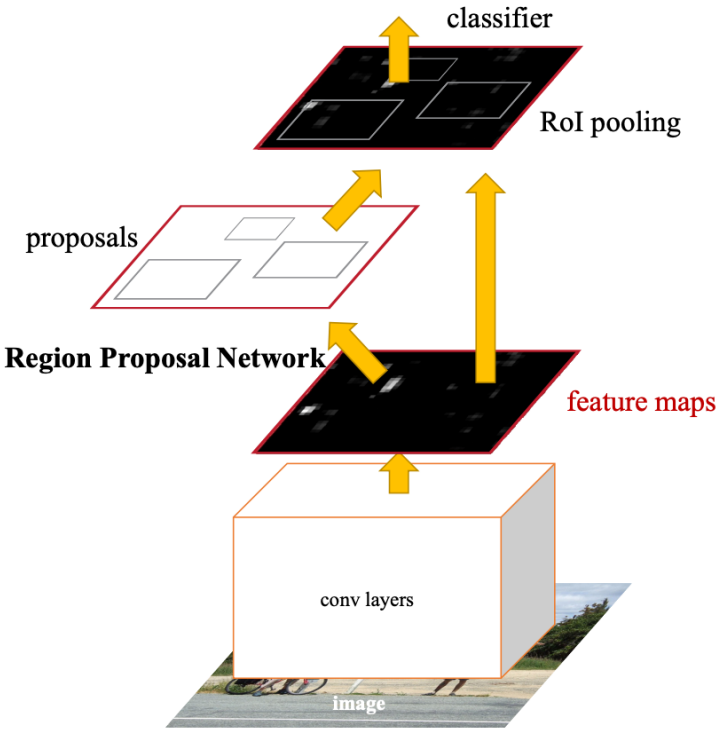


Image features
(e.g. 512 x 20 x 15)

Sort the K*20*15 boxes by their “objectness” score, take top ~300 as our proposals

Faster R-CNN

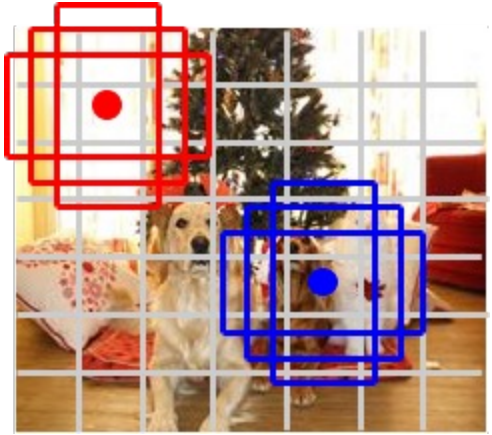


- **The Core Innovation: "Shared Convolutional Features"**
- **The Problem:** Previous methods (Fast R-CNN) relied on external, slow proposal methods (Selective Search) running on CPU.
- **The Solution:** Integrate a **Region Proposal Network (RPN)** that shares the same backbone features with the **Fast R-CNN detector**.
- **Result:** The cost of generating proposals is nearly **zero** (approx. 10ms per image).

Single-Stage Object Detectors: YOLO / SSD / RetinaNet



Input image
3 x H x W



Divide image into grid
7 x 7

a set of base boxes
centered at each grid cell
Here B = 3

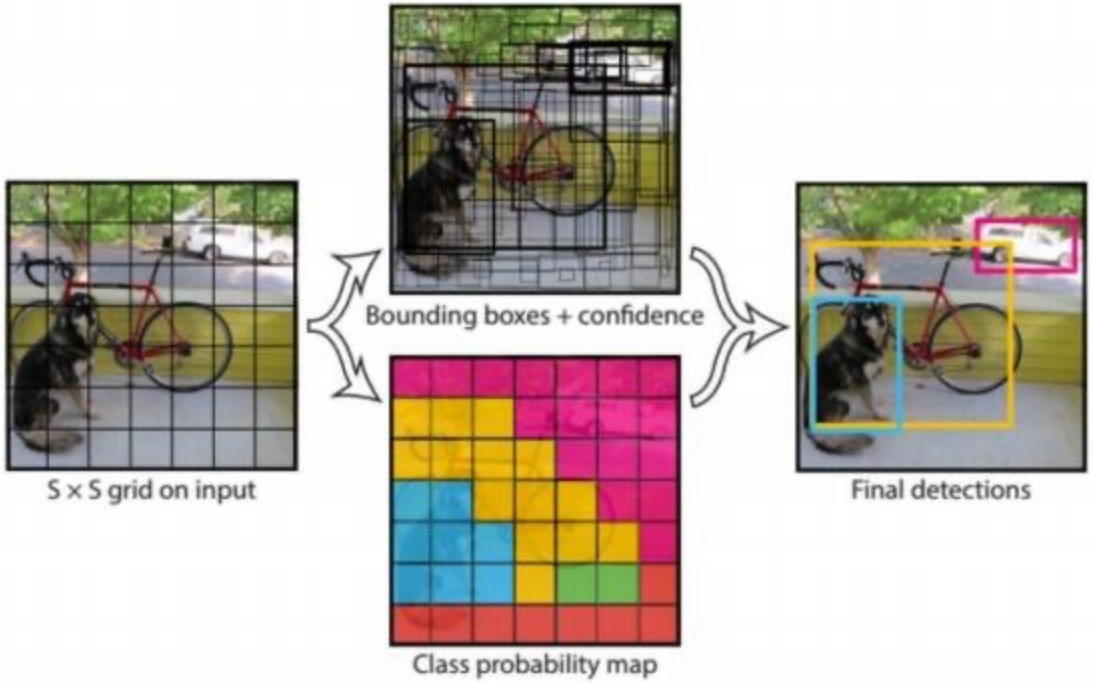
Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers:
  (dx, dy, dh, dw, confidence)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

Output:
7 x 7 x (5 * B + C)

Redmon et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

YOLO (You Only Look Once) real-time object detection



Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO



SxS Grid



Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO



SxS
Grid

For each box output:

- P(object): probability that the box contains an object
- B bounding boxes (x, y, h, w)
- P(class): probability of belonging to a class



Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO



SxS
Grid

For each box output:

- P(object): probability that the box contains an object
- B bounding boxes (x, y, h, w)
- P(class): probability of belonging to a class

B=2



Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO



SxS
Grid

For each box output:

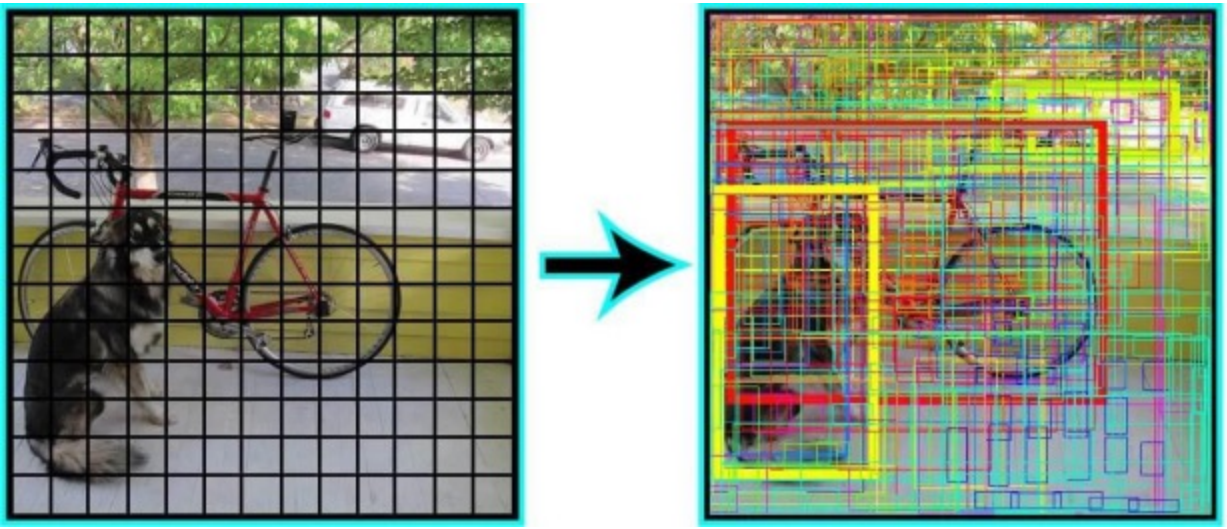
- P(object): probability that the box contains an object
- B bounding boxes (x, y, h, w)
- P(class): probability of belonging to a class

B=2



Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO



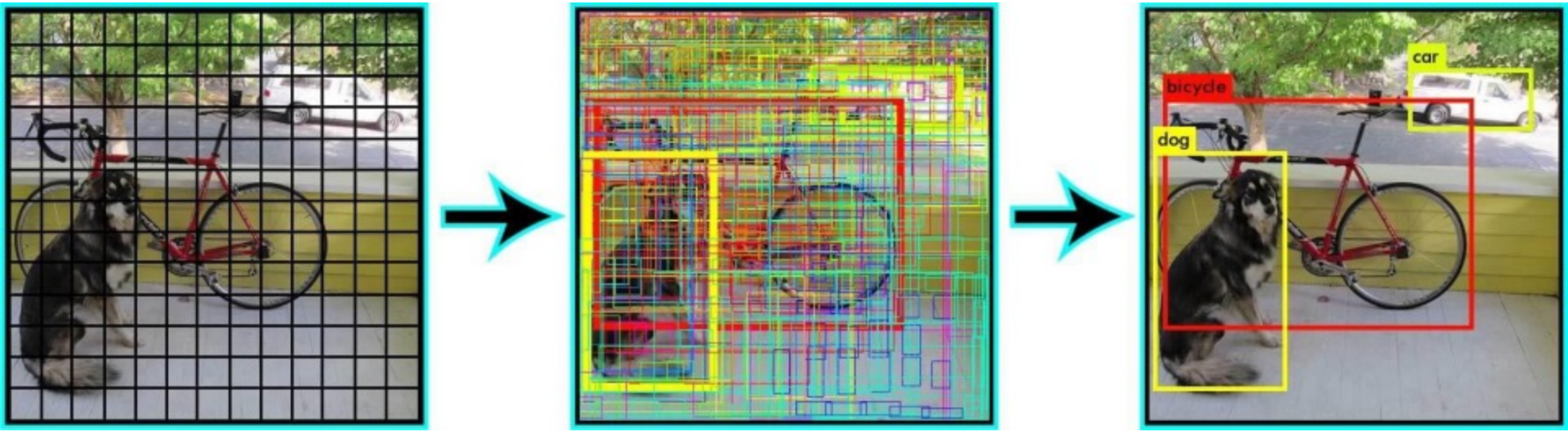
SxS Grid

Many bounding boxes with different object probabilities



Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO



SxS Grid

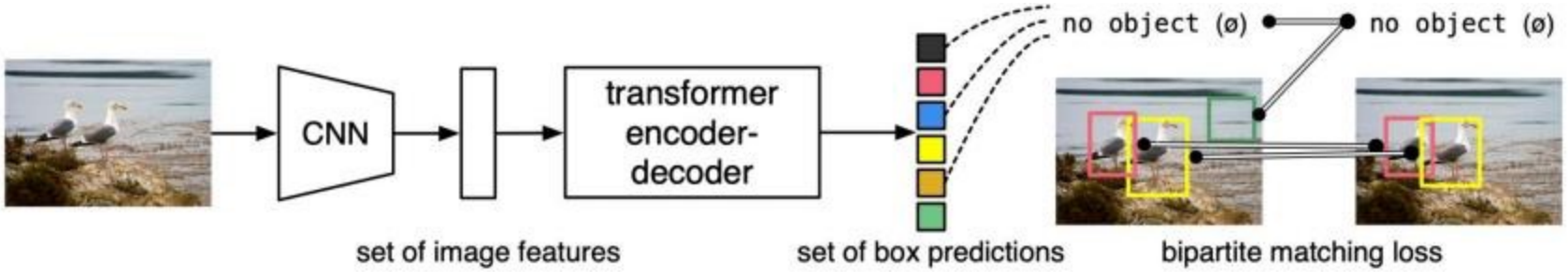
Redmon et al. "You only look once: unified, real-time object detection (2015)."

Object **De**tection with **Tr**ansformers: DETR

Simple object detection pipeline: directly output a set of boxes from a Transformer

No anchors, no regression of box transforms

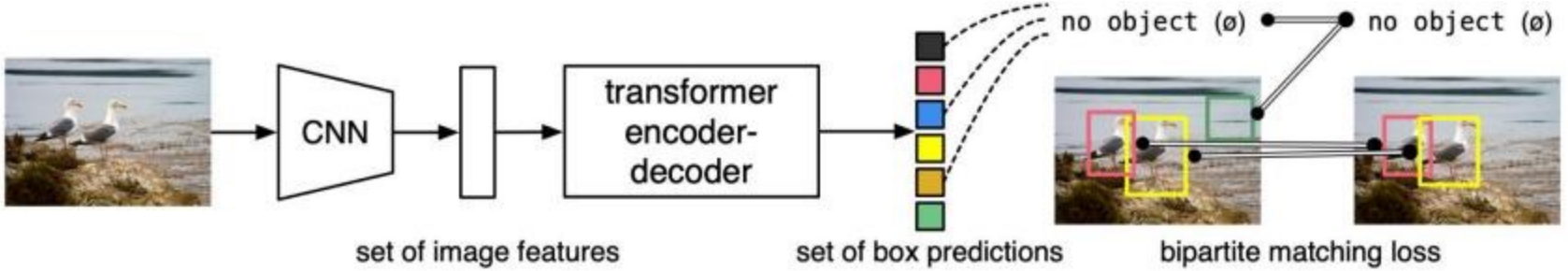
Match predicted boxes to GT boxes with bipartite matching; train to regress box coordinates

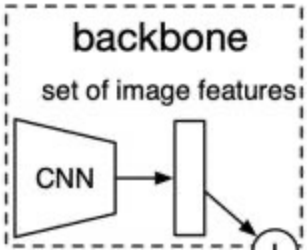




Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

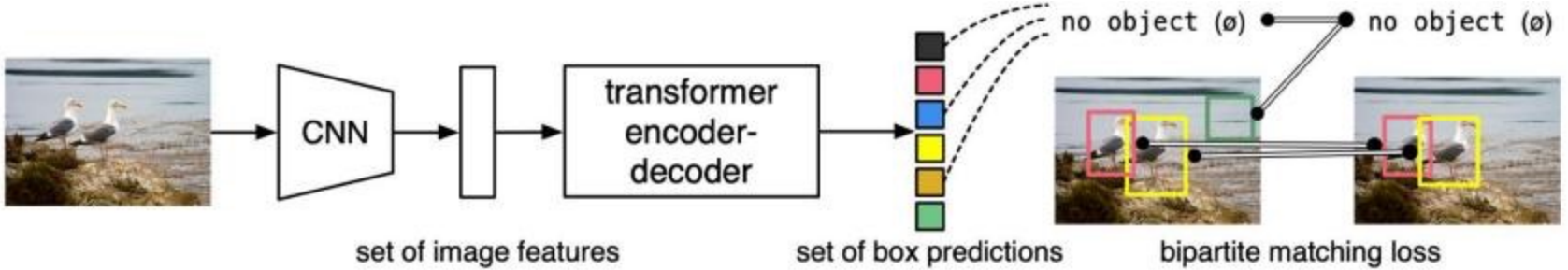
Object Detection with Transformers: DETR

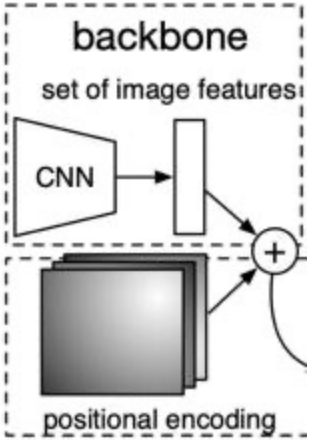




Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

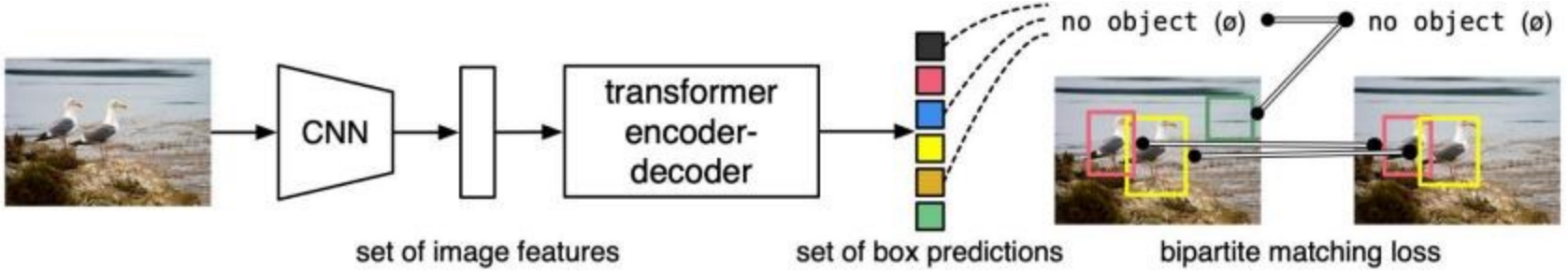
Object Detection with Transformers: DETR

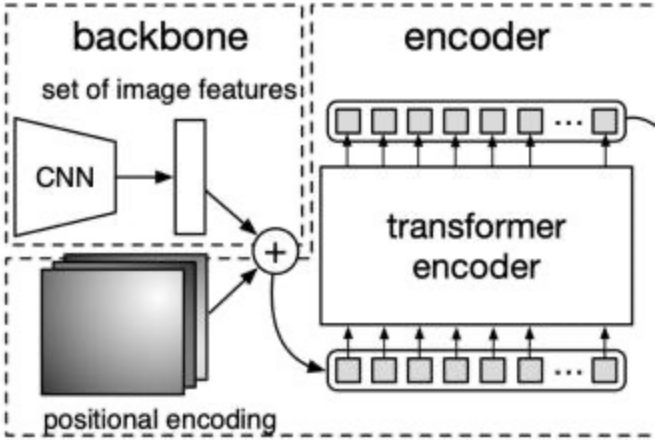




Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

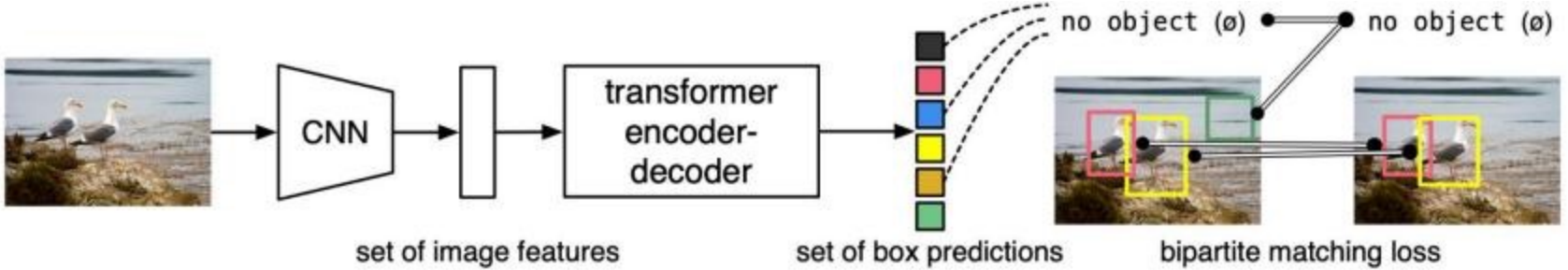
Object Detection with Transformers: DETR

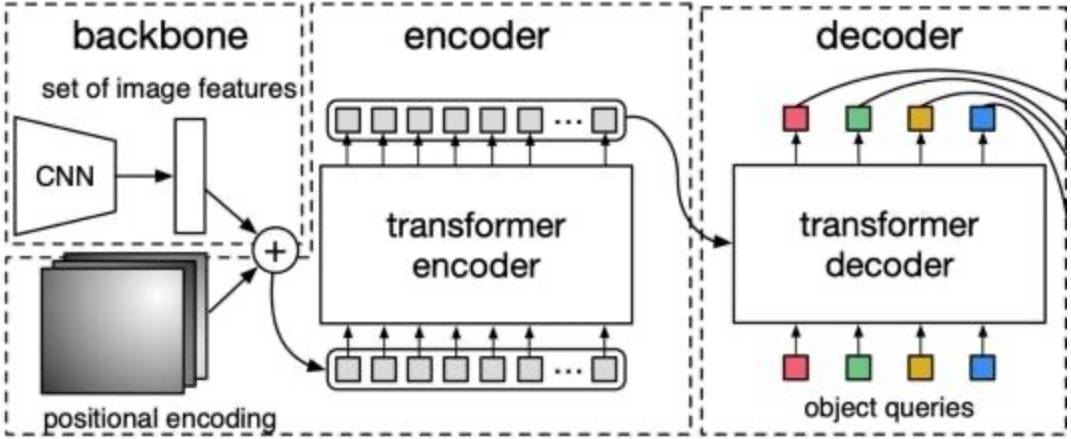




Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

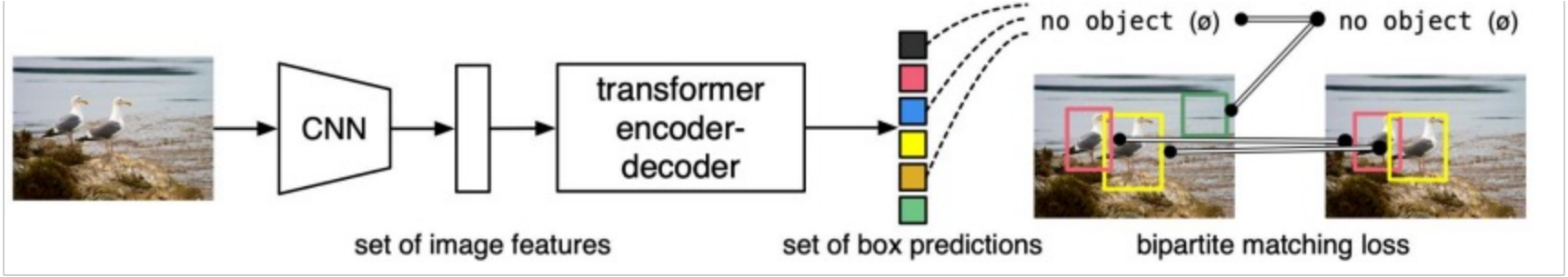
Object Detection with Transformers: DETR

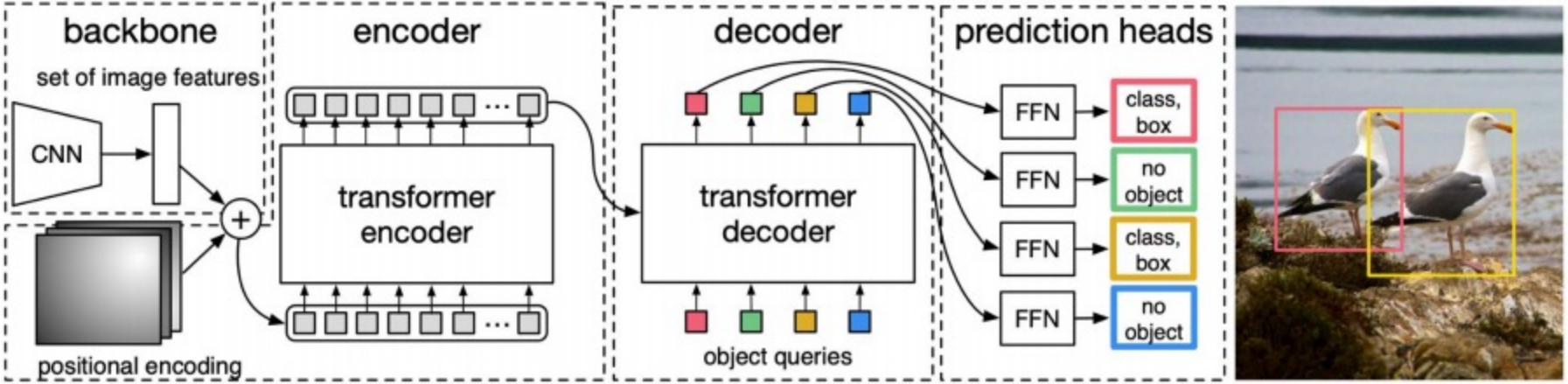




Object Detection with Transformers: DETR

Set Prediction + Hungarian Matching





Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

Why DETR?

Previous Methods:

- Complex design (anchors, IoU thresholds, NMS)
- Heavily engineered components
- Duplicate predictions → post-processing needed
- Limited global reasoning (mostly local receptive fields)

DETR:

- Reformulates detection as set prediction: CNN backbone → Transformer → Object Queries
- Key features:
  - No anchors
  - No NMS
  - End-to-end training



Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

Today

- **Computer Vision Tasks**
  - Semantic Segmentation
  - Object Detection
  - **Instance Segmentation**

- Visualization & Understanding
  - Model Layers Visualization
  - Saliency Maps
  - CAM & Grad-CAM

- Foundation Models and Prompt Segmentation



Instance Segmentation

Classification

CAT

No spatial extent

Semantic Segmentation

GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection

DOG, DOG, CAT

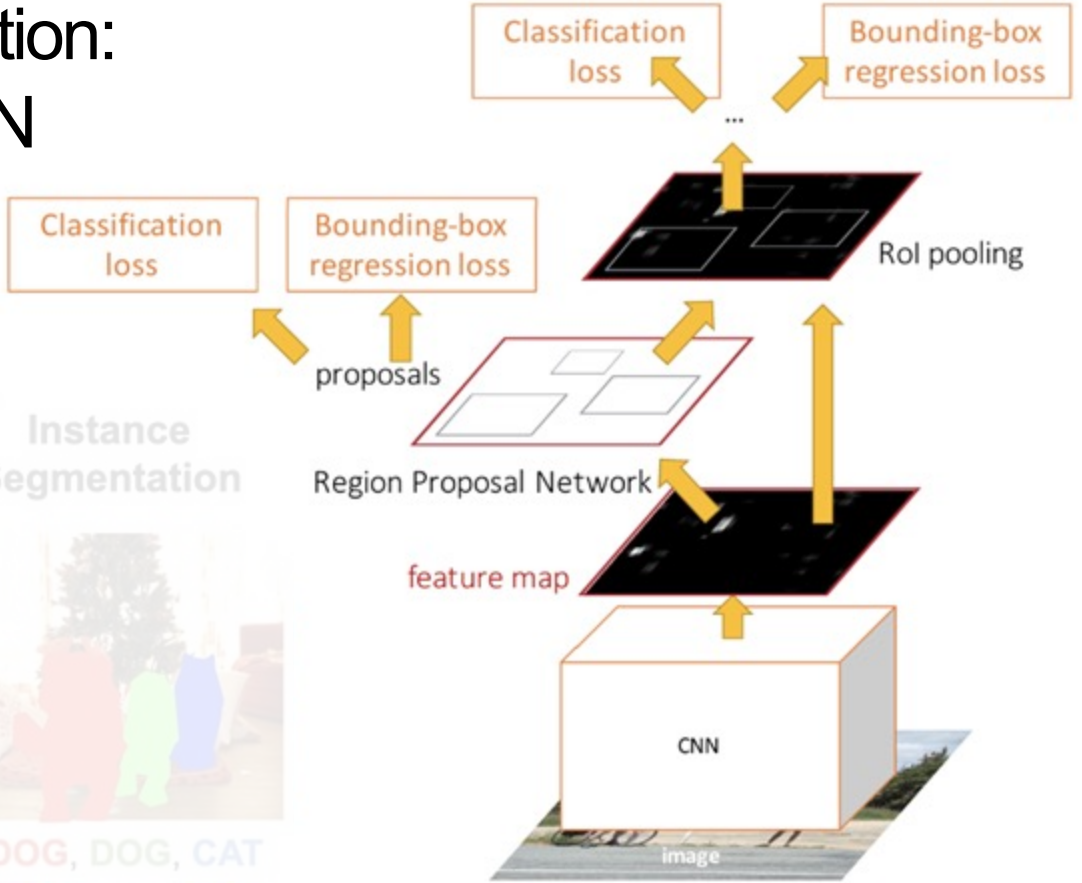
Instance Segmentation



DOG, DOG, CAT

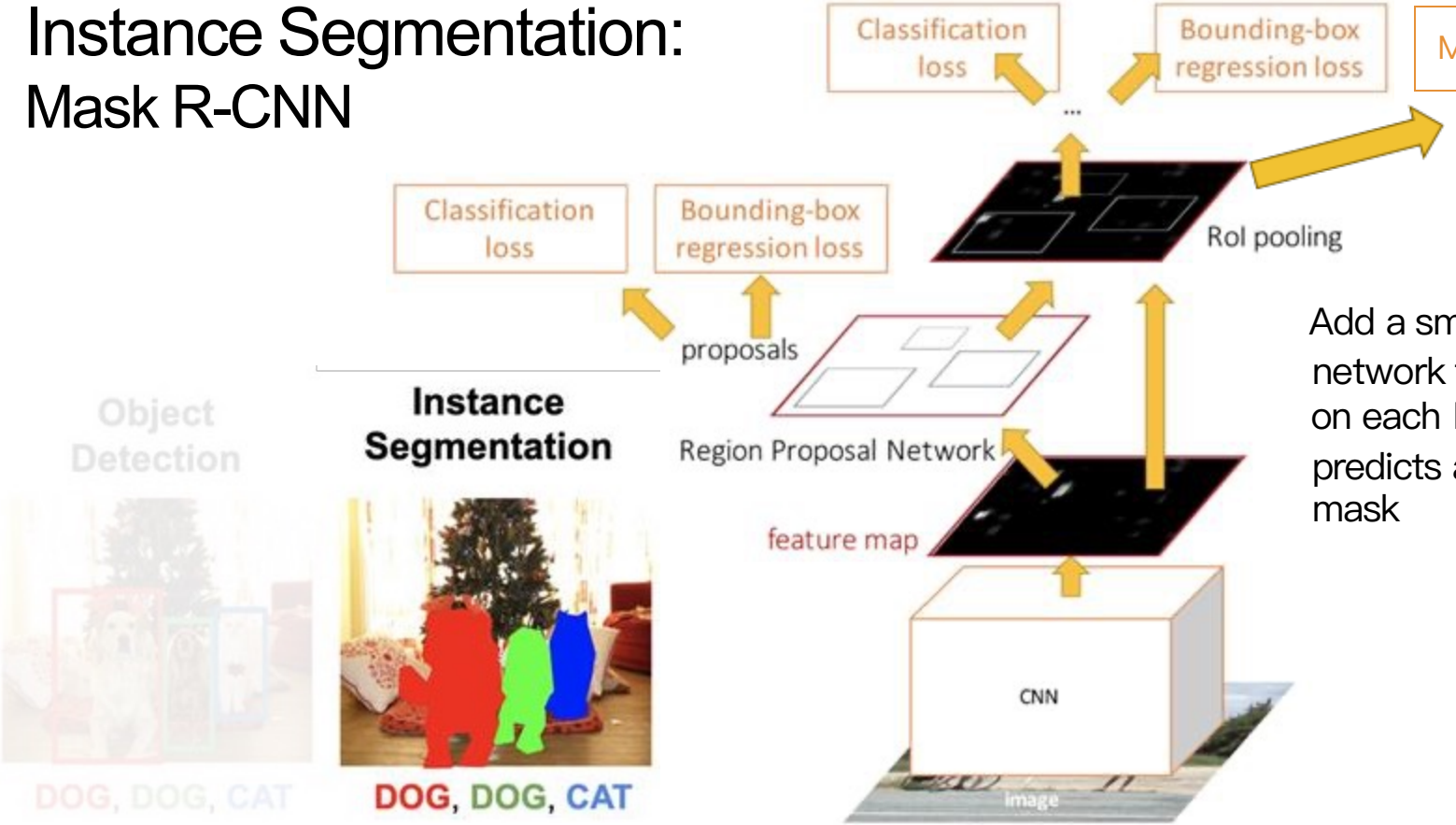
Multiple Object

Object Detection: Faster R-CNN





Instance Segmentation:
Mask R-CNN

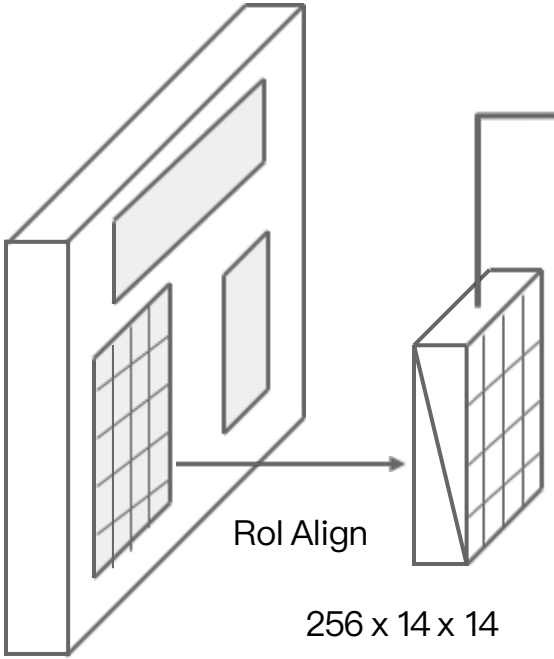


Add a small mask network that operates on each RoI and predicts a 28x28 binary mask

Mask R-CNN



CNN
+RPN



RoI Align

256 x 14 x 14

Classification Scores: C

Box coordinates (per class): 4 * C

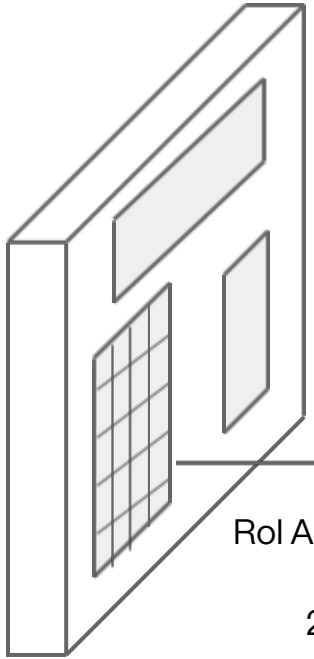


He et al, “Mask R-CNN”, ICCV 2017

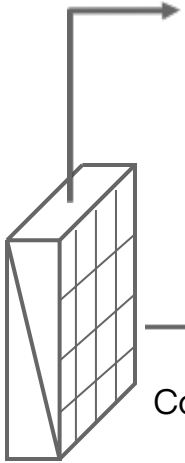
Mask R-CNN



CNN
+RPN



RoI Align

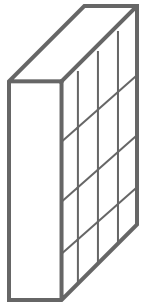


256 x 14 x 14

Classification Scores: C

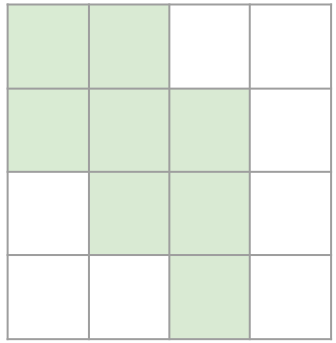
Box coordinates (per class): 4 * C

Conv



256 x 14 x 14

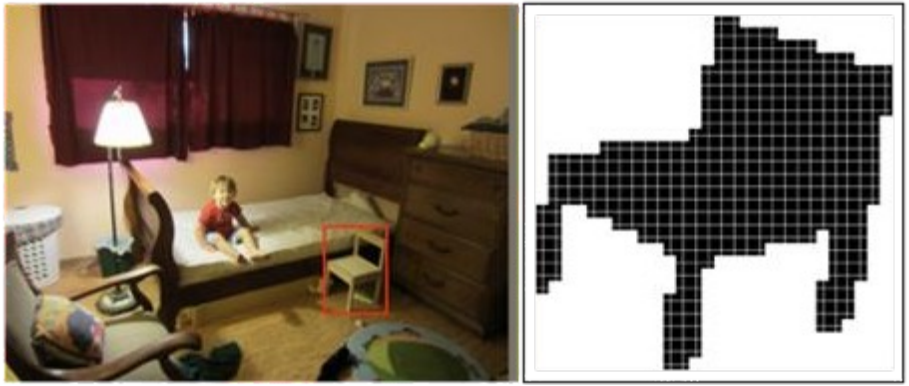
Conv



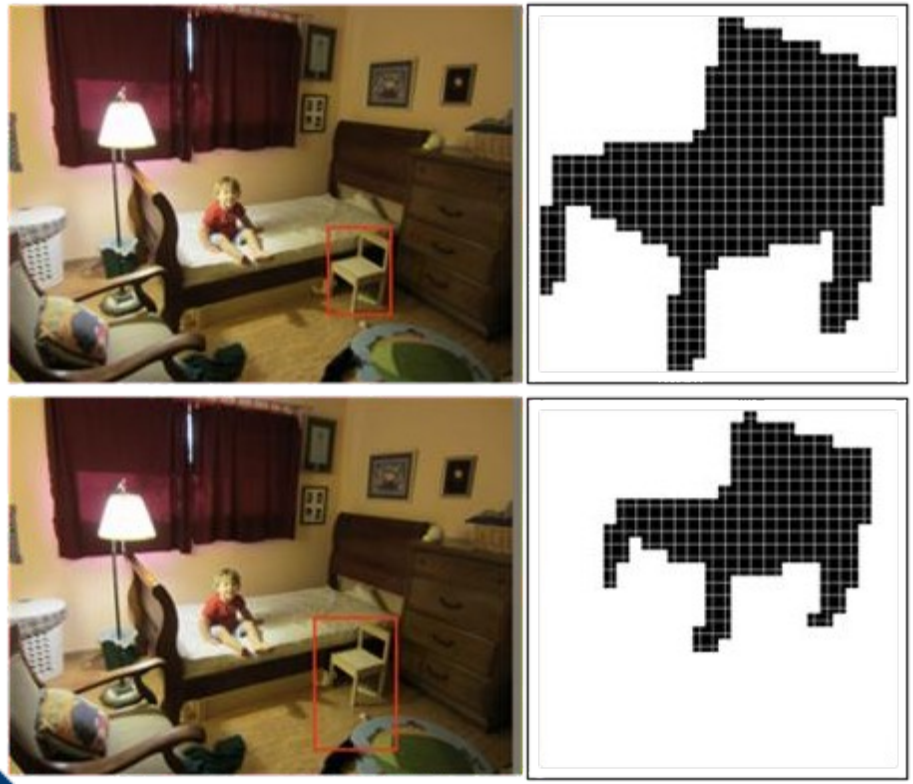
Predict a mask for each of C classes

Cx 28 x 28

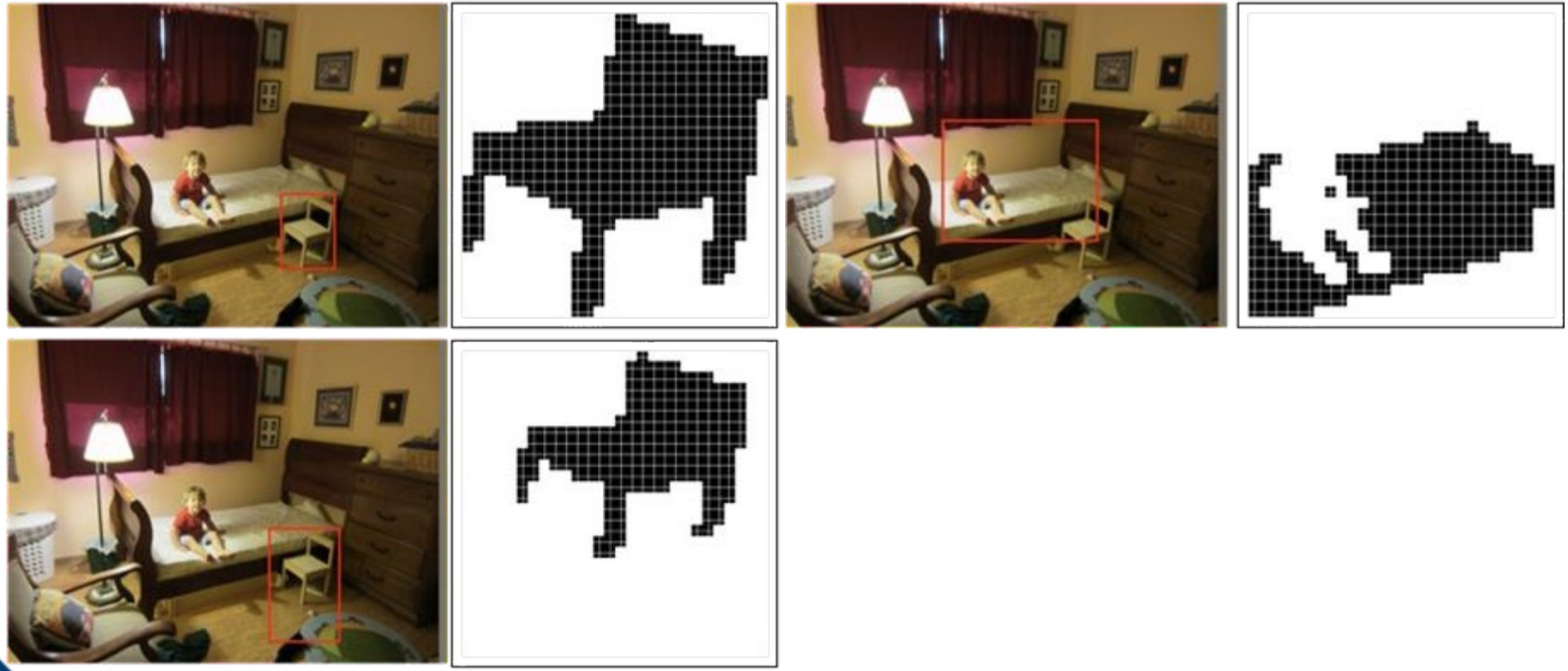
Mask R-CNN: Example Mask Training Targets



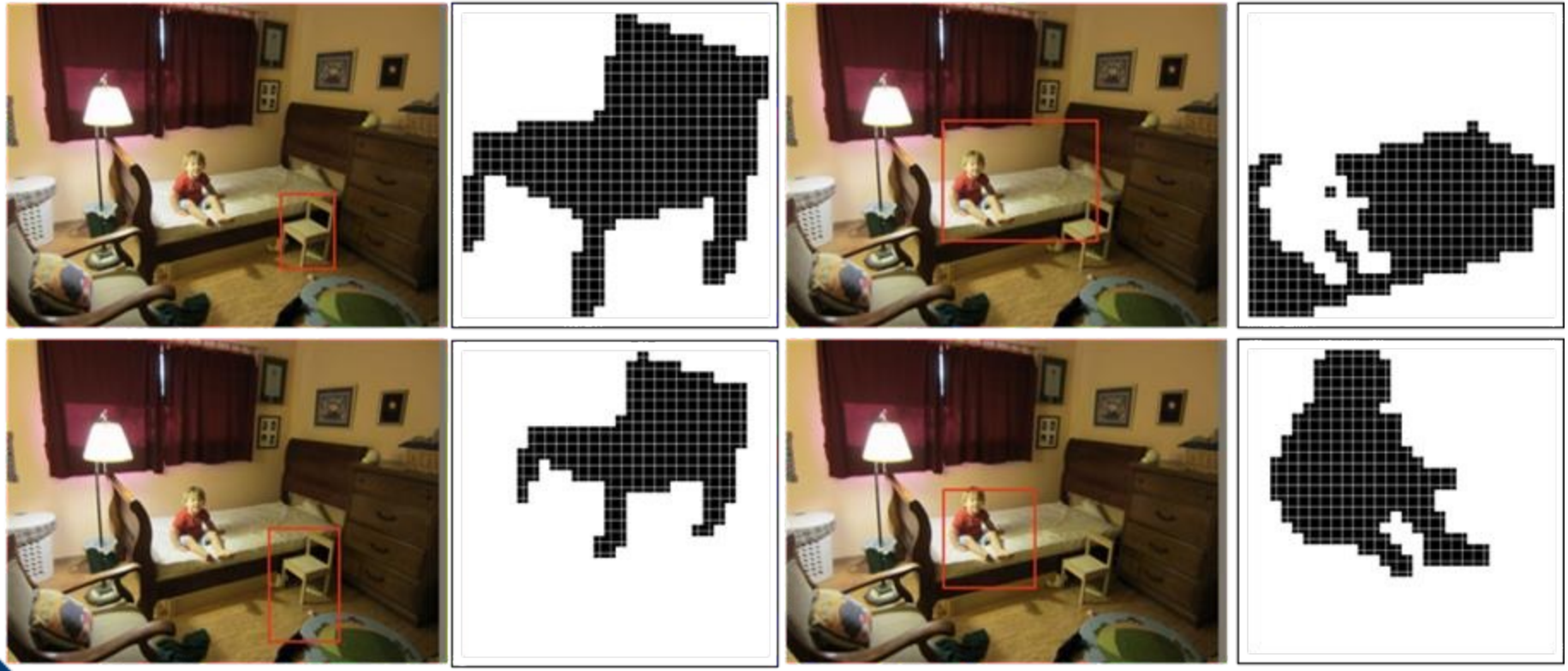
Mask R-CNN: Example Mask Training Targets



Mask R-CNN: Example Mask Training Targets

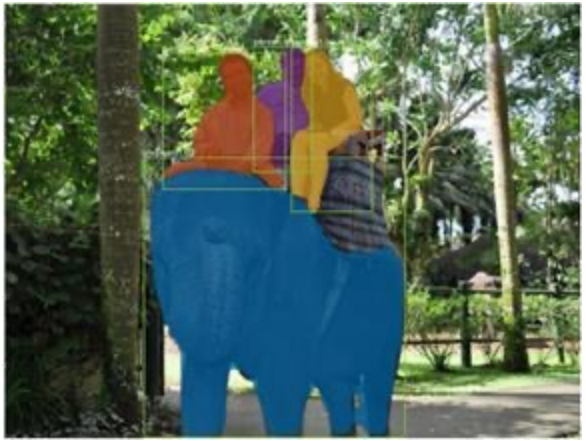


Mask R-CNN: Example Mask Training Targets



Mask R-CNN: Very Good Results!





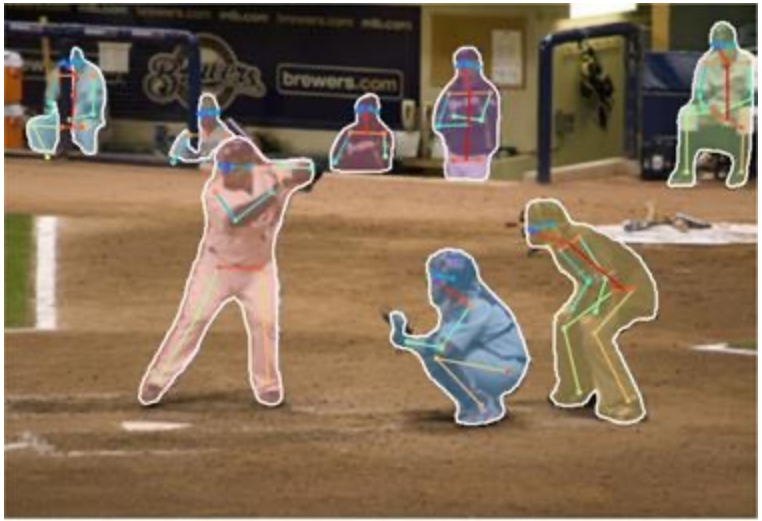




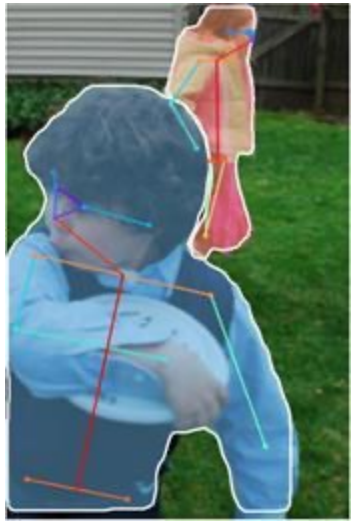
He et al, “Mask R-CNN”, ICCV 2017

Mask R-CNN

Also does pose







Recap: Lots of computer vision tasks!

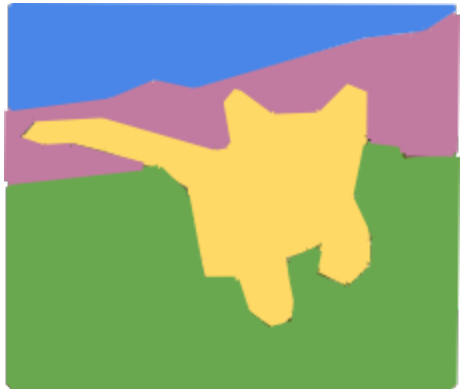
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple Object

This image is CC0 public domain

Today

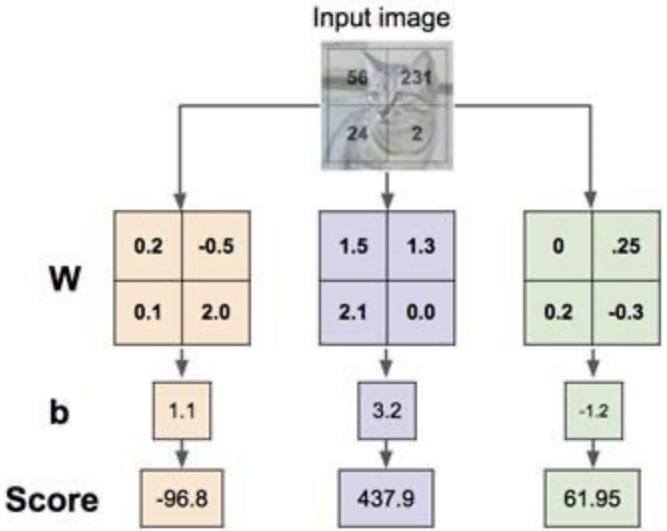
- Computer Vision Tasks
  - Semantic Segmentation
  - Object Detection
  - Instance Segmentation

- **Visualization & Understanding**
  - **Model Layers Visualization**
  - Saliency Maps
  - CAM & Grad-CAM



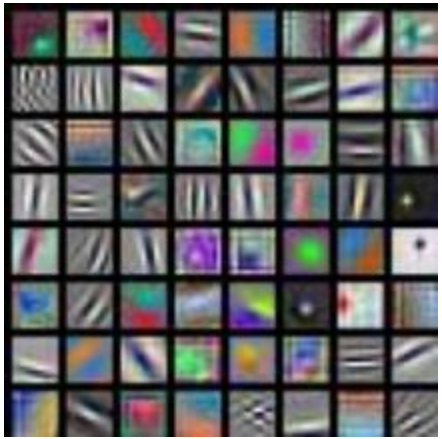
Interpreting a Linear Classifier: Visual Viewpoint



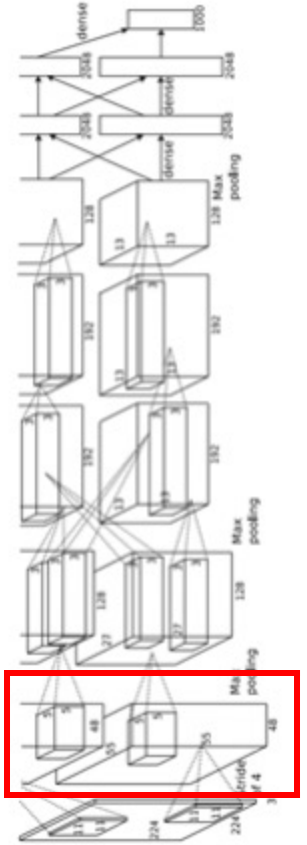




First Layer: Visualize Filters



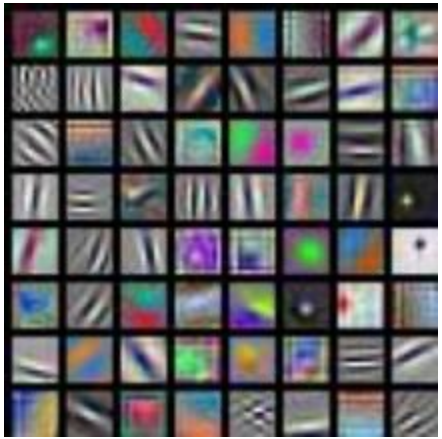
AlexNet:
64 x 3 x 11 x 11



Krizhevsky , “One weird trick for parallelizing convolutional neural networks”, arXiv 2014 He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

Huang et al, “Densely Connected Convolutional Networks”, CVPR 2017

First Layer: Visualize Filters



AlexNet:
64 x 3 x 11 x 11



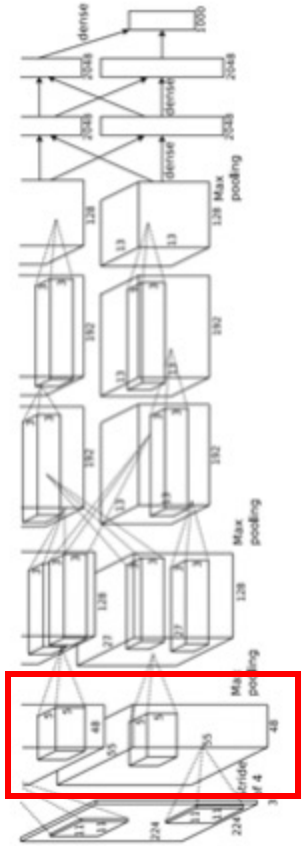
ResNet-18:
64 x 3 x 7 x 7



ResNet-101:
64 x 3 x 7 x 7



DenseNet-121:
64 x 3 x 7 x 7



Krizhevsky , “One weird trick for parallelizing convolutional neural networks”, arXiv 2014 He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016
Huang et al, “Densely Connected Convolutional Networks”, CVPR 2017

Today

- Computer Vision Tasks
  - Semantic Segmentation
  - Object Detection
  - Instance Segmentation

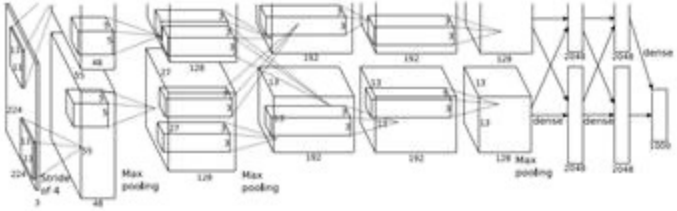
- **Visualization & Understanding**
  - Model Layers Visualization
  - **Saliency Maps**
  - CAM & Grad-CAM



Which pixels matter: Saliency via Backprop

Forward pass: Compute probabilities





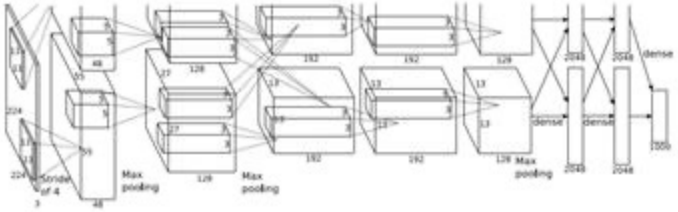
Dog

Simonyan, Vedaldi, and Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Salien cy Maps”, ICLR Workshop 2014.
Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.

Which pixels matter: Saliency via Backprop

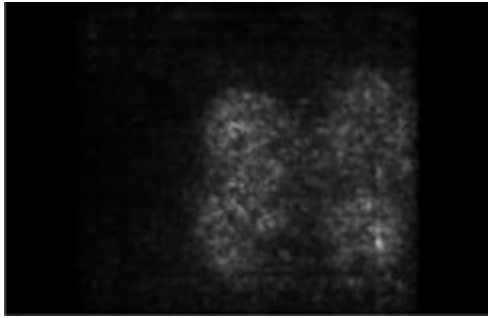
Forward pass: Compute probabilities





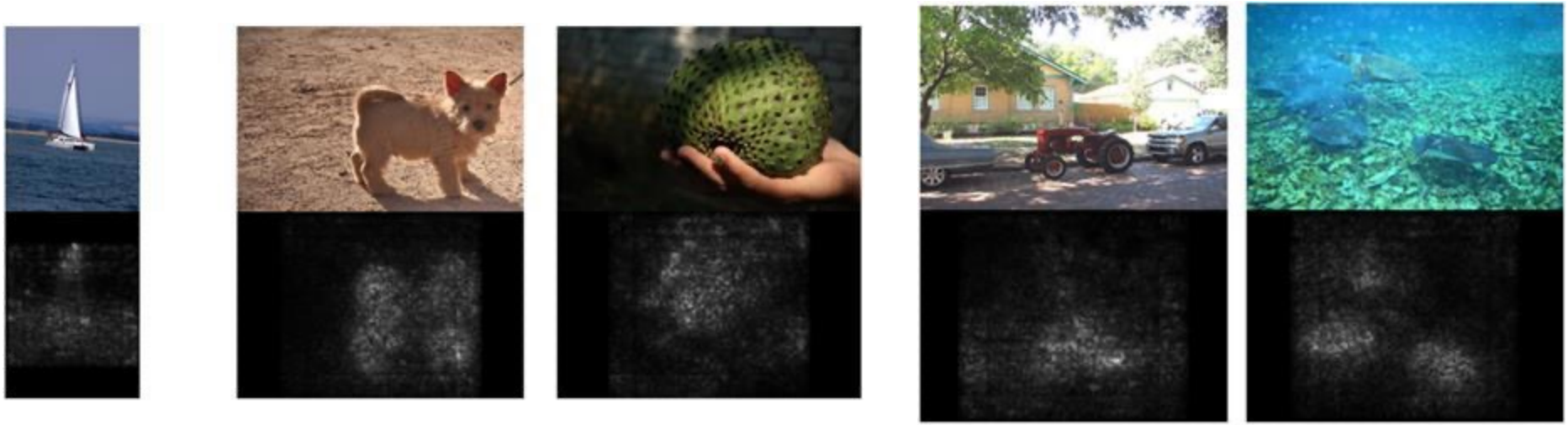
Dog

Compute gradient of (unnormalized) class score with respect to image pixels, take absolute value and max over RGB channels



Simonyan, Vedaldi, and Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Salien cy Maps", ICLR Workshop 2014.
Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.

Saliency Maps



Simonyan, Vedaldi, and Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Salien cy Maps”, ICLR Workshop 2014.
Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.

Today

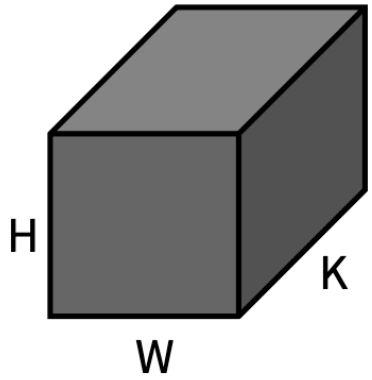
- Computer Vision Tasks
  - Semantic Segmentation
  - Object Detection
  - Instance Segmentation

- **Visualization & Understanding**
  - Model Layers Visualization
  - Saliency Maps
  - **CAM & Grad-CAM**

- Foundation Models and Prompt Segmentation

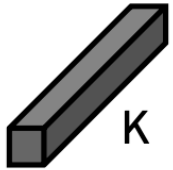


Class Activation Mapping (CAM)



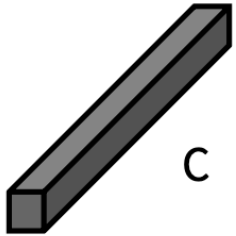
Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Global Average Pooling



Pooled features:
$F \in \mathbb{R}^{K}$

Fully Connected Layer, weights
$w \in \mathbb{R}^{K \times C}$

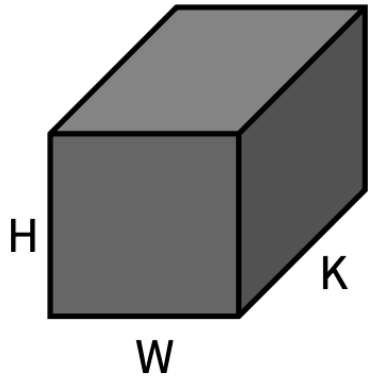


Class Scores:
$S \in \mathbb{R}^{C}$

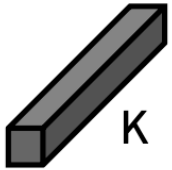


Zhou et al, "Learning Deep Features for Discriminative Localization", CVPR 2016

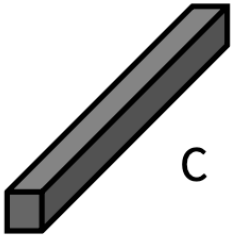
Class Activation Mapping (CAM)



Global Average Pooling



Fully Connected Layer, weights $w \in \mathbb{R}^{K \times C}$



Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
$F \in \mathbb{R}^{K}$

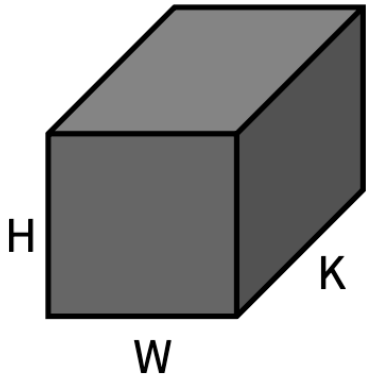
Class Scores:
$S \in \mathbb{R}^{C}$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k}$$

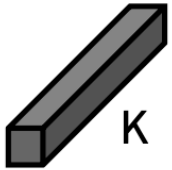


Zhou et al, "Learning Deep Features for Discriminative Localization", CVPR 2016

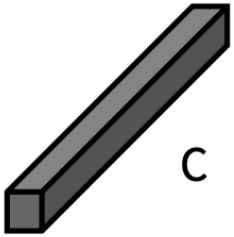
Class Activation Mapping (CAM)



Global Average Pooling



Fully Connected Layer, weights $w \in \mathbb{R}^{K \times C}$



Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
$F \in \mathbb{R}^{K}$

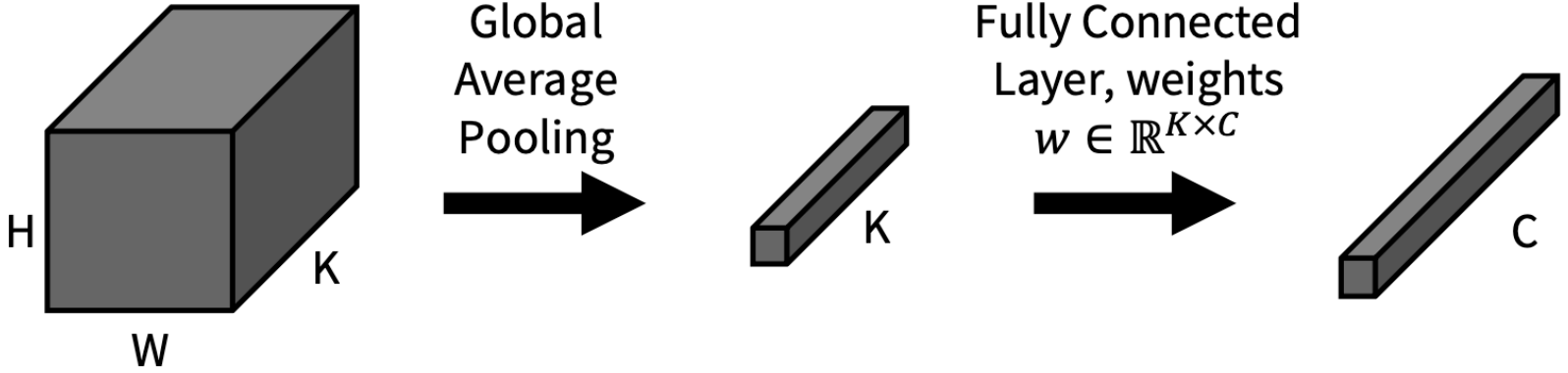
Class Scores:
$S \in \mathbb{R}^{C}$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \qquad S_c = \sum_k w_{k,c} F_k$$



Zhou et al, "Learning Deep Features for Discriminative Localization", CVPR 2016

Class Activation Mapping (CAM)



Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
$F \in \mathbb{R}^{K}$

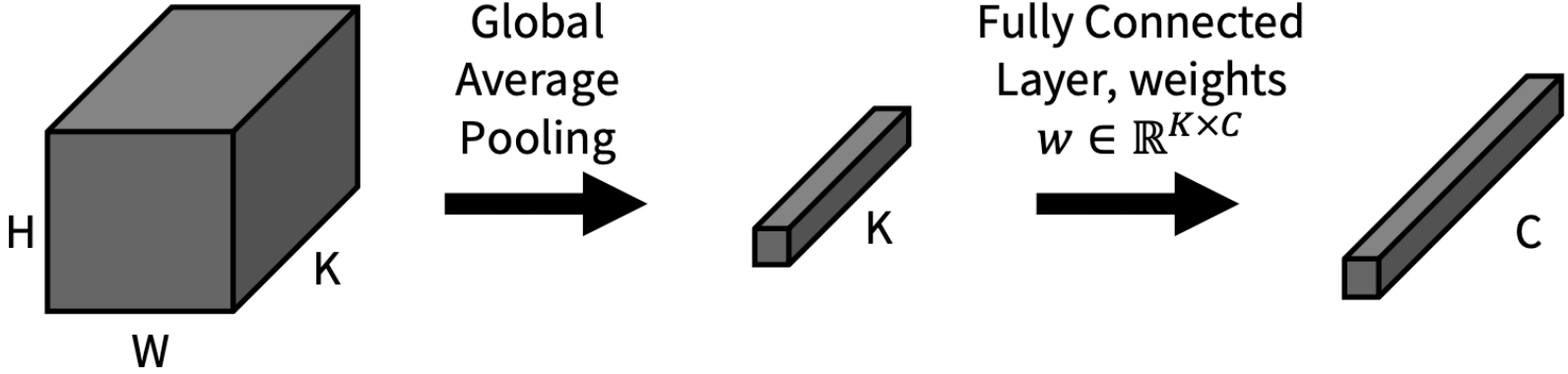
Class Scores:
$S \in \mathbb{R}^{C}$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \qquad S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$



Zhou et al, “Learning Deep Features for Discriminative Localization”, CVPR 2016

Class Activation Mapping (CAM)



Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
$F \in \mathbb{R}^{K}$

Class Scores:
$S \in \mathbb{R}^{C}$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k}$$

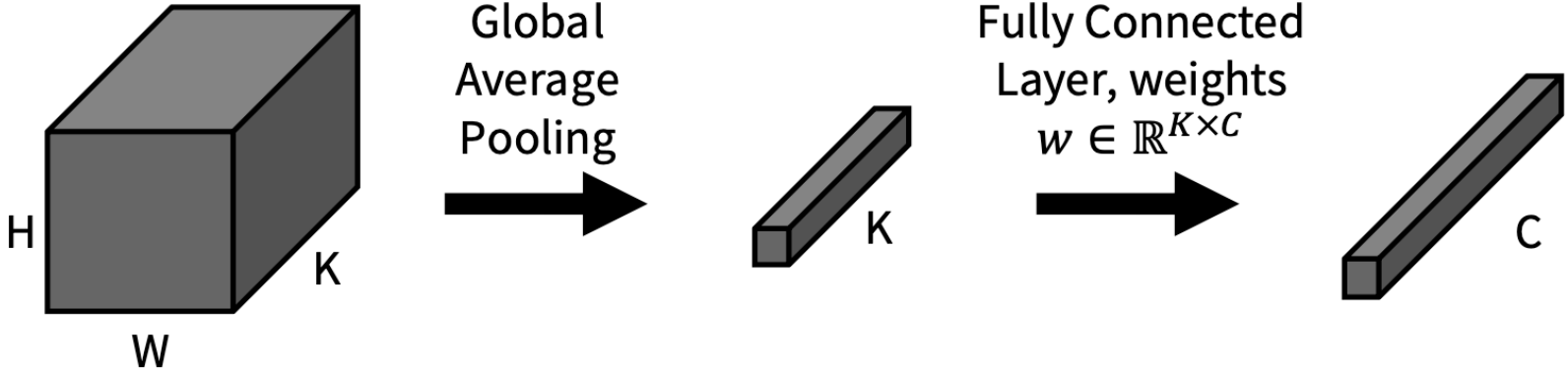
$$S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$

$$= \frac{1}{HW} \sum_{h,w} \sum_k w_{k,c} f_{h,w,k}$$



Zhou et al, “Learning Deep Features for Discriminative Localization”, CVPR 2016

Class Activation Mapping (CAM)



Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
$F \in \mathbb{R}^{K}$

Class Scores:
$S \in \mathbb{R}^{C}$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k}$$

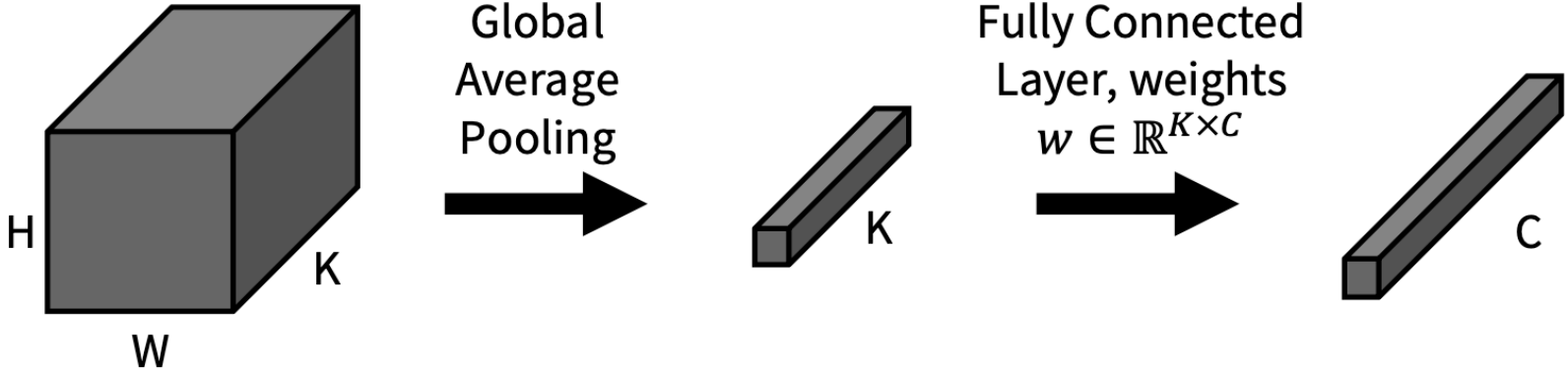
$$S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$

$$= \frac{1}{HW} \sum_{h,w} \sum_k w_{k,c} f_{h,w,k}$$



Zhou et al, "Learning Deep Features for Discriminative Localization", CVPR 2016

Class Activation Mapping (CAM)



Last layer CNN features:
$f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
$F \in \mathbb{R}^{K}$

Class Scores:
$S \in \mathbb{R}^{C}$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k}$$

$$S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$

$$= \frac{1}{HW} \sum_{h,w} \sum_k w_{k,c} f_{h,w,k}$$

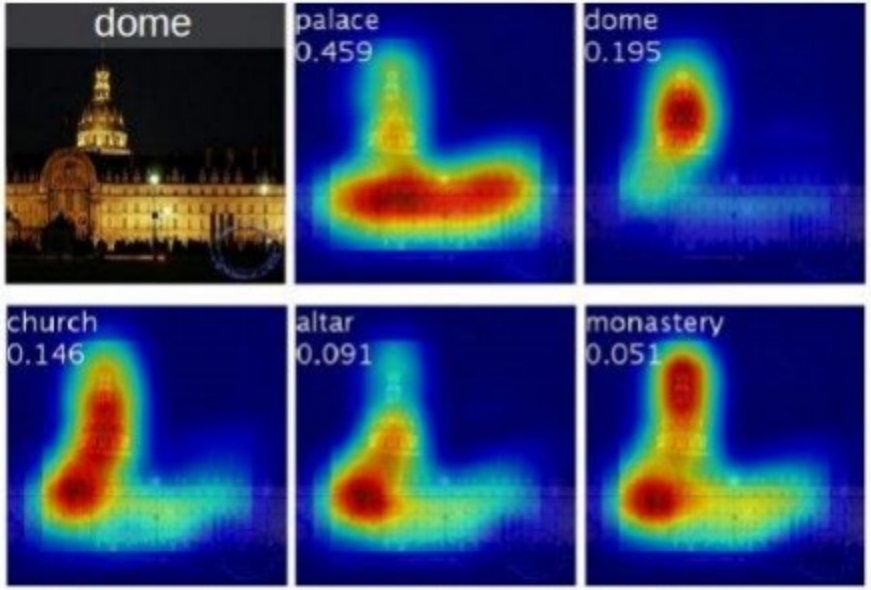
Class Activation Maps:
$M \in \mathbb{R}^{C,H,W}$

$$M_{c,h,w} = \sum_k w_{k,c} f_{h,w,k}$$



Zhou et al, "Learning Deep Features for Discriminative Localization", CVPR 2016

Class Activation Mapping (CAM)



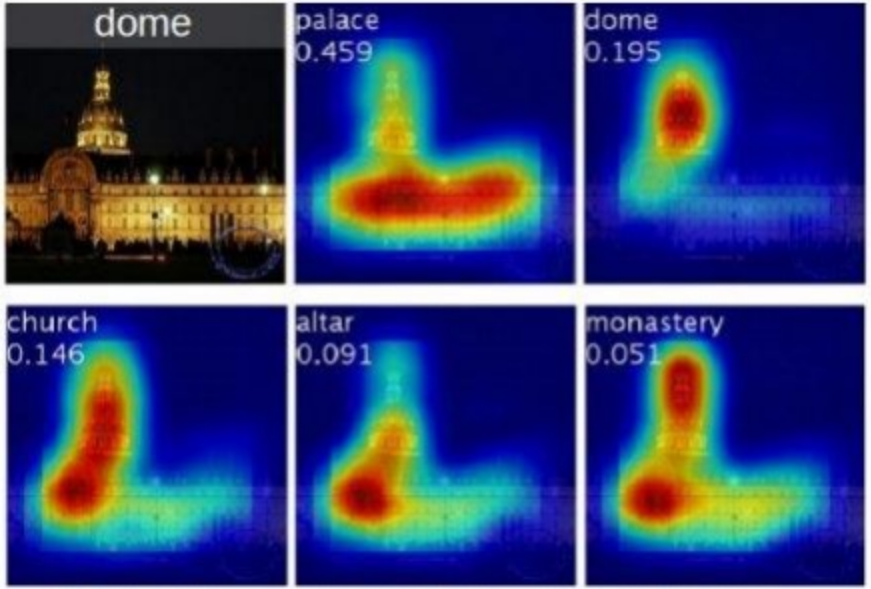
Class activation maps of top 5 predictions



Class activation maps for one object class

Class Activation Mapping (CAM)

Problem: Can only apply to last conv layer



Class activation maps of top 5 predictions



Class activation maps for one object class

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times w \times k}$



Selvaraju et al, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization”, CVPR 2017

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times K}$
2. Compute gradient of class score S_c with respect to A:

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times K}$$



Selvaraju et al, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization”, CVPR 2017

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times k}$
2. Compute gradient of class score S_c with respect to A:

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times k}$$

3. Global Average Pool the gradients to get weights $\alpha \in \mathbb{R}^k$:

$$\alpha_k = \frac{1}{HW} \sum_{h,w} \frac{\partial S_c}{\partial A_{h,w,k}}$$



Selvaraju et al, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", CVPR 2017

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times k}$
2. Compute gradient of class score S_c with respect to A:

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times k}$$

3. Global Average Pool the gradients to get weights $\alpha \in \mathbb{R}^{k}$:

$$\alpha_k = \frac{1}{HW} \sum_{h,w} \frac{\partial S_c}{\partial A_{h,w,k}}$$

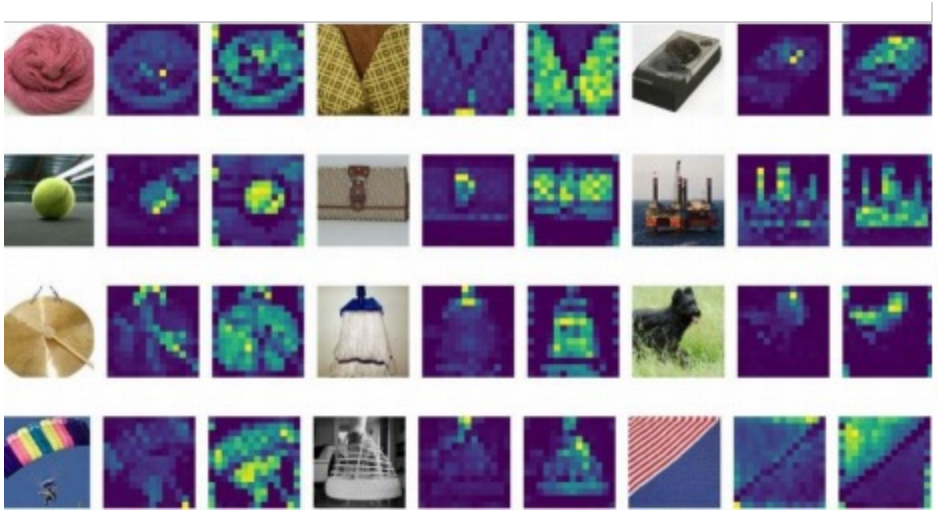
4. Compute activation map $M^c \in \mathbb{R}^{H,w}$:

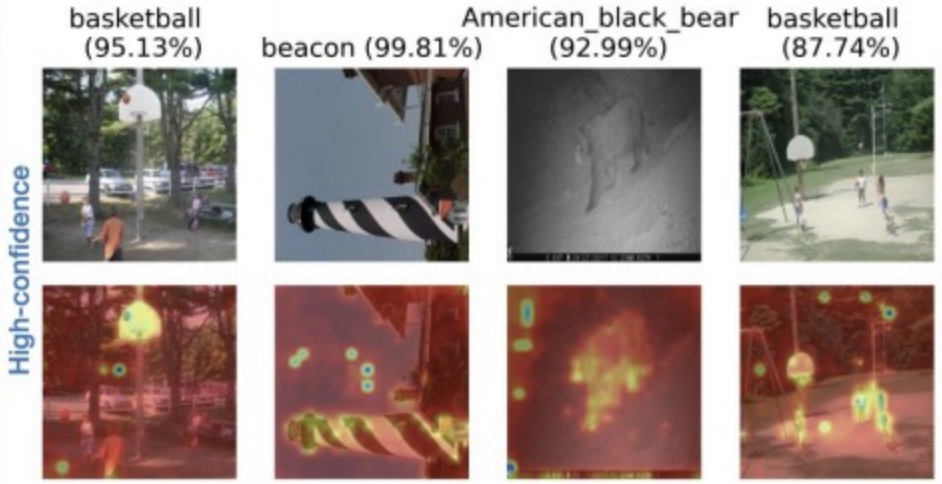
$$M^c_{h,w} = ReLU\left(\sum_k \alpha_k A_{h,w,k}\right)$$



Selvaraju et al, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", CVPR 2017

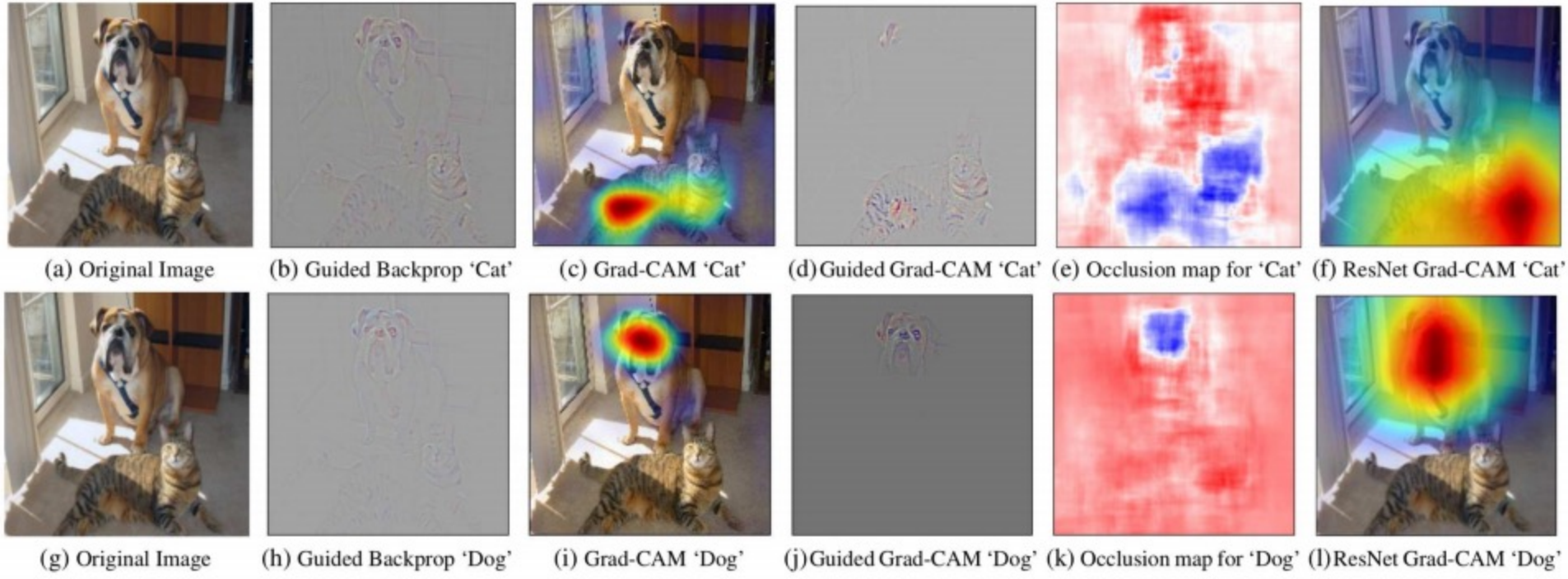
Visualizing ViT features





Chen et al., When Vision Transformers OutperformResnets Without Pre-training Or Strong Data Augmentations, ICLR 2022;
Paul and Chen, Vision Transformers are Robust Learners, AAAI 2022. Reproduced for educational purposes.

Gradient-Weighted Class Activation Mapping (Grad-CAM)



(a) Original Image (b) Guided Backprop 'Cat' (c) Grad-CAM 'Cat' (d) Guided Grad-CAM 'Cat' (e) Occlusion map for 'Cat' (f) ResNet Grad-CAM 'Cat'

(g) Original Image (h) Guided Backprop 'Dog' (i) Grad-CAM 'Dog' (j) Guided Grad-CAM 'Dog' (k) Occlusion map for 'Dog' (l) ResNet Grad-CAM 'Dog'



Selvaraju et al, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", CVPR 2017

Today

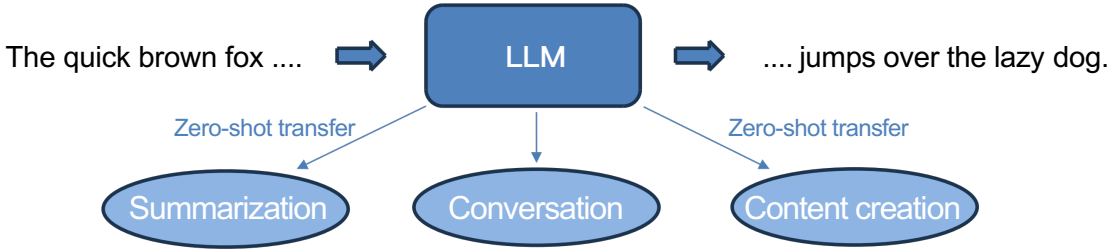
- Computer Vision Tasks
  - Semantic Segmentation
  - Object Detection
  - Instance Segmentation

- Visualization & Understanding
  - Model Layers Visualization
  - Saliency Maps
  - CAM & Grad-CAM

- **Foundation Models and Prompt Segmentation**

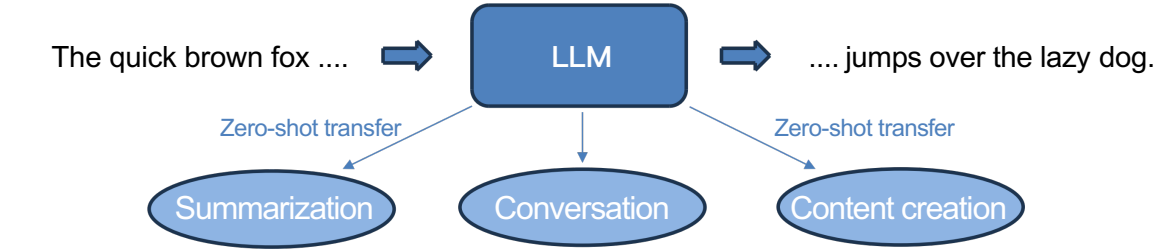


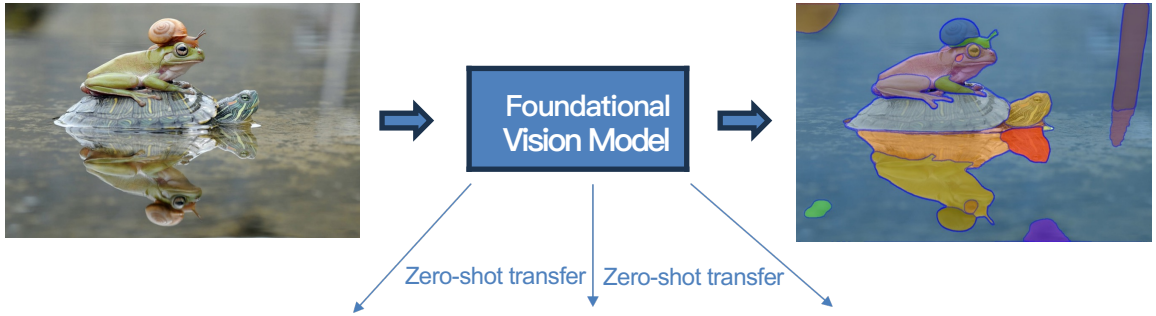
Prompting for Zero-Shot Transfer in NLP



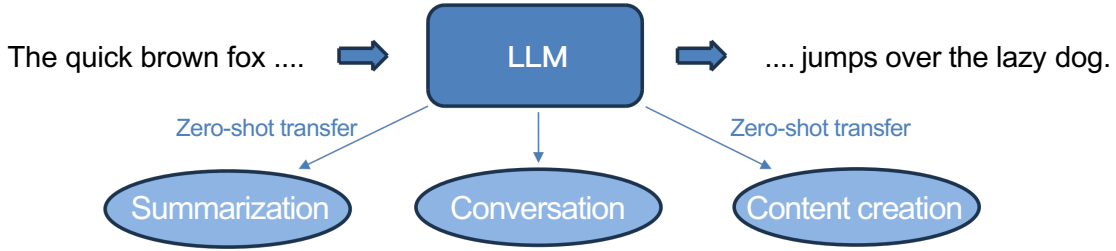


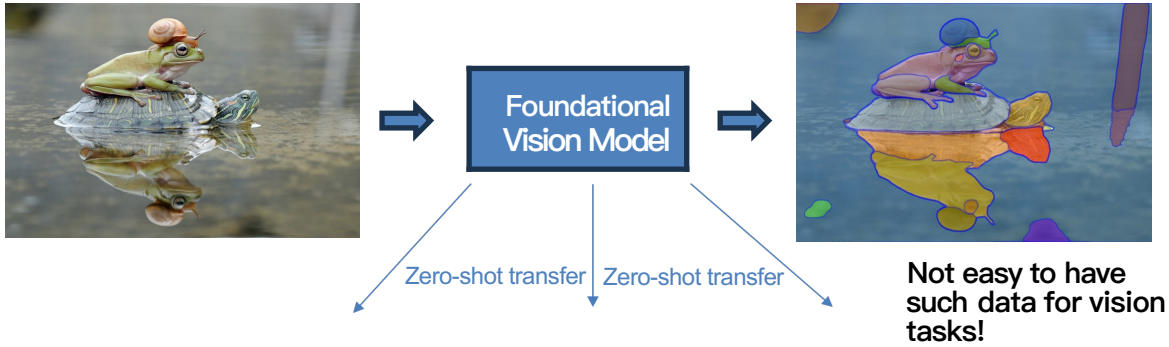
Foundational Vision Model



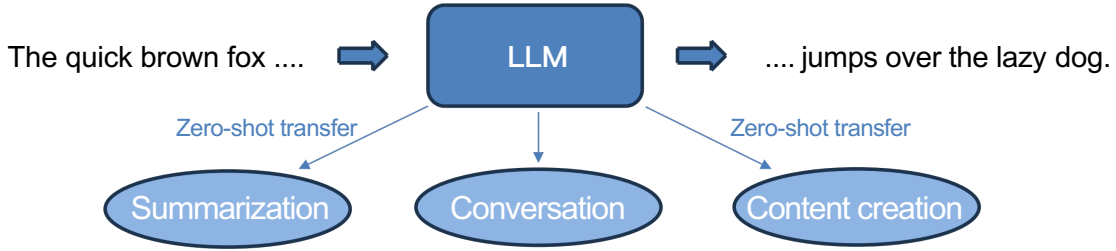


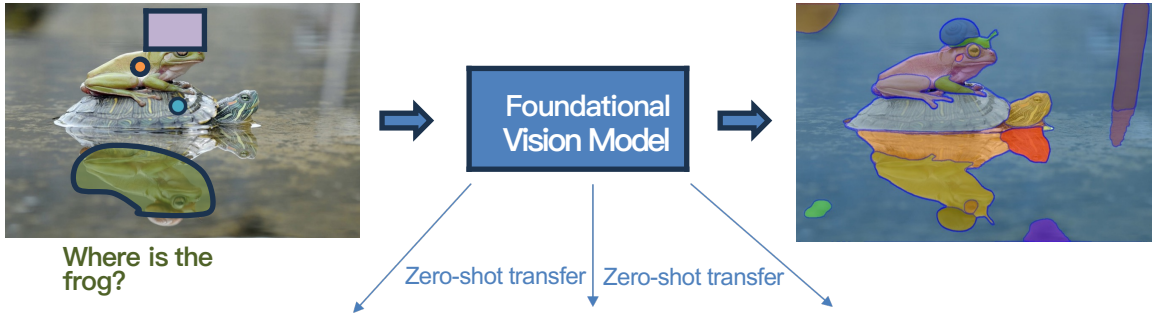
Foundational Vision Model: Data Paucity



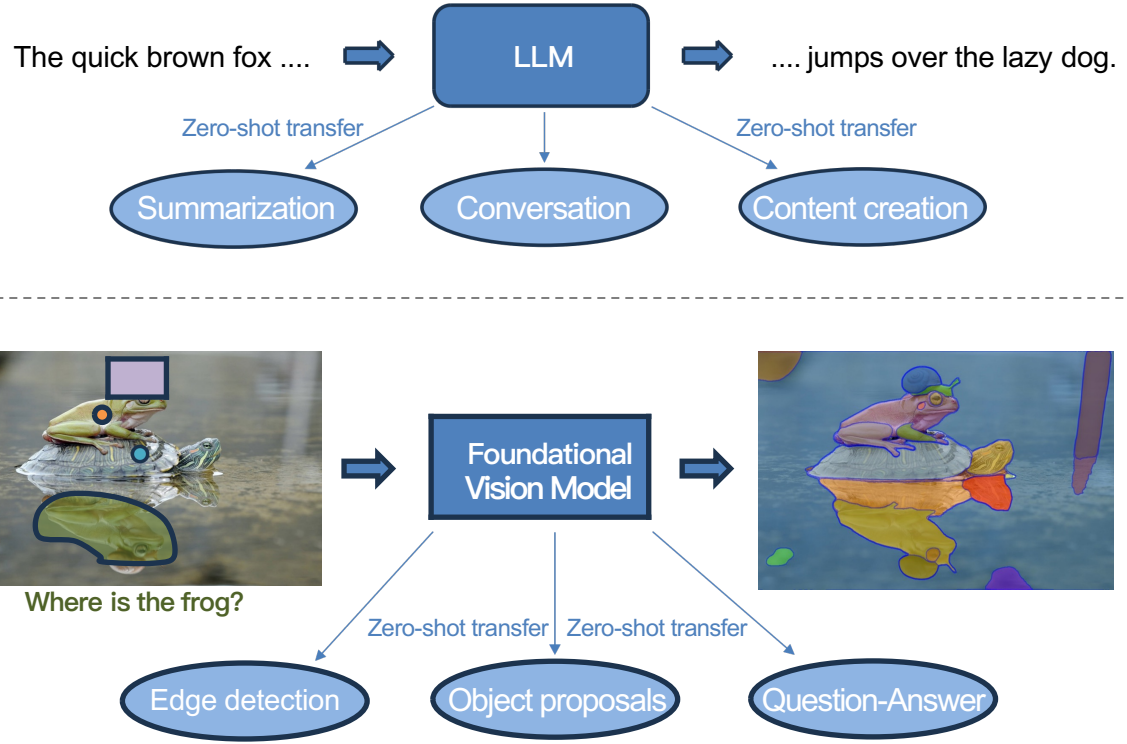


Foundational Vision Model: Promptable Segmentation



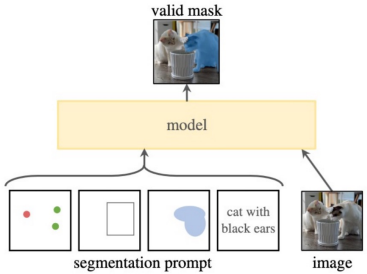


Foundational Vision Model: Zero-Shot Transfer

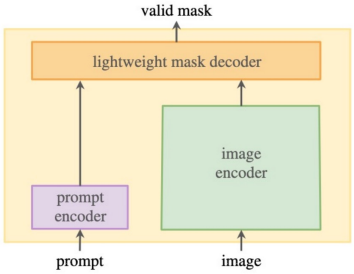


Segment Anything (ViT Based)

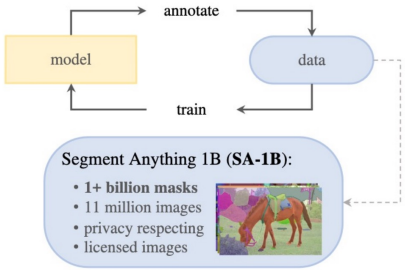
Task: Allow for interactive and automatic use, with zero-shot generalization and re-use



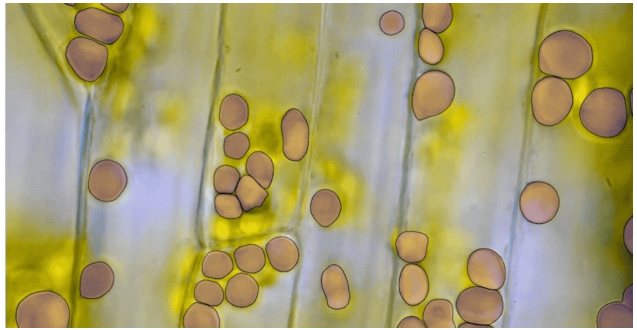
Model: Allow flexible prompts, support real-time usage, be ambiguity-aware



Data engine: Model-in-the-loop dataset annotation, with equity and privacy considerations







Segment Anything: Task

Promptable segmentation

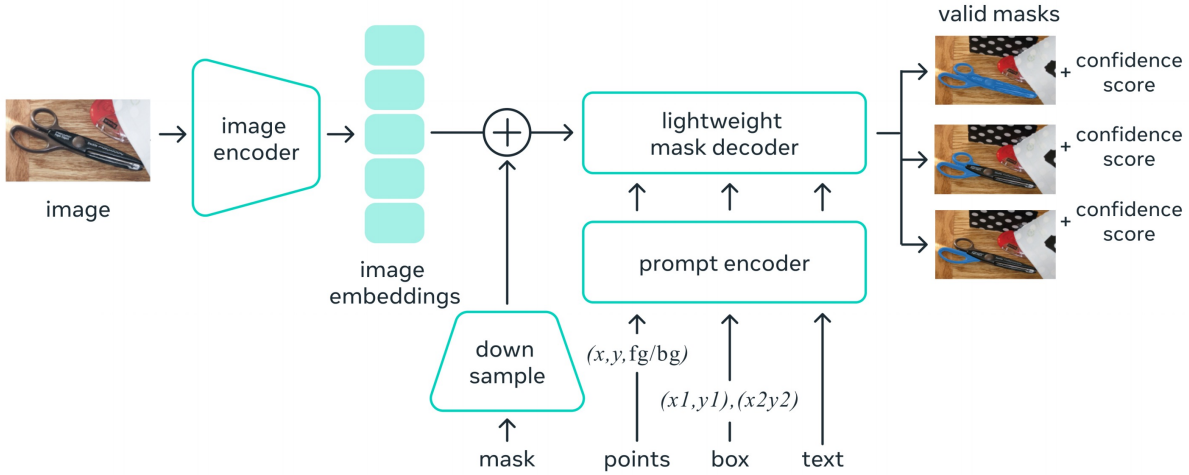
- Generate a *valid* mask for a given *prompt* as input
- Prompts might be *ambiguous*, but always return a valid mask

Allows large-scale model-in-the-loop data annotation

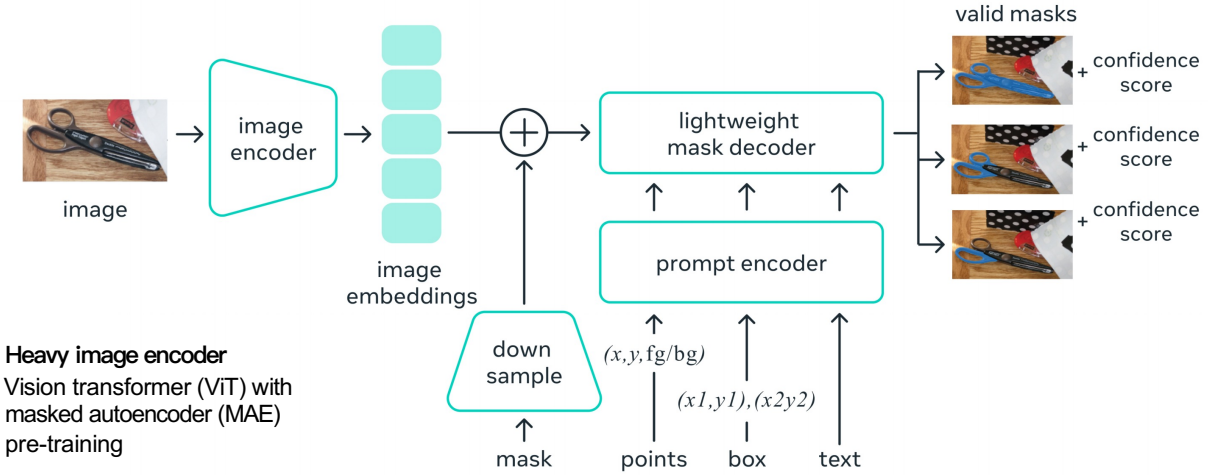
Allows zero-shot transfer similar to prompt engineering



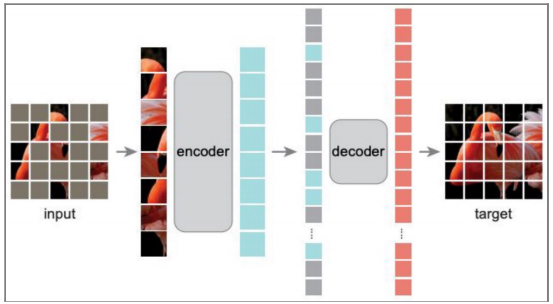
Segment Anything: Universal Segmentation Model



Segment Anything: Universal Segmentation Model

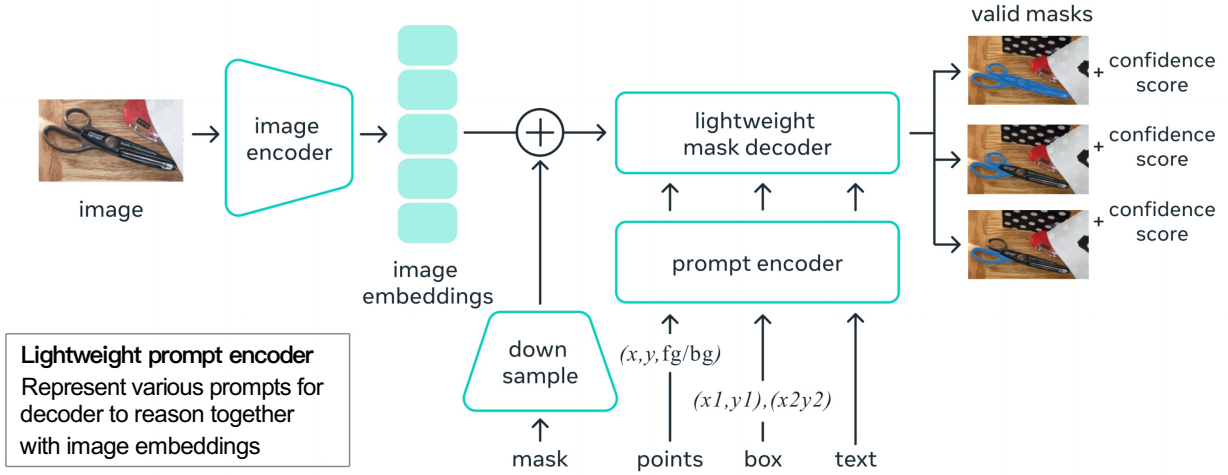


Heavy image encoder
Vision transformer (ViT) with masked autoencoder (MAE) pre-training



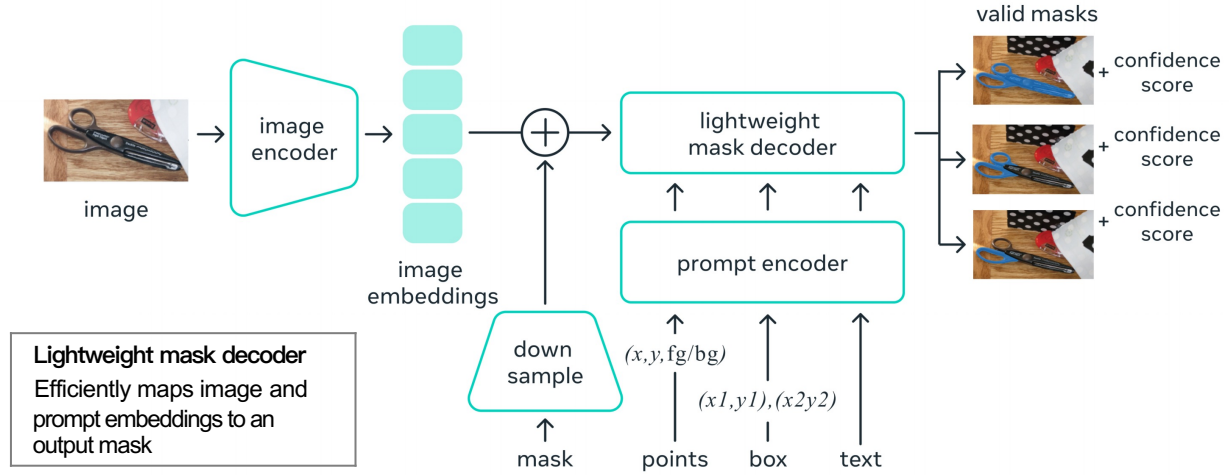
- Successful in NLP, but special considerations for vision
- **Information:** neighbor patches reconstruct more easily
- *Encoder* processes only 25% patches (75% masking)
- **Semantic level:** reconstructing pixels different from words
- Strong *decoder* (just MLP for text), but lighter than encoder
- Process encoded patches and mask tokens to predict pixels
- Scalable feature learning in high capacity networks

Segment Anything: Universal Segmentation Model

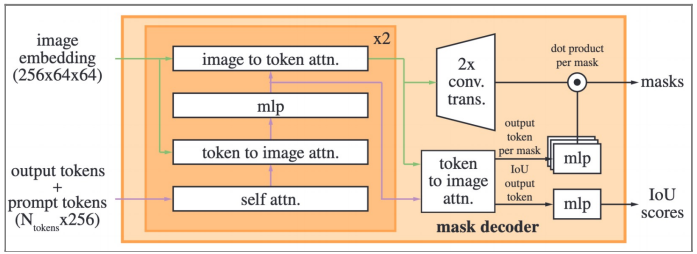


- **Point:** Positional encoding of point location + learnable encoding for foreground or background
- **Box:** Position encoding of corner + learnable encoding of which corner
- **Text:** CLIP text encoder
- **Mask:** Convolutions and element-wise sum with image embeddings

Segment Anything: Universal Segmentation Model



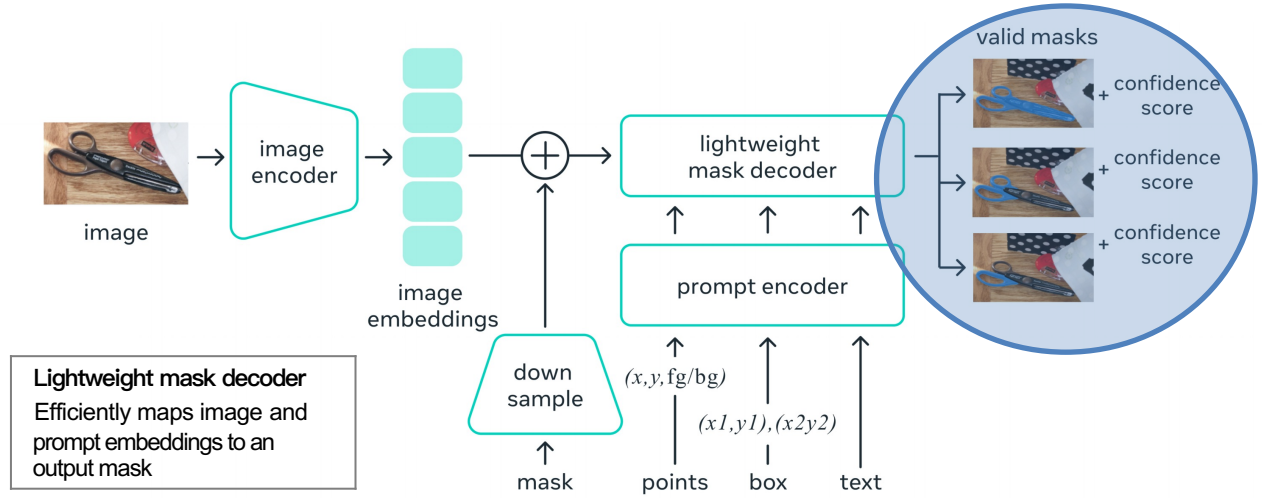
Lightweight mask decoder
Efficiently maps image and prompt embeddings to an output mask



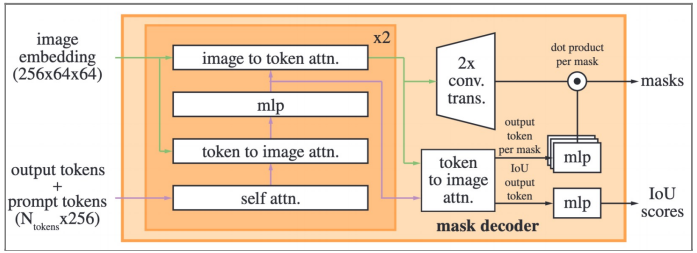
Two layers of transformer decoders
- Self-attention on the prompts
- Cross-attention from query prompts to image
- Pointwise MLP for non-linearity
- Cross-attention from query image to prompts

Transposed convolution to upsample to mask

Segment Anything: Universal Segmentation Model



Lightweight mask decoder
Efficiently maps image and prompt embeddings to an output mask



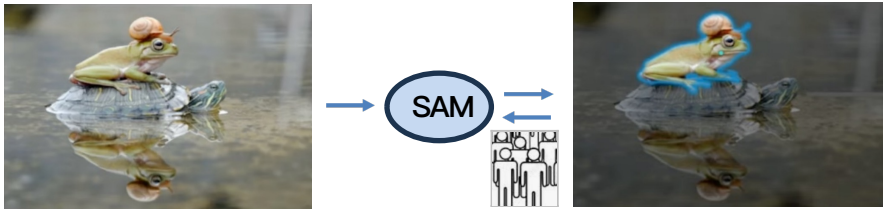
Predict three mask outputs

- Suffices for whole, part, sub-part concepts
- An output token for each mask predicted
- Backpropagate only minimum loss over masks
- Token for IoU prediction to rank masks

Segment Anything: Model-in-the-Loop Data Engine

Assisted manual stage

- Start from datasets
- Iteratively retrain model with several rounds of human correction

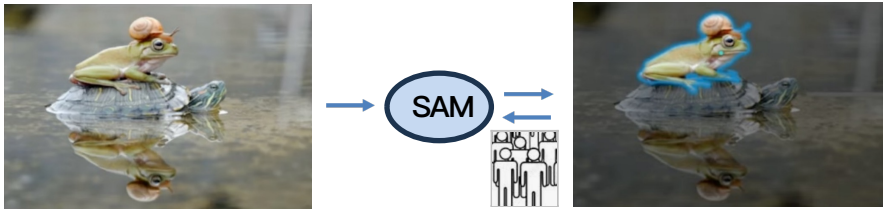


120k images, 4.3M masks

Segment Anything: Model-in-the-Loop Data Engine

Assisted manual stage

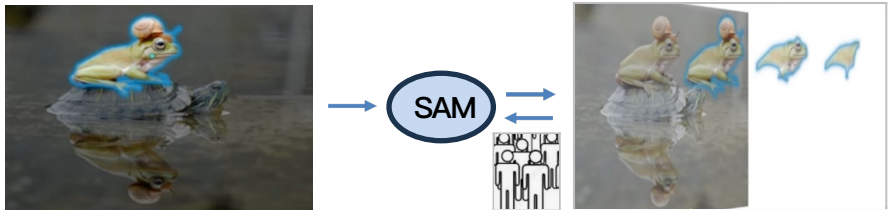
- Start from datasets
- Iteratively retrain model with several rounds of human correction



120k images, 4.3M masks

Semi–automatic stage

- Improve diversity with human refinement of current outputs
- Periodically retrain model

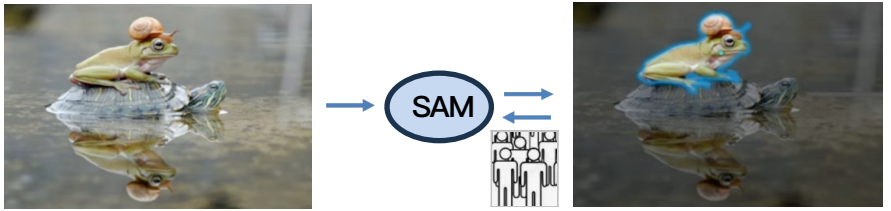


180k images, 5.9M masks

Segment Anything: Model-in-the-Loop Data Engine

Assisted manual stage

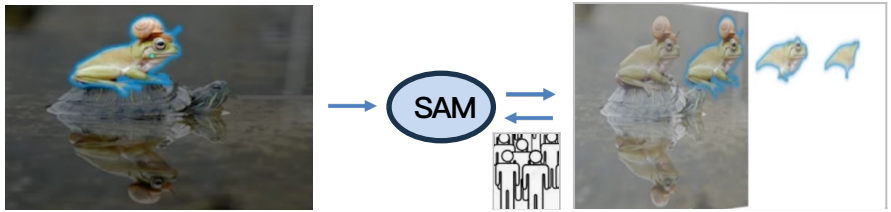
- Start from datasets
- Iteratively retrain model with several rounds of human correction



120k images, 4.3M masks

Semi–automatic stage

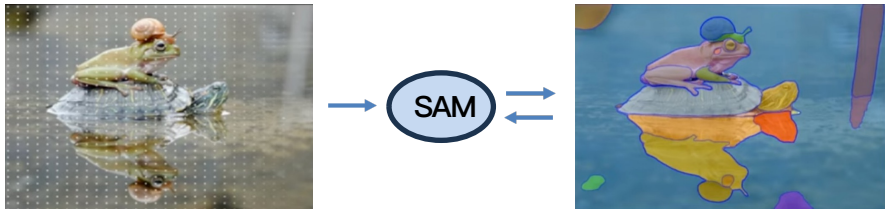
- Improve diversity with human refinement of current outputs
- Periodically retrain model



180k images, 5.9M masks

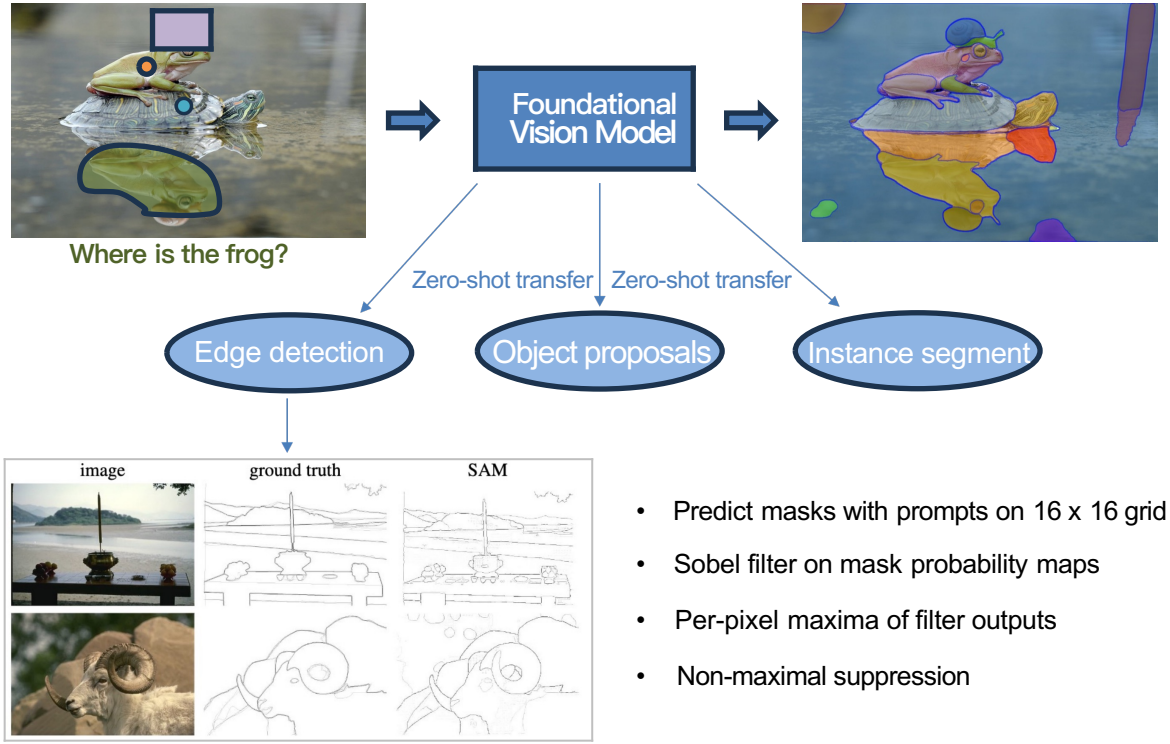
Fully automatic stage

- Already have trained an ambiguity-aware model
- Introduce prompting with 32 x 32 grid of points
- Keep high confidence masks (IoU prediction), do NMS



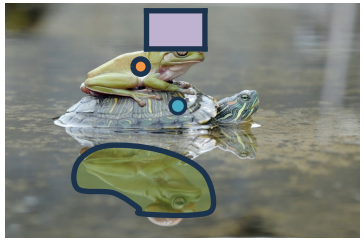
11M images, 1.1B masks

Segment Anything: Prompting for Zero-Shot Transfer



- Predict masks with prompts on 16 x 16 grid
- Sobel filter on mask probability maps
- Per-pixel maxima of filter outputs
- Non-maximal suppression

Segment Anything: Prompting for Zero-Shot Transfer



Where is the frog?





Zero-shot transfer Zero-shot transfer

Edge detection Object proposals Instance segment

- Bounding boxes from object detector
- Use as prompts for SAM to get mask
- Refine most confident mask by feeding back to decoder with box prompt

